A Unified Random Walk, Its Induced Laplacians and Spectral Convolutions for Deep Hypergraph Learning

Jiying Zhang, Fuyang Li, Xi Xiao, Guanzi Chen, Tingyang Xu, Yu Rong, Junzhou Huang, Yatao Bian

Abstract—Hypergraph-based modeling has gained significant attention for capturing complex higher-order interactions among vertices. While random walks serve as fundamental tools for analyzing hypergraphs, existing approaches either fail to fully leverage edge-dependent vertex weights (EDVWs) or lack sufficient expressiveness to model intricate hypergraph structures. To address these limitations, we propose a unified random walk framework that integrates hyperedge degrees and vertex weights, offering a more robust approach to hypergraph modeling. We establish equivalence conditions between hypergraph and graph random walks, leading to a novel unified random-walk-based hypergraph Laplacian that incorporates EDVWs, ensuring expressiveness and desirable spectral properties. Building on this foundation, we introduce the General Hypergraph Spectral Convolution (GHSC) framework, which extends existing Graph Convolutional Neural Networks (GCNNs) for effective hypergraph learning, supporting both edge-independent and edge-dependent vertex weights. Extensive experiments across diverse datasets, including citation networks, visual objects, and protein modeling tasks, demonstrate state-of-the-art performance, with notable improvements in protein structure modeling using EDVW-hypergraphs. This work advances the theoretical understanding of hypergraph random walks and spectral theory while providing a versatile framework for deep hypergraph learning. Code is available at GHSC_H_GNNs.

Index Terms—Hypergraph Learning, Hypergraph Random Walks, Hypergraph Laplaican, Equivalence

1 Introduction

Graphs are fundamental abstract structures used to represent relationships and features across diverse domains, such as social and biological networks [1], [2], [3]. However, conventional graphs are inherently limited to pairwise relationships, making them insufficient for capturing higher-order interactions among entities. For example, in biology, protein relationships often require modeling beyond pairwise interactions to accurately represent the complex connections among multiple proteins [4], [5], [6]. Similarly, the intricate tertiary structure of proteins reveals higher-order interactions between amino acids that cannot be effectively represented by pairwise models [7]. To overcome these limitations, hypergraphs, which allow hyperedges to connect arbitrary numbers of vertices, have emerged as a powerful paradigm for modeling complex higher-order interactions. Hypergraphs have gained significant attention for real-world applications, including citation networks analysis [8], [9], text classification [10], and visual object classification [11], etc.

Random walks on hypergraphs are a widely used approach for understanding how to extract information from complex systems with high-order interactions. The first theoretical work can probably be traced back to Zhou et al. [12], who proposed a two-step procedure to describe random walks on hypergraphs (Fig. 2). Following this work, many researchers have attempted to design more comprehensive random walk models on hypergraphs [13], [14], [15]. However, these existing random walk methods do not fully utilize edge-dependent vertex weights (EDVWs) – where each vertex v is assigned a weight $q_e(v) \in \mathbb{R}$ for each incident hyperedge e – and do not sufficiently consider the influence of the degree of hyperedges on the random walk. To address these gaps, we propose a unified two-step random walk framework for hypergraphs that incorporates both hyperedge degrees and vertex weights at each step, allowing for distinct vertex weights to be applied in the first and second steps. Notably, our framework generalizes existing hypergraph random walk variants [13], [14], [15], which can be viewed as specific instances of our approach.

To deepen the understanding of this unified random walk, we examine its equivalence to graph-based random walks and derive a novel hypergraph Laplacian. Specifically, we establish that 1) random walks on hypergraphs are always equivalent to the random walks on the digraphs (Thm. 1), and 2) under certain conditions, hypergraph random walks are equivalent to the random walks

- *J. Zhang is with the EPFL, Switzerland, and International Digital Economy Academy, China. E-mail: jiyingzh88@gmail.com;*
- *F. Li is with the Tsinghua University, and Marketing Department of China Huaneng Group., Ltd. E-mail: fuyang419@163.com;*
- *Y. Bian is with the Department of Computer Science, National University of Singapore, Singapore. E-mail: bianyt@comp.nus.edu.sg;*
- *X. Xiao is with the Shenzhen International Graduate School, Tsinghua University, China. E-mail: xiaox@sz.tsinghua.edu.cn*
- *G. Chen is with Tsinghua University, China. E-mail: guanzichen99@gmail.com*
- *T. Xu, Y. Rong are with DAMO Academy, Alibaba Group, and Hupan Lab, Hangzhou, China. E-mail: xuty_007@hotmail.com; yu.rong@hotmail.com*
- *J. Huang is with The University of Texas at Arlington, USA. E-mail: jzhuang@uta.edu*
- *Correspondence to Yatao Bian and Xi Xiao.*

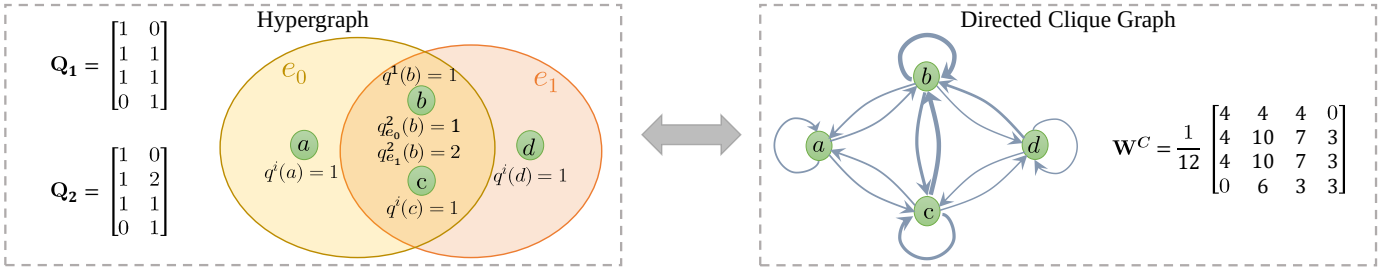


Fig. 1: An example of hypergraph and its equivalent weighted directed graph, where $q^i(\cdot) = Q_i(\cdot, e), i \in \{1, 2\}$. Here, $\mathbf{Q}_1$ is edge-independent and $\mathbf{Q}_2$ is edge-dependent. The characteristics of the hypergraph are encoded in the weighted incidence matrices (i.e. vertex weights) $\mathbf{Q}_1, \mathbf{Q}_2$. $\mathbf{W}^C := \mathbf{Q}_1 \mathbf{D}_e^{-1} \mathbf{Q}_2^\top$ denotes the edge-weight matrix of the clique graph and can be viewed as the embedding of high-order relationships.

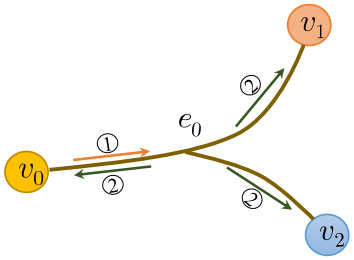


Fig. 2: Hypergraph Random Walk. ① denotes the first step and ② represents the second.

on undirected graphs (Thm. 2). Based on the equivalence with the digraph, we derive a new hypergraph Laplacian of our unified random walk. It is expressive, yet enjoys a simple form similar to the graph Laplacian of [16], however, they use very different transition probabilities and serve for different scenarios (hypergraphs vs. graphs). We prove that this Laplacian is well-defined (semi-definite) and adheres to common spectral properties. Additionally, we provide an explicit expression for the Laplacian matrix, drawing on the equivalence between hypergraphs and undirected graphs.

We further propose the General Hypergraph Spectral Convolution (GHSC) framework to validate the practical utility of the random walk equivalence. This versatile framework not only supports both edge-dependent and edge-independent vertex weights (EDVWs and EIVWs) but also enables the direct and theoretical utilization of existing powerful Graph Convolutional Neural Networks (GCNNs). Using both EDVW and EIVW allows the framework to generalize across applications. By replacing the graph Laplacian in GCNNs (e.g., GCN [17] and GCNII [18]) with our unified Laplacian, GHSC provides a seamless extension for effective hypergraph learning and supports compatibility with a wide range of spectral methods, significantly simplifying the design of Hypergraph Neural Networks (HNNs).

Experiments across diverse tasks highlight the robustness and effectiveness of our framework. Notably, H-GCNII consistently outperforms state-of-the-art (SOTA) methods in citation network node classification, particularly surpassing prior spectral-based approaches [11]. Additionally, our framework achieves SOTA performance on the visual classification benchmark ModelNet40. Furthermore, in protein representation learning tasks, including Fold Classification and Quality Assessment, our hypergraph-based methods demonstrate clear advantages over both sequence-based [19] and simple-graph-based [20] approaches.

The main contributions of this work are as follows:

1. We introduce a unified framework for random walks on hypergraphs that incorporates the influence of vertex weights and hyperedge degrees. This framework includes the definition of a generalized hypergraph that effectively captures edge-dependent vertex weights (EDVWs) through random walks.

2. We analyze the equivalence between hypergraph and graph random walks and derive a unified random-walk-based Laplacian. Our study shows that random walks on hypergraphs are always equivalent to those on directed graphs. Additionally, we identify two sufficient conditions under which hypergraphs can be reduced to undirected graphs and further explore the spectral properties of the resulting hypergraph Laplacian.

3. We present a General Hypergraph Spectral Convolution framework (GHSC) that extends existing graph neural networks (GCNNs) to hypergraph learning. This is achieved by replacing the graph Laplacian with the unified hypergraph Laplacian, enabling effective learning for both edge-independent and edge-dependent vertex weight hypergraphs.

4. We conduct extensive experiments across various domains, including social network analysis, visual object classification, and protein classification. The results demonstrate the generalization and effectiveness of our framework for both edge-independent and edge-dependent hypergraph learning tasks. In particular, the application of EDVW-hypergraphs to protein structure modeling yields significant performance improvements.

This paper is organized as follows. Section 2 reviews the related works. Section 3 describes the definition of unified random walks, which is a generalization of previous work. We also induce a generalized hypergraph associated with the random walks. Section 4 provide the equivalence conditions between hypergraphs and graphs, including undirected graphs and directed graphs. Section 5 is devoted to formulation and theoretical analyses of hypergraph Laplacian. Section 6 gives the GHSC framework for deep hypergraph learning. Section 7 shows experiments on various real-world tasks and show how unified Laplacian can help improve learning performances. We then conclude the paper. The paper is partly based on the previous work [21].

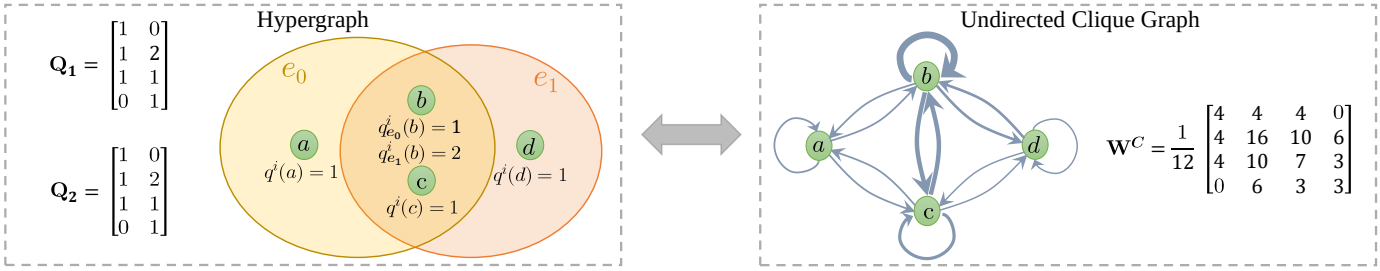


Fig. 3: An example of hypergraph and its equivalent weighted undirected graph ($\mathbf{W}^C = (\mathbf{W}^C)^\top$), where $q^i(\cdot) = Q_i(\cdot, e), i \in \{1, 2\}$. Here, $\mathbf{Q}_1 = \mathbf{Q}_2$ are both edge-dependent. The characteristics of the hypergraph are encoded in the weighted incidence matrices (i.e. vertex weights) $\mathbf{Q}_1, \mathbf{Q}_2$. $\mathbf{W}^C := \mathbf{Q}_1 \mathbf{D}_e^{-1} \mathbf{Q}_2^\top$ denotes the edge-weight matrix of the clique graph and can be viewed as the embedding of high-order relationships.

2 Related Work

Deep Learning for Hypergraphs. Many hypergraph learning algorithms can essentially be viewed as redefining hypergraphs propagation schemes on equivalent clique graphs [22] or their variants [8], [9], [11], [23]. On the other hand, [24], [25] and [26] propose message-passing and set function learning frameworks that cover the MPNN [27] paradigm. However, these frameworks fail to provide theoretical guarantees. Recently, based on the energy optimization, PhenomNN [28] and ED-HNN [29] proposes message passing neural networks for hypergraph learning. Nevertheless, they depend on the elaborate design of propagation schemes. It remains open in deep hypergraph learning are: (i) a framework that can handle EDVW-hypergraphs and (ii) a general theoretical framework that can directly utilize the build on the Laplacian of hypergraph.

Graphs Neural Networks. GNNs have received significant attention recently and many successful architectures have been raised for various graph learning tasks [17], [27], [30], [31]. Spectral-based graph convolutional network bridges the gap between spectral-based methods and spatial-based methods, and it is widely used in the applications of data mining and feature construction due to its solid foundation in graph signal processing [1], [32]. Spectral-based GNNs have a solid foundation in graph signal processing [33] and allow for simple model property analysis. There exist many powerful spectral-based models such as ChebNet [34], GCN [1], SSGC [35], GCNII [18], RevGNN [36], APPNP [37]. Most of these spectral graph convolutions can be viewed as derived from graph random walk-based Laplacians, which are important tools to represent graphs and have been well-studied. However, for hypergraph, there is still a lack of comprehensive spectral convolutions due to various unsolved problems regarding its random walk and Laplacian. So there is an urgent need to have a thorough study of random walks, Laplacians, and spectral convolutions for deep hypergraph learning.

Hypergraph Random Walk and Spectral Theory. In machine learning applications, Laplacian is an important tool to represent graphs or hypergraphs. While the random walk-based graph Laplacian has a well-studied spectral theory, the spectral methods of hypergraphs are surprisingly lagged. A two-step random walk Laplaciain was proposed by [12] for the general hypergraph and was improved by [13] and [14], [15]. On the other hand, [38] and [39] define a s-th Laplacian through random s-walks on hypergraphs. More recently, non-linear Laplacian, as a new important tool to represent hypergraphs, has been extended to several settings, including directed hypergraphs [40], [41], submodular hypergraphs [42], etc.

Equivalence Between Hypergraphs and Graphs. A fundamental problem in the spectral theoretical study of hypergraphs is the equivalency with undigraphs. So far, equivalence is established from two perspectives: the spectrum of Laplacian [22], [43] or random walk [13]. An important application of building up the equivalence between hypergraphs and graphs is to transform hypergraphs to clique graphs [12], [13], [22], [44].

Hypergraph with Edge-Dependent Vertex Weights. EDVW-hypergraphs have been widely adopted in machine learning applications, including 3D object classification [45], image segmentation [46], e-commerce [47], image search [48], hypergraph clustering [43], text ranking [49]. However, the design of deep learning algorithms for processing EDVW-hypergraphs remains an open question.

Due to the space limit, more discussions on related works are deferred to Appendix 9.

3 Unified Random Walk and Generalized Hypergraph

Here, we propose a novel unified random walk and its non-lazy version.

Notations. $\mathbf{I} \in \mathbb{R}^{n \times n}$ denotes the identity matrix, and $\mathbf{1} \in \mathbb{R}^n$ represents the vector with ones in each component. We use boldface letter $\mathbf{x} \in \mathbb{R}^n$ to indicate an n-dimensional vector, where $x(i)$ is the i^{th} entry of $\mathbf{x}$. We use a boldface capital letter $\mathbf{A} \in \mathbb{R}^{m \times n}$ to denote an m by n matrix and use $A(i, j)$ to denote its ij^{th} entry. Let $\mathcal{G}(\mathcal{V}, \mathcal{E}, \omega)$ be a graph with vertex set $\mathcal{V}$, edge set $\mathcal{E}$, and edge weights ω. Let $\mathcal{H}(\mathcal{V}, \mathcal{E}, \mathbf{W}, \mathbf{Q})$ be the hypergraph with vertex set $\mathcal{V}$, edge set $\mathcal{E}$. Let $w(e)$ denote the weight of hyperedge e. $\mathbf{W} \in \mathbb{R}^{|\mathcal{E}| \times |\mathcal{E}|}$ is the edge-weights diagonal matrix with entries $W(e, e) = w(e)$. $\mathbf{Q}$ denotes edge-vertex-weights matrix with entries $Q(u, e) = q_e(u) \in \mathbb{R}$ if $u \in e$ and 0 if $u \notin e$. $\mathbf{Q}$ is said to be *edge-independent* if $q_e(u) = q(u) \in \mathbb{R}$ for all $e \ni u$, and called *edge-dependent* otherwise [13]. If $q_e(u)$ equals to 1 for all linked u and e ,$\mathbf{Q}$ would reduce to the binary incident matrix $\mathbf{H}$, which is widely used to represent the structure of hypergraph [12], [15]. The matrix $\mathbf{Q}$ can also be viewed as a weighted incident matrix. Note that a graph is a special case of a hypergraph with hyperedge degree $|e| = 2$. We say a hypergraph is connected when its unweighted clique graph [13] is connected and we assume hypergraphs

are connected. Throughout this paper, equivalence refers to equivalence between hypergraphs and undigraphs if not otherwise specified.

3.1 Unified Random Walk on Hypergraphs

Hypergraph random walk is commonly defined by a two-step manner [12], [50] (Fig. 2). [13] raise a new random walk involving the edge-dependent vertex weights into the second step and build the equivalency between EDVW-hypergraphs and digraphs. However, due to the limitation of their definition of hypergraphs, They failed to answer whether the EDVW-hypergraphs can be equal to undigraphs. So in this part, we integrate the existing two-step random walk methods [13], [14], [15] to obtain a comprehensive *unified random walk* with the vertex weights added into the first step, based on which we further define a generalized hypergraph.

Definition 1 (Unified Random Walk on Hypergraphs). The unified random walk on a hypergraph $\mathcal{H}$ is defined in a two-step manner: Given the current vertex u,

Step I: choose an arbitrary hyperedge e incident to u, with the probability

$$p_1 = \frac{w(e)\sum_{v\in\mathcal{V}} Q_2(v,e)\rho(\sum_{v\in\mathcal{V}} Q_2(v,e))Q_1(u,e)}{\sum_{e\in\mathcal{E}} w(e)\sum_{v\in\mathcal{V}} Q_2(v,e)\rho(\sum_{v\in\mathcal{V}} Q_2(v,e))Q_1(u,e)}; \tag{1}$$

Step II: choose an arbitrary vertex v from e, with the probability

$$p_2 = \frac{Q_2(v,e)}{\sum_{v\in\mathcal{V}} Q_2(v,e)}, \tag{2}$$

Define $\delta(e) := \sum_{v\in\mathcal{V}} Q_2(v,e)$ as the degree of hyperedge, and define $d(v) := \sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(\delta(e))Q_1(v,e)$ as the degree of vertex v. Integrating the two steps, the transition probability of our unified random walk on a hypergraph from vertex u to vertex v is:

$$P(u,v) = \sum_{e\in\mathcal{E}} \frac{w(e)Q_1(u,e)Q_2(v,e)\rho(\delta(e))}{d(u)}. \tag{3}$$

Here, $w(e)$ denotes the hyperedge weight; $Q_1(u,e)$ denotes the contribution of hyperedge e to vertex u in the first step while $Q_2(v,e)$ represents the contribution of vertex v to hyperedge e in the second step. $\rho(\cdot)$ is a real-valued function that acts on the degree of the hyperedge and is used to control the random process. For example, we set $\rho = (\cdot)^\sigma$ (power function). For positive values of σ, the hyperedges with larger degrees will dominate the random process. Conversely, when σ is negative, hyperedges with small degrees are likely to drive the random walk process. Here, $P(u,v)$ can be written in a $|\mathcal{V}| \times |\mathcal{V}|$ matrix form: $\mathbf{P} = \mathbf{D}_v^{-1}\mathbf{Q}_1\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}_2^\top$, where $\mathbf{D}_v$, $\mathbf{D}_e$ are diagonal matrices with entries $D_v(v,v) = d(v)$ and $D_e(e,e) = \delta(e)$, respectively. $\rho(\mathbf{D}_e)$ represents the function ρ acting on each element of $\mathbf{D}_e$. It is worth noting that the ρ is possible to lose its effect and we formulate below.

Proposition 1. *$\rho(\cdot)$ fails to work in the unified random walk (Definition 1) if the hyperedge degrees are edge-independent (i.e. $\delta(e) = \delta(e'), \forall e' \in \mathcal{E}$).*

Notably, the k-uniform hypergraphs have the edge-independent hyperedge degree(i.e. $\delta(e) = k, \forall e \in \mathcal{E}$). We provide the proof in Appendix 14.5.

The two introduced vertex weights $\mathbf{Q}_1$ and $\mathbf{Q}_2$ bring the following benefits: i) Providing the theoretical basis to investigate the equivalency between EDVW-hypergraphs and undigraphs. The results show in Thm. 2 suggest that the equivalence depends on the relationship between these two $\mathbf{Q}$. ii) Modeling the fine-grained high-order information. The two matrices can be constructed heuristically to model the more fine-grained high-order information associated with the first step and the second step of the random walk. $\mathbf{Q}_1 \neq \mathbf{Q_2}$ means that the contribution of v to edge e in the in-edge process is different from the out-edge process. Furthermore, the introduced two $\mathbf{Q}$ make the random walk able to capture the more comprehensive hypergraph information by establishing equivalency to digraphs (Thm. 1). More detailed intuition of our purpose to design $P(u,v)$ can be found in Appendix 10.2.

Notably, the existing hypergraph random walk [12], [13], [14], [15] can be seen as special cases with specific p_1, p_2 (see Appendix 10.3). The definition of the random walk is lazy since it allows self-loops ($P(v,v) > 0$). Based on the unified random walk, we define the generalized hypergraph as follows.

Definition 2 (Generalized Hypergraph). A generalized hypergraph is a hypergraph associated with the unified random walk in Definition 1, denoted as $\mathcal{H}(\mathcal{V},\mathcal{E},\mathbf{W},\mathbf{Q}_1,\mathbf{Q}_2)$. Here, $\mathbf{Q}_1$, $\mathbf{Q}_2$ are vertex weights matrices, each of which can be edge-independent or edge-dependent.

The generalized hypergraph is capable of carrying EDVWs and EIVWs information flexibly due to the two $\mathbf{Q}$ introduced, laying the foundation for building the equivalence theory.

3.2 Unified Non-Lazy Random Walk on Hypergraphs

Next, we introduce the non-lazy version of the unified random walks on hypergraphs for a comprehensive study of random walks on hypergraphs. We start by defining the two-step process of the non-lazy random walk.

Definition 3 (Unified Non-lazy Random Walk on Hypergraphs). Suppose $\delta(e) := \sum_{v\in\mathcal{V}} Q_2(v,e)$ is the degree of hyperedge, and $d_{nl}(v) := \sum_{e\in\mathcal{E}} w(e)(\delta(e) - Q_2(v,e))\rho(\delta(e) - Q_2(v,e))Q_1(v,e)$ is the degree of vertex v in the non-lazy version. The unified non-lazy random walk on a hypergraph $\mathcal{H}$ is defined in a two-step manner: Given the current vertex u,

Step I: choose a hyperedge e incident to u, with probability

$$p_1 = \frac{w(e)(\delta(e) - Q_2(u,e))\rho(\delta(e) - Q_2(u,e))Q_1(u,e)}{d_{nl}(u)};$$

Step II: choose an arbitrary vertex $v \neq u$ from e, with probability

$$p_2 = \frac{Q_2(v,e)}{\delta(e) - Q_2(u,e)},$$

Thus, the transition probability of our unified non-lazy random walk from vertex u to v is

$$P_{nl}(u,v) = \begin{cases} \sum_{e\in\mathcal{E}} \frac{w(e)\rho(\delta(e)-Q_2(u,e))Q_1(u,e)Q_2(v,e)}{d_{nl}(u)} & \text{if } u \neq v, \\ 0 & \text{if } u = v \end{cases} \tag{4}$$

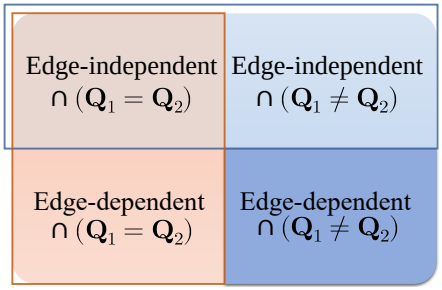


Fig. 4: The division of Equivalency problem (undigraph). The blue box indicates condition (1) in Thm. 2 and red box indicates condition (2). The dark blue part remains open yet.

which can be written in matrix form $\mathbf{P}_{nl} = \mathbf{D}_{nl}^{-1}(\mathbf{Q}_1 \odot \rho(\mathbf{1}\mathbf{D}_e - \mathbf{Q}_2))\mathbf{W}\mathbf{Q}_2^\top$, where $\mathbf{D}_{nl}$, $\mathbf{D}_e$ are diagonal matrices with entries $D_{nl}(v,v) = d_{nl}(v)$ and $D_e(e,e) = \delta(e)$, respectively. $\odot$ denotes the Hadamard product and $\mathbf{1}$ denotes a $|\mathcal{V}| \times |\mathcal{E}|$ ones matrix(i.e. all elements equal to 1). $\rho(\mathbf{1}\mathbf{D}_e - \mathbf{Q}_2))$ represents the function ρ acting on each element of $(\mathbf{1}\mathbf{D}_e - \mathbf{Q}_2)$.

Obviously, it is hard to factor P_{nl}, so in this paper, we focus on lazy random walks. If not specifically stated, all references to random walks in this paper refer to lazy random.

4 The Equivalency Between Hypergraphs and Graphs

In this part, we would like to illustrate that our unified random walk on a hypergraph is always equivalent to a random walk on the weighted directed clique graph and provide the condition under which the hypergraph is equivalent to a weighted undirected clique graph. As long as the equivalent conditions are established, the graphs can be viewed as the low-order encoders of hypergraphs, and many machine-learning methods of graphs (directed or undirected) could be easily generalized to hypergraphs. Especially, this makes it possible to utilize undigraphs as low-order encoders of hypergraphs thus enabling hypergraph learning directly from the equivalent undirected graphs by means of undigraph convolutional networks, that is, any undigraph technique can be used as a subroutine for hypergraph learning.

The clique graph of $\mathcal{H}(\mathcal{V},\mathcal{E},\mathbf{W},\mathbf{Q}_1,\mathbf{Q}_2)$ is denoted as $\mathcal{G}^C$, which is a unweighted graph with vertex set $\mathcal{V}$ and edge set $\{(u,v) : u,v \in e, e \in \mathcal{E}\}$. Actually, $\mathcal{G}^C$ turns all hyperedges into cliques. Similarly, the modified version of the clique graph without self-loops is described as follows. The clique graph of the hypergraph $\mathcal{H}(\mathcal{V},\mathcal{E},\mathbf{W},\mathbf{Q}_1,\mathbf{Q}_2)$ associated with the non-lazy random walk in definition 3 denotes as $\mathcal{G}_{nl}^C$, which is a weighted, undirected non-self-loop graph with vertex set $\mathcal{V}$ and edge set $\mathcal{E}' = \{(v,w) \in \mathcal{V}\times\mathcal{V} : v,w \in e \text{ for some } e \in \mathcal{E}, \text{ and } v \neq w\}$.

We first present the definition of equivalency in connection with Markov chains.

Definition 4. ([13]) Let M_1, M_2 be the Markov chains with the same finite state space, and let $\mathbf{P}^{M_1}$ and $\mathbf{P}^{M_2}$ be their probability transition matrices, respectively. M_1, M_2 are equivalent if $\mathbf{P}_{u,v}^{M_1} = \mathbf{P}_{u,v}^{M_2}$ for all states u and v.

Chitra & Raphael [13], whose random walk is a special case of our framework with $\rho(\cdot) = (\cdot)^{-1}, \mathbf{Q}_1 = \mathbf{H}$, claims that the hypergraph with edge-dependent weights (i.e. $\mathbf{Q}_2$ is edge-dependent) is equivalent to a directed graph. Here we extend this conclusion to a more general version of the above definition.

Theorem 1 (Equivalency between generalized hypergraph and weighted digraph). Let $\mathcal{H}(\mathcal{V},\mathcal{E},\mathbf{W},\mathbf{Q}_1,\mathbf{Q}_2)$ denote the generalized hypergraph in Definition 2. There exists a weighted directed clique graph $\mathcal{G}^C$ such that the random walk on $\mathcal{H}$ is equivalent to a random walk on $\mathcal{G}^C$. Moreover, the weighted matrix of $\mathcal{G}^C$ is $\mathbf{Q}_1\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}_2^\top$.

Thm. 1 indicates that any hypergraph is equivalent to a digraph under our unified random walks (Fig. 1) The proof can be found in Appendix 14.1). Similarly, when the random walk on hypergraphs is non-lazy, we can also obtain the conclusion like Thm. 1 with an only difference that $\mathcal{G}^C$ has no self-loops. Meanwhile, studying the equivalency with undigraph is also vital, since many graph learning algorithms are based on undigraphs. Next, we introduce the condition under which random walks on hypergraphs are equivalent to undigraphs. First, we give one of the implicit equations for formulating the equivalency problem to explore the explicit equivalency conditions.

Lemma 1. Let $\mathcal{H}(\mathcal{V},\mathcal{E},\mathbf{W},\mathbf{Q}_1,\mathbf{Q}_2)$ denote the generalized hypergraph in Definition 2. Let $\mathcal{F}_{(Q_1,Q_2)}(u,v) := \sum_{e\in\mathcal{E}} w(e)\rho(\delta(e))Q_1(u,e)Q_2(v,e)$ and $\mathcal{T}_{(Q)}(u) := \sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(\delta(e))Q(u,e)$. When $\mathbf{Q}_1, \mathbf{Q}_2$ satisfies the following equation

$$\begin{aligned}&\mathcal{T}_{(Q_2)}(u)\mathcal{T}_{(Q_1)}(v)\mathcal{F}_{(Q_1,Q_2)}(u,v)\\ &= \mathcal{T}_{(Q_2)}(v)\mathcal{T}_{(Q_1)}(u)\mathcal{F}_{(Q_1,Q_2)}(v,u), \forall u,v \in \mathcal{V}\end{aligned} \quad (5)$$

there exists a weighted undirected clique graph $\mathcal{G}^C$ such that a random walk on $\mathcal{H}$ is equivalent to a random walk on $\mathcal{G}^C$ with edge weights $\omega(u,v) = \mathcal{T}_{(Q_2)}(u)\mathcal{F}_{(Q_1,Q_2)}(u,v)/\mathcal{T}_{(Q_1)}(u)$ if $\mathcal{T}_{(Q_1)}(u) \neq 0$ and 0 otherwise.

This Lemma suggests a direction to explore the equivalency condition and we provide the proof in the Appendix, based on the property of random walk known as *time-reversibility* (definition 6 in appendix). Any generalized hypergraph composed of the vertex weights $\mathbf{Q}_1$ and $\mathbf{Q}_2$ satisfying Eq. (5) is equivalent to the undirected graph. Next, we derive the explicit condition under which random walks on hypergraphs are equivalent to undigraphs.

Theorem 2 (Equivalency between generalized hypergraph and weighted undigraph). Let $\mathcal{H}(\mathcal{V},\mathcal{E},\mathbf{W},\mathbf{Q}_1,\mathbf{Q}_2)$ denote the generalized hypergraph in Definition 2. When $\mathcal{H}$ satisfies any of the conditions below:
Condition (1): $\mathbf{Q}_1$ and $\mathbf{Q}_2$ are both edge-independent;
Condition (2): $\mathbf{Q}_1 = k\mathbf{Q}_2$ $(k \in \mathbb{R})$,
there exists a weighted undirected clique graph $\mathcal{G}^C$ such that a random walk on $\mathcal{H}$ is equivalent to a random walk on $\mathcal{G}^C$.

It is easy to verify that conditions (1) and (2) satisfy Eq. (5). This theorem brings insights from three aspects: i)

Hypergraph foundation. The equivalency itself is a fundamental problem and remains open before this work [13], [22]. Our conclusion provides more adaptive and explores more essential conditions thanks to the unity of our random walk and the generalization of hypergraph defined (Figure 4). Notably, [13] shows that the equivalency condition is: $\mathbf{Q}_1 = \mathbf{H}$ and $\mathbf{Q}_2$ is edge-independent, which can be viewed as a special case of the condition (1), while Theorem 2 implies the equivalency is not only related to the dependent relationships between vertex weights and edges but also to whether the vertex weights used in the first and second step of the unified random walk are proportional. Furthermore, thanks to the introduced $\mathbf{Q}_1$ in the generalized hypergraph, we can obtain the equivalency between EDVW-hypergraph and undigraph (i.e. condition (2)), filling an important gap in the equivalency theory of EDVW-hypergraph. ii) **Providing a theoretical basis for hypergraph applications.** The condition (2), containing both the edge-independent vertex weights and edge-dependent vertex weights cases that match different hypergraph applications [45], [46], [51], provides those with theoretical explanations. The condition (2) also reveals that the existing random walks [14], [15] on hypergraphs are equivalent to undigraph. iii) **Hypergraph learning.** The equivalent undigraphs can be viewed as the lower-order encoders of hypergraphs. Thus, one can obtain hypergraph representations directly by exploiting the undigraph learning methods. Especially, condition (2), which implies that an EDVW-hypergraph can be equivalent to an undigraph, gives the theoretical guarantee for learning hypergraphs without losing edge-dependent vertex weights via undigraph neural networks.

In fact, when condition (1) holds, $P(u,v)$ is equal to $\frac{w(e)Q_2(u,e)Q_2(v,e)\rho(\delta(e))}{\sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(\delta(e))Q_2(u,e)}$, which can be seen as a special case $P(u,v)$ of condition (2) (i.e. $\mathbf{Q}_1 = \mathbf{Q}_2$). Formally, we have

Corollary 2.1. Let $\mathcal{H}(\mathcal{V},\mathcal{E},\mathbf{W},\mathbf{Q}_1,\mathbf{Q}_2)$ denote the generalized hypergraph in Definition 2. Given a $\mathcal{H}$ satisfying condition (1) in Thm. 2, there exists a $\mathcal{H}'$ satisfying the condition (2) such that the random walk on $\mathcal{H}$ is equivalent to that on $\mathcal{H}'$.

A natural follow-up question is whether the conditions in Thm. 2 are necessary conditions, namely, whether random walks on any undigraph can be equivalent to random walks on some hypergraph $\mathcal{H}$ satisfying Thm. 2. The answer is negative and we formulated it as follows.

Theorem 3. There at least exists one generalized hypergraph $\mathcal{H}(\mathcal{V},\mathcal{E},\mathbf{W},\mathbf{Q}_1,\mathbf{Q}_2)$ (Definition 2) with edge-dependent weights and $\mathbf{Q}_1 \neq k\mathbf{Q}_2$, such that a random walk on $\mathcal{H}$ is not equivalent to a random walk on its undirected clique graph $\mathcal{G}^C$ for any choice of edge weights on $\mathcal{G}^C$.

This theorem states that the random walk on a generalized hypergraph can not be always equivalent to a random walk on an undigraph. Appendix 14.4 shows an example of Thm. 3. Thm. 3 opens up the possibility to construct a hypergraph that is not equivalent to an undigraph, inspiring researchers to further study higher-order interactions and other insurmountable bottlenecks on hypergraphs.

Different from the lazy random walk, a unified non-lazy random walk on a hypergraph with condition (1) or condition (2) in Thm. 2 is not guaranteed to satisfy *time-reversibility*. However, if $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$, then reversibility holds, and we obtain the result below.

Theorem 4. Let $\mathcal{H}(\mathcal{V},\mathcal{E},\mathbf{W},\mathbf{Q}_1,\mathbf{Q}_2)$ be a hypergraph associated with the unified non-lazy random walk in Definition 3. Let $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$, i.e. $Q_i(v,e) = 1, i \in \{1,2\}$ for all vertices v incident hyperedges e. There exist weights $\omega(u,v)$ on the clique graph without self-loops $\mathcal{G}^C_{nl}$ such that a non-lazy random walk on $\mathcal{H}$ is equivalent to a random walk on $\mathcal{G}^C_{nl}$.

The proof can be found in the Appendix. Next, we provide the spectral theory for the generalized hypergraphs.

5 Unified Hypergraph Laplacian and Its Spectral Properties

In this section, adopting the results of Thm. 1 and Thm. 2, we design the Laplacian for our unified random walk framework and explore its spectral properties. Thm. 1 says that the unified random walk framework is equivalent to a directed graph, so we extend the definition of directed graph Laplacian [16] to obtain the normalized Laplacian matrix as follows.

Definition 5 (Random Walk-based Hypergraph Laplacian). Let $\mathcal{H}(\mathcal{V},\mathcal{E},\mathbf{W},\mathbf{Q}_1,\mathbf{Q}_2)$ denote the generalized hypergraph in Definition 2. Let $\mathbf{P}$ be the transition matrix of our random walk framework on $\mathcal{H}$ with stationary distribution π (suppose that exists). Let $\mathbf{\Phi}$ be a $|\mathcal{V}| \times |\mathcal{V}|$ diagonal matrix with $\Phi(v,v) = \pi(v)$. The unified random walk-based hypergraph Laplacian matrix $\mathbf{L}_{rw}$ is defined as:

$$\mathbf{L}_{rw} = \mathbf{I} - \frac{\mathbf{\Phi}^{1/2}\mathbf{P}\mathbf{\Phi}^{-1/2} + \mathbf{\Phi}^{-1/2}\mathbf{P}^\top\mathbf{\Phi}^{1/2}}{2}. \quad (6)$$

Following [16], it is easy to study many properties based on the Rayleigh quotient of $\mathbf{L}_{rw}$, such as positive semi-definite, Cheeger inequality, etc. The Laplacian $\mathbf{L}_{rw}$ has great expressive power thanks to the comprehensive random walk, which unifies most of the existing random walk-based Laplacian variants due to the generalization ability of the framework (see Appendix 10.3). Note that the distribution π is not easy to obtain and does not admit a unitized expression that limits the application of $\mathbf{L}_{rw}$ in Eq. (6). In this paper, we focus on the case when hypergraph is equivalent to undigraph (Thm. 2) and deduce the explicit expression of $\mathbf{L}_{rw}$.

Recall Corollary 2.1, the unified random walk on $\mathcal{H}$ has the same transition probabilities as the unified random walk on $\mathcal{H}'$ if $\mathcal{H}$ satisfies condition (1) in Thm. 2 and $\mathcal{H}'$ satisfies condition (2). Thus, based on Eq. (6), we conclude that $\mathcal{H}$ and $\mathcal{H}'$ have the same stationary distribution and Laplacian matrix if the stationary distribution of the random walk is unique. Formally, we have

Corollary 4.1. Let $\mathcal{H}(\mathcal{V},\mathcal{E},\mathbf{W},\mathbf{Q}_1,\mathbf{Q}_2)$ be the generalized hypergraph in Definition 2. Let $\hat{\mathbf{D}}_v$ be a $|\mathcal{V}| \times |\mathcal{V}|$ diagonal matrix with entries $\hat{D}_v(v,v) := \hat{d}(v) := \sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(\delta(e))Q_2(v,e)$. No matter $\mathcal{H}$ satisfies condition (1) or condition (2) in Thm. 2, it obtains the

TABLE 1: Summary of classification accuracy(%) results. We report the average test accuracy and its standard deviation over 10 train-test splits.

Dataset	Architecture	Cora (co-authorship)	DBLP (co-authorship)	Cora (co-citation)	Pubmed (co-citation)	Citeseer (co-citation)
MLP+HLR	-	59.8±4.7	63.6±4.7	61.0±4.1	64.7±3.1	56.1±2.6
FastHyperGCN	spectral-based	61.1±8.2	68.1±9.6	61.3±10.3	65.7±11.1	56.2±8.1
HyperGCN	spectral-based	63.9±7.3	70.9±8.3	62.5±9.7	68.3±9.5	57.3±7.3
HGNN	spectral-based	63.2±3.1	68.1±9.6	70.9±2.9	66.8±3.7	56.7±3.8
HNHN	message-passing	64.0±2.4	84.4±0.3	41.6±3.1	41.9±4.7	33.6±2.1
HGAT	message-passing	65.4±1.5	85.83±0.3	52.2±3.5	46.3±0.5	38.3±1.5
HyperSAGE	message-passing	72.4±1.6	77.4±3.8	69.3±2.7	72.9±1.3	61.8±2.3
UniGNN	message-passing	75.3±1.2	88.8±0.2	70.1±1.4	74.4±1.0	63.6±1.3
ED-HNN [29]	energy optimization	73.3±1.0	89.1±0.19	68.4±1.3	71.6±1.8	62.4±1.6
PhenomNN [28]	energy optimization	77.1±0.5	89.8±0.05	73.1±0.65	78.1±0.24	65.8±0.45
H-ChebNet	spectral-based	70.6±2.1	87.9±0.24	69.7±2.0	74.3±1.5	63.5±1.3
H-APPNP	spectral-based	76.4±0.8	89.4±0.18	70.9±0.7	75.3±1.1	64.5±1.4
H-SSGC	spectral-based	72.0±1.2	88.6±0.16	68.8±2.1	74.5±1.3	60.5±1.7
H-GCN	spectral-based	74.8±0.9	89.0±0.19	69.5±2.0	75.4±1.2	62.7±1.2
H-GCNII	spectral-based	**80.5±2.01**	**90.6±0.30**	**77.4±2.4**	**78.4±0.8**	**69.7±1.8**

unified explicit form of stationary distribution π and Laplacian matrix $\mathbf{L}$ as:

$$\pi = \frac{\mathbf{1}^\top \hat{\mathbf{D}}_v}{\mathbf{1}^\top \hat{\mathbf{D}}_v \mathbf{1}} \text{ and } \mathbf{L} = \mathbf{I} - \hat{\mathbf{D}}_v^{-1/2} \mathbf{Q}_2 \mathbf{W} \rho(\mathbf{D}_e) \mathbf{Q}_2^\top \hat{\mathbf{D}}_v^{-1/2}. \tag{7}$$

There are two observations from Corollary 4.1: (i) This stationary distribution is different from the classical $\pi(v) = d(v)/\sum_v d(v)$ [12], which depends on the vertex degree $d(v)$. Our $\pi(v)$ is related to $\hat{d}(v)$, implying that we cannot trivially extend from classical theory. The detailed proof is deferred to Appendix 14.6; (ii) Both π and $\mathbf{L}$ are only related to $\mathbf{Q}_2$, independent of $\mathbf{Q}_1$. Meanwhile, $\mathbf{L}$ can be viewed as a normalized Laplacian led by the equivalent undigraph $\mathcal{G}^C$ with adjacency matrix $\mathbf{Q}_2 \mathbf{W} \rho(\mathbf{D}_e) \mathbf{Q}_2^\top$, based on which we can develop spectral convolutions for EDVW-hypergraphs. x Corollary 4.1 implies that when $\mathbf{Q}_1$, $\mathbf{Q}_2$ are both edge-independent or $\mathbf{Q}_1 = k\mathbf{Q}_2$, we can directly use the Laplacian matrix $\mathbf{L}$ to analyze the spectral properties. The spectral properties of Laplacian [52] are vital in the research of graph neural networks to design stable and effective convolution operators. Therefore, we deduce two important conclusions concerning eigenvalues of $\mathbf{L}$ as the basic theory of Laplacian application under the equivalency conditions in Thm. 2. One claims the eigenvalues range of $\mathbf{L}$ is $[0, 2]$ (details in Appendix 14.7), and the other describes the rate of convergence of our unified random walk listed below (see Appendix 14.8), which is important to analyze the property of our GHSC framework in section 6 (details in Appendix 13.2).

Corollary 4.2. Let $\mathcal{H}(\mathcal{V}, \mathcal{E}, \mathbf{W}, \mathbf{Q}_1, \mathbf{Q}_2)$ denote the generalized hypergraph in Definition 2. When $\mathcal{H}$ satisfies any of two conditions in Thm. 2, let $\mathbf{L}$ and π be the hypergraph Laplacian matrix and stationary distribution from Corollary 4.1. Let λ_H denote the smallest nonzero eigenvalue of $\mathbf{L}$. Assume an initial distribution $\mathbf{f}$ with $f(i) = 1$ ($f(j) = 0, \forall\, j \neq i$) which means the corresponding walk starts from vertex v_i. Let $\mathbf{p}^{(k)} = \mathbf{f}\mathbf{P}^k$ be the probability distribution after k steps unified random walk where $\mathbf{P}$ denotes the transition matrix, then $p^{(k)}(j)$ denotes the probability of finding the walker in vertex v_j after k steps. We have:

$$\left| p^{(k)}(j) - \pi(j) \right| \leq \sqrt{\frac{\hat{d}(j)}{\hat{d}(i)}} (1 - \lambda_H)^k. \tag{8}$$

Equivalence from Transform View. The equivalence can be viewed as transforming hypergraphs into undigraphs. Noted that conditions (1) and (2) both lead to the same graph $\mathcal{G}^C$ with weights $\mathbf{Q}_2 \mathbf{W} \rho(\mathbf{D}_e) \mathbf{Q}_2^\top$, indicating the transform is not an injection, that is, different hypergraphs are possibly mapped to the same graphs. Actually, GNNs are also not injective mappings between the graph universe and the representation space, since the expressive power of GNNs is limited by the 1-WL test [53], [54], [55]. Therefore, this property does not prevent the transform from being a powerful conversion tool suitable for downstream tasks. Indeed, one can design $\mathbf{Q}_1 = \mathbf{Q}_2$ in practice for alleviating the information loss caused by the transformation. In our experiments, we all follow the setting $\mathbf{Q}_1 = \mathbf{Q}_2$, under which the conversion from hypergraph $\mathcal{H}(\mathcal{V}, \mathcal{E}, \mathbf{W}, \mathbf{Q}_1, \mathbf{Q}_1)$ to undirected graph $\mathcal{G}^C$ is injective.

6 Hypergraphs Spectral Convolutions Derived from Undirected GCNNs

There exist several heuristic message-passing frameworks for EIVW-hypergraph learning [24], [25]. However, their frameworks require careful design and take no advantage of the existing graph learning algorithms. The heuristic design also makes the network difficult to analyze the properties of corresponding neural networks directly, such as over-smoothing issue [66]. We thus in this part aim to develop a general framework that can utilize existing GNNs to handle both EDVW and EIVW hypergraphs. We are devoted to spectral-based methods, which are considered to be robust and allow for simple model property analysis in simple graph learning [32].

From a random walk perspective, the focus in previous works [13] has been on transforming the EIVW-hypergraph and EDVW-hypergraph into respective undigraphs and digraphs to handle hypergraphs. Based on it, one can directly use the digraph Neural Networks to learn hypergraphs

TABLE 2: Test accuracy on visual object classification. Each model is ran with 10 random seeds and report the mean ± standard deviation. BOTH means GVCNN+MVCNN, which represents combining the features or structures to generate multi-modal data.

Datasets	Feature	Structure	HGNN	UniGNN	HGAT	ED-HNN	PhenomNN	H-ChebNet	H-SSGC	H-APPNP	H-GCN	H-GCNII
NTU	MVCNN	MVCNN	80.11±0.38	75.25±0.17	80.40±0.47	82.52±0.40	-	78.04±0.46	81.23±0.24	80.16±0.36	81.37±0.63	**84.58±0.34**
	GVCNN	GVCNN	84.26±0.30	84.63±0.21	84.45±0.12	85.20±0.28	-	83.51±0.40	84.26±0.12	84.96±0.41	85.15±0.34	**85.36±0.19**
	BOTH	BOTH	83.54±0.50	84.45±0.40	84.05±0.36	85.52±0.31	85.40±0.42	83.16±0.46	84.13±0.34	83.57±0.42	84.45±0.40	**85.71±0.18**
Model-Net40	MVCNN	MVCNN	91.28±0.11	90.36±0.10	91.29±0.15	91.99± 0.12	-	90.86±0.29	91.21±0.11	91.18±0.21	91.99±0.16	**92.04±0.07**
	GVCNN	GVCNN	92.53±0.06	**92.88±0.10**	92.44±0.11	92.55±0.08	-	92.46±0.15	92.74±0.04	92.46±0.09	92.66±0.10	92.80±0.05
	BOTH	BOTH	97.15±0.14	96.69±0.07	96.44±0.15	97.22±0.09	97.77±0.11	96.95±0.09	97.07±0.07	97.20±0.14	97.28±0.15	**97.90±0.05**

TABLE 3: Classification accuracy (%) on ModelNet40. The *embedding* means the output representations of MVCNN+GVCNN Extractor.

Methods	input	Accuracy
MVCNN [56]	image	90.1
PointNet [57]	point	89.2
PointNet++ [58]	point	90.1
DGCNN [59]	point	92.2
InterpCNN [60]	point	93.0
SimpleView [61]	image	93.6
pAConv [62]	point	93.9
Point2Vec [63]	point	94.8
ReCon++ [64]	point	95.0
PointGST [65]	point	95.3
HGAT [10]	embedding	96.4
UniGNN [24]	embedding	96.7
HGNN [11]	embedding	97.2
ED-HNN [29]	embedding	97.2
PhenomNN [28]	embedding	97.8
H-ChebNet	embedding	97.0
H-SSGC	embedding	97.1
H-APPNP	embedding	97.2
H-GCN	embedding	97.3
H-GCNII	embedding	**97.9**

via the well-established equivalence-induced Laplacian. Despite this, most convolutional algorithms for digraphs are more or less related to undirected graphs [67], [68], [69], [70]. Essentially, they transform digraphs to undigraphs with various techniques due to the difficulties of directed graph Laplacian.

An arguably more succinct way for hypergraph learning is to use undigraph convolutional networks via transforming hypergraphs into undigraphs. Compared to digraphs, the undigraph convolutional networks have been extensively studied and there exist various effective strategies for analyzing their theoretical properties, to name a few [18], [32], [35], [71].

Here, we develop a hypergraph spectral convolutions framework, based on equivalency conditions in Thm. 2. As the Laplacian $\mathbf{L}$ enjoys the same formula under any of the two equivalency conditions, algorithms deduced by $\mathbf{L}$ would be adaptive to any generalized hypergraph satisfying Thm. 2, including EIVW-hypergraph and EDVW-hypergraph.

General Hypergraph Spectral Convolution Framework. The General Hypergraph Spectral Convolutions (**GHSC**) is defined as a general end-to-end framework that can utilize any GCNNs as a backbone,

$$Y = g_f(\mathbf{X}, \mathbf{L}) = f(\mathbf{X}, \mathbf{L}_g \leftarrow \mathbf{L}), \tag{9}$$

where $\mathbf{L}_g$ is the graph Laplacian used in undigraph GCNNs f. $\leftarrow$ denotes replacing the graph Laplacian with hypergraph Laplacian $\mathbf{L}$ that defined in Corollary 4.1. Y denotes the output of g_f. We call these GNNs-induced Hypergraph neural network g_f **H-GNNs**. For example, H-GCN and H-SSGC represent hypergraph NNs induced by the GNN models GCN [17] and SSGC [35], respectively. Furthermore, we provide various H-GNN variants in Appendix 13.1.

The GHSC holds the following advantages: (1) It can handle both EIVW-hypergraphs and EDVW-hypergraphs via the unified Laplacian $\mathbf{L}$. (2) The unity of random walk makes it possible to aggregate more fine-grained information. (3) By using the framework, one can take established powerful techniques on undigraphs as a subroutine for hypergraph learning, which would largely ease hypergraph learning in real-world scenarios. (4) The GHSC is a spectral framework that may share the same properties as GCNNs, such as over-smoothing issues [66], [72]. Specifically, Corollary 4.2 implies that the spectral convolution deduced from the Laplacian $\mathbf{L}$ might suffer from the over-smoothing issues [18] (we give an example of analysis in Appendix 13.2). This means that the existing solution to the over-smoothing issue also can be extended for hypergraph learning, which leaves for future work.

7 Experimental Results

We evaluate the proposed GHSC Framework on four tasks: citation network classification, visual object classification, protein quality assessment, and fold classification. For simplicity, we set $\rho(\cdot)$ to be a power function $(\cdot)^\sigma$ (σ is a hyper-parameter). The weight matrix of hyperedges $\mathbf{W}$ is set to be an identity matrix by default. Citation network classification belongs to EIVW-hypergraph learning tasks ($\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$), and visual object classification and protein learning are EDVW-hypergraph learning tasks. The details regarding experiments can be found in Appendix 16. In our experiments, we follow the setting of $\mathbf{Q}_1 = \mathbf{Q}_2$, under which the conversion from hypergraph $\mathcal{H}(\mathcal{V}, \mathcal{E}, \mathbf{W}, \mathbf{Q}_1, \mathbf{Q}_1)$ to undirected graph is injective.

We defer additional experimental results on protein Quality Assessment (QA), Over-smoothing Analysis, Ablation Analysis, Robustness Analysis, and Running Time to Appendices 16.6, 16.7, 16.9, 16.8, and 16.11 respectively. The insights obtained are: 1) H-GNNs obtain a competitive performance on protein QA (Table 15); 2) The proposed methods can efficiently avoid over-smoothing; 3) The renormalization trick and edge-dependent vertex weights are both effective; 4) The proposed model exhibits consistent robustness under various perturbations, including hyperedge removal, hyperedge addition (even in heterophilic settings), and missing vertex weights, highlighting its stability across noisy or incomplete hypergraph structures. 5) The results illustrate that our H-GNNs are of the same order of magni-

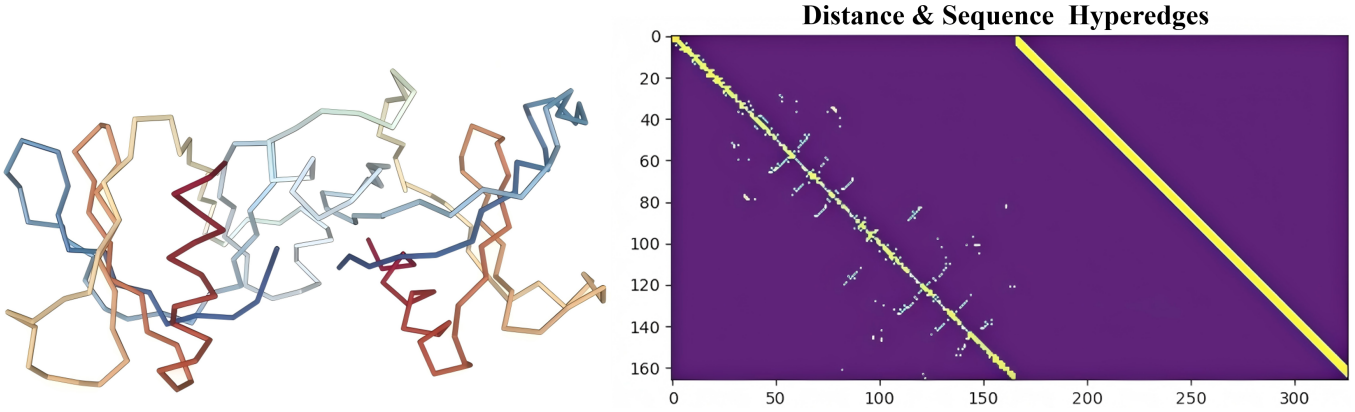


Fig. 5: **Left**: Visualization of protein 3D structure. The protein are composed of chains of amino acids (residues). **Right**: An example of edge-dependent vertex weight matrix $\mathbf{Q}$ of the protein. Each row represents an amino-acid and each column means one hyperedge. We model the protein as an EDVW-hypergraph for protein representation learning.

tude as SOTA's approaches and outperform most baselines on citation network classification.

7.1 Citation Network Classification

This is a semi-supervised node classification task. The datasets we use are hypergraph benchmarks constructed by [8](See Appendix Table 11). We adopt the same public datasets and train-test splits of [8]. Note that these datasets are EIVW-hypergraphs (i.e. $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$). For baselines, we include Multi-Layer Perceptron with explicit Hypergraph Laplacian Regularization (MLP+HLR), HNHN [23] , HyperSAGE [73], HGAT [10], UniGNN [24], ED-HNN [29], PhenomNN [28] and two recent spectral-based hypergraph convolutional neural networks (HGCNNs): HGNN [11] and HyperGCN [8]. To verify the effectiveness of the GHSC framework, we construct five H-GNNs variants: H-ChebNet, H-GCN, H-APPNP, H-SSGC, H-GCNII with ChebNet [34], GCN [17], APPNP [37], SSGC [35] GCNII [18] as f in Eq. (9), respectively.

Comparison with SOTAs. As shown in Table 1, the results successfully verify the effectiveness of our models and achieve a new SOTA performance across all five datasets. Observation (1): H-GNNs consistently gain improvement to the hypergraph-tailored baselines. This significant performance benefits from the GHSC framework, which is capable of taking advantage of the existing powerful undigraph NNs. Observation (2): HGNN, HNHN, and HGAT show poor performance on disconnected datasets (e.g. Citeseer), mainly due to the values of the row corresponding to an isolated vertex in the adjacency matrix of equivalent undigraph being 0, leading to direct loss of its vertex information (an example in Fig. 6 of the appendix). H-GNNs utilize the re-normalization trick, which can maintain the features of isolated vertices during aggregation.

7.2 Visual Object Classification

This experiment is about semi-supervised learning. We employ two public benchmarks: Princeton ModelNet40 dataset [74] and the National Taiwan University (NTU) 3D model dataset [75] to evaluate our methods. We follow HGNN [11] to preprocess the data by MVCNN [76] and GVCNN [56] and obtain the EDVW-hypergraphs.

Results. Observation (1): Table 3 depicts that our methods significantly outperform the image-input or point-input methods. These results demonstrate that our methods can capture the similarity of objects in the feature space to improve the performance of the classification task. Observation (2): From the results in Table 2, we can see our methods achieve competitive performance on both single-modality and multi-modality (BOTH) tasks and H-GCNII outperforms baselines on multi-modality. These results reveal that our methods have the advantage of combining such multi-modal information through concatenating the weighted incidence matrices ($\mathbf{Q}$) of hypergraphs, which means merging the multi-level hyperedges.

7.3 Protein Fold Classification

Protein fold classification is vital for mining the properties of proteins and studying the relationship between protein structure and function, and protein evolution. The function of a protein is primarily determined by its 3D structure, which contains the high-order interaction of amino acids. However, most researchers in protein learning represent a protein as a sequence or a simple graph [81], [82] that does not model the higher-order interactions between amino acids well. Inspired by [7], who construct the protein hypergraph to solve the conformation problem, we represent the protein as an EDVW-hypergraph to explore the high-order relationship between amino-acids for obtaining a comprehensive protein representation via H-GNNs.

In this experiment, we'd like to investigate whether hypergraphs are better than simple graphs for protein modeling. We select some classical baselines that use the GCNNs as the architecture for a fair comparison, instead of other SOTA methods on this task that adopt complicated techniques and specific architecture [83], [84], [85]. The following results confirm this conjecture.

Protein hypergraph. A protein is a chain of amino acids (residues) that will fold into a 3D structure. To simultaneously model protein sequence and spatial structure information, we build *sequence hyperedges* and *distance hyperedges* (c.f. Fig. 5): we choose τ consecutive amino acids

TABLE 4: classification accuracy ± standard deviation) with different $\rho(x)$ used in Eq. (9). ($\phi(x)$ is Gaussian PDF.)

Dataset	Methods	Random	x^{-1}	x^0	x^1	sigmoid(x)	$\phi(x)$	log(x)	exp(x)	exp$(-x)$
coauthorship/cora	H-APPNP	75.21±0.6	**76.38±0.8**	75.51±1.0	75.06±0.9	75.49±1.0	75.78±0.6	75.18±0.9	74.16±0.8	64.6±1.4
cocitation/pubmed	H-APPNP	75.43±1.0	75.31±1.1	75.46±1.0	75.29±1.2	75.45±1.0	**75.47±1.1**	75.40±1.1	74.44±1.4	73.54±1.3
coauthorship/cora	H-GCNII	75.68±0.8	**76.21±1.0**	75.57±0.8	75.39±0.8	75.64±1.0	76.15±0.9	75.29±0.8	74.64±0.9	65.47±1.0
cocitation/pubmed	H-GCNII	75.50±1.2	**75.79±1.1**	75.74±1.3	74.85±1.4	75.61±1.4	74.64±0.9	75.39±1.3	73.62±1.9	74.68±1.2

TABLE 5: Comparison of our methods to others on fold classification. We report the *mean accuracy* (%) of all proteins. (Seq: Sequence; Stru: Structure)

Methods	Architecture	Input	#params	Fold	Super.	Fam.
Hou et al. [77]	1D ResNet	Seq	41.7M	17.0	31.0	77.0
Rao et al. [78]*	1D Transformer	Seq	38.4M	21.0	34.0	88.0
Bepler & Berge [79]*	LSTM	Seq	31.7M	17.0	20.0	79.0
Strodthoff et al. [19]*	LSTM	Seq	22.7M	14.9	21.5	83.6
GAT [30]	GCNN	Stru	2.0M	12.4	16.5	72.7
GCN [17]	GCNN	Stru	1.0M	16.8	21.3	82.8
Diehl [20]	GCNN	Stru	1.0 M	12.9	16.3	72.5
Gligorijevic et al. [80]*	LSTM+GCNN	Seq+Stru	6.2M	15.3	20.6	73.2
Baldassarre et al. [81]	GCNN	Stru	1.3M	23.7	32.5	84.4
HGNN	HGCNN	Stru	2.1M	24.2	34.4	90.0
H-SSGC (Ours)	HGCNN	Stru	0.8M	21.4	23.0	78.4
H-GCN (Ours)	HGCNN	Stru	2.1M	25.0	36.3	91.6
H-GCNII (Ours)	HGCNN	Stru	0.6M	**27.8**	**38.0**	**92.7**

* Pre-trained unsupervised on 10-31 million protein sequences.

$(v_i, v_{i+1}, \cdots, v_{i+\tau})$ to form a sequence hyperedge and choose amino acids whose spatial Euclidean distance is less than the threshold $\epsilon > 0$ to form a distance hyperedge, where $v_i (i = 1, \cdots, |S|)$ represents the i-th amino acids in the sequence. Let $\mathcal{E}_s$ and $\mathcal{E}_d$ denote sequence hyperedges set and distance hyperedges set, respectively. Then, we design an edge-dependent vertex-weight matrix $\mathbf{Q}$ for capturing the more fine-grained high-order relationships of proteins below (actually, our H-GNNs allow one to design a more comprehensive $\mathbf{Q}$ for learning proteins better):

$$Q(i,j) = \begin{cases} 1, & \text{if } v_i \in e_j \text{ and } e_j \in \mathcal{E}_s \\ \exp(\frac{-d(v_i,v_c)}{\gamma \hat{d}^2_{v_c}}), & \text{if } v_i \in e_j \text{ and } e_j \in \mathcal{E}_d \\ 0, & \text{otherwise} \end{cases} \quad (10)$$

where $d(v, v_c) < \epsilon$ is the euclidean distance between an amino acid v and the centroid amino acid v_c in the hyperedge and $\hat{d}_{v_c}$ is the average distance between v_c and the other amino acids $\{v_i\}_{i \neq c}$. In our experiments, the v_c is set to be v_j. γ is a hyper-parameter to control the flatness. Here we design an edge-dependent vertex-weights matrix $\mathbf{Q}$ to satisfy the condition (2) in Thm. 2 (i.e. $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{Q}$). Appendix 16.6 contains more details.

Settings & Results. We use the well-known SCOPe 1.75 dataset [77] and the cross-entropy loss. Table 5 depicts that (1) H-GCNII outperforms all others, and (2) HGCNN-based methods achieve much better performance compared to GCNN-based. The results demonstrated that the hypergraph can better model the 3D structure of proteins than the simple graph, and our proposed methods can learn a better protein representation.

7.4 Ablation Studies

We conduct ablation studies to verify that our GHSC framework benefits from the proposed unified hypergraph Laplacian. First, we compare the naive Laplacian $\mathbf{L}_s = \mathbf{I} - \mathbf{D}_v^{-1/2}\mathbf{H}\mathbf{D}_e^{-1}\mathbf{H}\mathbf{D}_v^{-1/2}$ with the equivalency induced Laplaican $\mathbf{L}$ in Eq. (7) via replacing the $\mathbf{L}_g$ with $\mathbf{L}_s$ and $\mathbf{L}$ in Eq. (9), respectively. And then, we also investigate the performance of the proposed method with different ρ on citation datasets ($\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$), to further explore the impact of Laplcian's ρ on GHSC framework.

TABLE 6: Test accuracy (%) and standard deviation for ablation study with 10 random seeds.

Methods	Laplaican	NTU	ModelNet40
H-GCN	$\mathbf{L}_s$	84.93±0.31	97.18±0.20
	$\mathbf{L}$	85.15±0.34$\uparrow_{0.78}$	97.28±0.15$\uparrow_{0.1}$
H-SSGC	$\mathbf{L}_s$	83.03±0.30	96.82±0.10
	$\mathbf{L}$	84.13±0.29$\uparrow_{1.1}$	96.94±0.06$\uparrow_{0.12}$
H-GCNII	$\mathbf{L}_s$	85.04±0.29	97.62±0.07
	$\mathbf{L}$	85.17±0.36$\uparrow_{0.13}$	97.75±0.07$\uparrow_{0.13}$

Results. 1) As shown in Table 6, the unified Laplacian $\mathbf{L}$ significantly and consistently outperforms the naive Laplacian $\mathbf{L}_s$, indicating that the unified Laplacian captures more comprehensive information than the naive Laplacian. Meanwhile, this result also reveals that our H-GNNs successfully mine the information of the EDVWs embedded into the Laplacian. 2). From Table 4, we can see H-GNNs have different performances under various ρ, indicating that ρ can influence the performance of neural networks through the random walk. Meanwhile, the x^{-1} achieves a satisfactory performance. (For more analysis on $\rho = (\cdot)^\sigma$, see Appendix 16.10.) This can be explained as follows. In co-authorship graphs, papers with a larger number of co-authors are often the result of collaborations between researchers from various fields, and therefore will not be as predictive as papers with fewer authors. The co-citation graph enjoys a similar explanation. Therefore, the results suggest that our GHSC framework benefits from the unified Laplacian.

8 Discussions and Future Work

Despite the good results, several limitations remain and open promising directions for future exploration: 1) We provide sufficient conditions for the equivalence between hypergraphs and undirected graphs; however, identifying necessary conditions remains an open problem. 2) Building on the established equivalence between hypergraphs and undirected graphs, we have extended undirected graph-based GCNNs to hypergraph learning. Exploring the equivalence between hypergraphs and directed graphs may enable the adaptation of digraph-based GCNNs for capturing richer relational information in hypergraphs. 3) The proposed unified random walk framework could be further explored to design new clustering algorithms for hypergraph partitioning (see Appendix for initial discussions). 4) The current method is primarily designed for homophilic hypergraphs. Extending it to effectively handle heterophilic

hypergraphs represents a promising and impactful direction for future research [86].

ACKNOWLEDGMENT

This work was supported in part by the Natural Science Foundation of Guangdong Province (2025A1515011946), and by the Key Laboratory of Data Protection and Intelligent Management, Ministry of Education, Sichuan University, and also the Fundamental Research Funds for the Central Universities under Grant SCU2023D008.

REFERENCES

[1] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in Proceedings of the International Conference on Learning Representations, 2017.

[2] J. Zhang, X. Xiao, L.-K. Huang, Y. Rong, and Y. Bian, "Fine-tuning graph neural networks via graph topology induced optimal transport," in Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22, L. D. Raedt, Ed. International Joint Conferences on Artificial Intelligence Organization, 7 2022, pp. 3730–3736, main Track. [Online]. Available: https://doi.org/10.24963/ijcai.2022/518

[3] J. Zhang, Z. Liu, Y. Wang, B. Feng, and Y. Li, "Subgdiff: A subgraph diffusion model to improve molecular representation learning," Advances in Neural Information Processing Systems, vol. 37, pp. 29 620–29 656, 2024.

[4] T. Hwang, Z. Tian, R. Kuangy, and J.-P. Kocher, "Learning on weighted hypergraphs to integrate protein interactions and gene expressions for cancer outcome prediction," in 2008 Eighth IEEE International Conference on Data Mining. IEEE, 2008, pp. 293–302.

[5] F. Klimm, C. Deane, and G. Reinert, "Hypergraphs for predicting essential genes using multiprotein complex data," Journal of Complex Networks, 2020.

[6] Y. Gao, Z. Zhang, H. Lin, X. Zhao, S. Du, and C. Zou, "Hypergraph learning: Methods and practices," IEEE Transactions on Pattern Analysis and Machine Intelligence, 2020.

[7] O. Maruyama, T. Shoudai, E. Furuichi, S. Kuhara, and S. Miyano, "Learning conformation rules," in International Conference on Discovery Science. Springer, 2001, pp. 243–257.

[8] N. Yadati, M. Nimishakavi, P. Yadav, V. Nitin, A. Louis, and P. Talukdar, "Hypergcn: A new method for training graph convolutional networks on hypergraphs," in Advances in Neural Information Processing Systems, 2019, pp. 1511–1522.

[9] J. Zhang, Y. Chen, X. Xiao, R. Lu, and S.-T. Xia, "Learnable hypergraph laplacian for hypergraph learning," in ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2022, pp. 4503–4507.

[10] K. Ding, J. Wang, J. Li, D. Li, and H. Liu, "Be more with less: Hypergraph attention networks for inductive text classification," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 4927–4936.

[11] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao, "Hypergraph neural networks," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 33, 2019, pp. 3558–3565.

[12] D. Zhou, J. Huang, and B. Schölkopf, "Learning with hypergraphs: Clustering, classification, and embedding," Advances in neural information processing systems, vol. 19, pp. 1601–1608, 2006.

[13] U. Chitra and B. Raphael, "Random walks on hypergraphs with edge-dependent vertex weights," in International Conference on Machine Learning. PMLR, 2019, pp. 1172–1181.

[14] T. Carletti, F. Battiston, G. Cencetti, and D. Fanelli, "Random walks on hypergraphs," Physical Review E, vol. 101, no. 2, p. 022308, 2020.

[15] T. Carletti, D. Fanelli, and R. Lambiotte, "Random walks and community detection in hypergraphs," Journal of Physics: Complexity, 2021.

[16] F. Chung, "Laplacians and the cheeger inequality for directed graphs," Annals of Combinatorics, vol. 9, no. 1, pp. 1–19, 2005.

[17] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in Proceedings of the International Conference on Learning Representations, 2017.

[18] M. Chen, Z. Wei, Z. Huang, B. Ding, and Y. Li, "Simple and deep graph convolutional networks," in International Conference on Machine Learning. PMLR, 2020, pp. 1725–1735.

[19] N. Strodthoff, P. Wagner, M. Wenzel, and W. Samek, "Udsmprot: universal deep sequence models for protein classification," Bioinformatics, vol. 36, no. 8, pp. 2401–2409, 2020.

[20] F. Diehl, "Edge contraction pooling for graph neural networks," CoRR, 2019.

[21] J. Zhang, F. Li, X. Xiao, T. Xu, Y. Rong, J. Huang, and Y. Bian, "Hypergraph convolutional networks via equivalency between hypergraphs and undirected graphs," arXiv preprint arXiv:2203.16939, 2022.

[22] S. Agarwal, K. Branson, and S. Belongie, "Higher order learning with graphs," in Proceedings of the 23rd international conference on Machine learning, 2006, pp. 17–24.

[23] Y. Dong, W. Sawin, and Y. Bengio, "Hnhn: Hypergraph networks with hyperedge neurons," arXiv preprint arXiv:2006.12278, 2020.

[24] J. Huang and J. Yang, "Unignn: a unified framework for graph and hypergraph neural networks," in Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21, 2021.

[25] E. Chien, C. Pan, J. Peng, and O. Milenkovic, "You are allset: A multiset function framework for hypergraph neural networks," 2021.

[26] Y. Gao, Y. Feng, S. Ji, and R. Ji, "Hgnn+: General hypergraph neural networks." IEEE Transactions on Pattern Analysis and Machine Intelligence, 2022.

[27] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in International Conference on Machine Learning. PMLR, 2017, pp. 1263–1272.

[28] Y. Wang, Q. Gan, X. Qiu, X. Huang, and D. Wipf, "From hypergraph energy functions to hypergraph neural networks," in International Conference on Machine Learning. PMLR, 2023, pp. 35 605–35 623.

[29] P. Wang, S. Yang, Y. Liu, Z. Wang, and P. Li, "Equivariant hypergraph diffusion neural operators," in The Eleventh International Conference on Learning Representations, 2023. [Online]. Available: https://openreview.net/forum?id=RiTjKoscnNd

[30] P. Velivcković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in International Conference on Learning Representations, 2018.

[31] Y. Chen, Y. Bian, J. Zhang, X. Xiao, T. Xu, Y. Rong, and J. Huang, "Diversified multiscale graph learning with graph self-correction," arXiv preprint arXiv:2103.09754, 2021.

[32] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," IEEE transactions on neural networks and learning systems, vol. 32, no. 1, pp. 4–24, 2020.

[33] D. I. Shuman, S. K. Narang, P. Frossard, A. Ortega, and P. Vandergheynst, "The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains," IEEE signal processing magazine, vol. 30, no. 3, pp. 83–98, 2013.

[34] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," Advances in neural information processing systems, vol. 29, pp. 3844–3852, 2016.

[35] H. Zhu and P. Koniusz, "Simple spectral graph convolution," in International Conference on Learning Representations, 2021.

[36] G. Li, M. Müller, B. Ghanem, and V. Koltun, "Training graph neural networks with 1000 layers," in International conference on machine learning. PMLR, 2021, pp. 6437–6449.

[37] J. Klicpera, A. Bojchevski, and S. Günnemann, "Predict then propagate: Graph neural networks meet personalized pagerank," in International Conference on Learning Representations, 2018.

[38] L. Lu and X. Peng, "High-ordered random walks and generalized laplacians on hypergraphs," in International Workshop on Algorithms and Models for the Web-Graph. Springer, 2011, pp. 14–25.

[39] S. G. Aksoy, C. Joslyn, C. O. Marrero, B. Praggastis, and E. Purvine, "Hypernetwork science via high-order hypergraph walks," EPJ Data Science, vol. 9, no. 1, p. 16, 2020.

[40] C. Zhang, S. Hu, Z. G. Tang, and T. H. Chan, "Re-revisiting learning on hypergraphs: confidence interval and subgradient method," in International Conference on Machine Learning. PMLR, 2017, pp. 4026–4034.

[41] T.-H. H. Chan, Z. G. Tang, X. Wu, and C. Zhang, "Diffusion operator and spectral analysis for directed hypergraph laplacian," Theoretical Computer Science, vol. 784, pp. 46–64, 2019.

[42] P. Li and O. Milenkovic, "Inhomogeneous hypergraph clustering with applications," arXiv preprint arXiv:1709.01249, 2017.

[43] K. Hayashi, S. G. Aksoy, C. H. Park, and H. Park, "Hypergraph random walks, laplacians, and clustering," in Proceedings of the 29th ACM International Conference on Information & Knowledge Management, 2020, pp. 495–504.

[44] M. Hein, S. Setzer, L. Jost, and S. S. Rangapuram, "The total variation on hypergraphs-learning on hypergraphs revisited," Advances in Neural Information Processing Systems, vol. 26, pp. 2427–2435, 2013.

[45] Z. Zhang, H. Lin, Y. Gao, and K. BNRist, "Dynamic hypergraph structure learning." in IJCAI, 2018, pp. 3162–3169.

[46] L. Ding and A. Yilmaz, "Interactive image segmentation using probabilistic hypergraphs," Pattern Recognition, vol. 43, no. 5, pp. 1863–1873, 2010.

[47] J. Li, J. He, and Y. Zhu, "E-tail product return prediction via hypergraph-based local graph cut," in Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2018, pp. 519–527.

[48] Y. Huang, Q. Liu, S. Zhang, and D. N. Metaxas, "Image retrieval via probabilistic hypergraph ranking," in 2010 IEEE computer society conference on computer vision and pattern recognition. IEEE, 2010, pp. 3376–3383.

[49] A. Bellaachia and M. Al-Dhelaan, "Random walks in hypergraph," in Proceedings of the 2013 International Conference on Applied Mathematics and Computational Methods, Venice Italy, 2013, pp. 187–194.

[50] A. Ducournau and A. Bretto, "Random walks in directed hypergraphs and application to semi-supervised image segmentation," Computer Vision and Image Understanding, vol. 120, pp. 91–102, 2014.

[51] K. Zeng, N. Wu, A. Sargolzaei, and K. Yen, "Learn to rank images: A unified probabilistic hypergraph model for visual search," Mathematical Problems in Engineering, vol. 2016, 2016.

[52] F. R. Chung and F. C. Graham, Spectral graph theory. American Mathematical Soc., 1997, no. 92.

[53] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in International Conference on Learning Representations, 2019. [Online]. Available: https://openreview.net/forum?id=ryGs6iA5Km

[54] C. Morris, M. Ritzert, M. Fey, W. L. Hamilton, J. E. Lenssen, G. Rattan, and M. Grohe, "Weisfeiler and leman go neural: Higher-order graph neural networks," in Proceedings of the AAAI conference on artificial intelligence, vol. 33, no. 01, 2019, pp. 4602–4609.

[55] A. Leman and B. Weisfeiler, "A reduction of a graph to a canonical form and an algebra arising during this reduction," Nauchno-Technicheskaya Informatsiya, vol. 2, no. 9, pp. 12–16, 1968.

[56] Y. Feng, Z. Zhang, X. Zhao, R. Ji, and Y. Gao, "Gvcnn: Group-view convolutional neural networks for 3d shape recognition," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 264–272.

[57] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "Pointnet: Deep learning on point sets for 3d classification and segmentation," in Proceedings of the IEEE conference on computer vision and pattern recognition, 2017, pp. 652–660.

[58] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "Pointnet++: Deep hierarchical feature learning on point sets in a metric space," in NIPS, 2017.

[59] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, "Dynamic graph cnn for learning on point clouds," Acm Transactions On Graphics (tog), vol. 38, no. 5, pp. 1–12, 2019.

[60] J. Mao, X. Wang, and H. Li, "Interpolated convolutional networks for 3d point cloud understanding," in Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019, pp. 1578–1587.

[61] M. A. Uy, Q.-H. Pham, B.-S. Hua, T. Nguyen, and S.-K. Yeung, "Revisiting point cloud classification: A new benchmark dataset and classification model on real-world data," in 2019 IEEE/CVF International Conference on Computer Vision (ICCV). IEEE Computer Society, 2019, pp. 1588–1597.

[62] M. Xu, R. Ding, H. Zhao, and X. Qi, "Paconv: Position adaptive convolution with dynamic kernel assembling on point clouds," arXiv preprint arXiv:2103.14635, 2021.

[63] K. A. Zeid, J. Schult, A. Hermans, and B. Leibe, "Point2vec for self-supervised representation learning on point clouds," in DAGM German Conference on Pattern Recognition. Springer, 2023, pp. 131–146.

[64] Z. Qi, R. Dong, S. Zhang, H. Geng, C. Han, Z. Ge, L. Yi, and K. Ma, "Shapellm: Universal 3d object understanding for embodied interaction," in European Conference on Computer Vision. Springer, 2025, pp. 214–238.

[65] D. Liang, T. Feng, X. Zhou, Y. Zhang, Z. Zou, and X. Bai, "Parameter-efficient fine-tuning in spectral domain for point cloud learning," arXiv preprint arXiv:2410.08114, 2024.

[66] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32, no. 1, 2018.

[67] F. Monti, K. Otness, and M. M. Bronstein, "Motifnet: a motif-based graph convolutional network for directed graphs," in 2018 IEEE Data Science Workshop (DSW). IEEE, 2018, pp. 225–228.

[68] C. Li, X. Qin, X. Xu, D. Yang, and G. Wei, "Scalable graph convolutional networks with fast localized spectral filter for directed graphs," IEEE Access, vol. 8, pp. 105 634–105 644, 2020.

[69] Y. Ma, J. Hao, Y. Yang, H. Li, J. Jin, and G. Chen, "Spectral-based graph convolutional network for directed graphs," arXiv preprint arXiv:1907.08990, 2019.

[70] Z. Tong, Y. Liang, C. Sun, D. S. Rosenblum, and A. Lim, "Directed graph convolutional network," arXiv preprint arXiv:2004.13970, 2020.

[71] L. Zhao and L. Akoglu, "Pairnorm: Tackling oversmoothing in {gnn}s," in International Conference on Learning Representations, 2020. [Online]. Available: https://openreview.net/forum?id=rkecl1rtwB

[72] G. Chen, J. Zhang, X. Xiao, and Y. Li, "Preventing over-smoothing for hypergraph neural networks," arXiv preprint arXiv:2203.17159, 2022.

[73] D. Arya, D. K. Gupta, S. Rudinac, and M. Worring, "Hypersage: Generalizing inductive representation learning on hypergraphs," arXiv preprint arXiv:2010.04558, 2020.

[74] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao, "3d shapenets: A deep representation for volumetric shapes," in Proceedings of the IEEE conference on computer vision and pattern recognition, 2015, pp. 1912–1920.

[75] D.-Y. Chen, X.-P. Tian, Y.-T. Shen, and M. Ouhyoung, "On visual similarity based 3d model retrieval," in Computer graphics forum, vol. 22, no. 3. Wiley Online Library, 2003, pp. 223–232.

[76] H. Su, S. Maji, E. Kalogerakis, and E. Learned-Miller, "Multi-view convolutional neural networks for 3d shape recognition," in Proceedings of the IEEE international conference on computer vision, 2015, pp. 945–953.

[77] J. Hou, B. Adhikari, and J. Cheng, "Deepsf: deep convolutional neural network for mapping protein sequences to folds," Bioinformatics, vol. 34, no. 8, pp. 1295–1303, 2018.

[78] R. Rao, N. Bhattacharya, N. Thomas, Y. Duan, X. Chen, J. Canny, P. Abbeel, and Y. S. Song, "Evaluating protein transfer learning with tape," Advances in Neural Information Processing Systems, vol. 32, p. 9689, 2019.

[79] T. Bepler and B. Berger, "Learning protein sequence embeddings using information from structure," in International Conference on Learning Representations, 2018.

[80] V. Gligorijević, P. D. Renfrew, T. Kosciolek, J. K. Leman, D. Berenberg, T. Vatanen, C. Chandler, B. C. Taylor, I. M. Fisk, H. Vlamakis et al., "Structure-based protein function prediction using graph convolutional networks," Nature communications, vol. 12, no. 1, p. 3168, 2021.

[81] F. Baldassarre, D. Hurtado, A. Elofsson, and H. Azizpour, "Graphqa: Protein model quality assessment using graph convolutional networks." Bioinformatics (Oxford, England), 2020.

[82] P. Hermosilla, M. Schäfer, M. Lang, G. Fackelmann, P. P. Vázquez, B. Kozlíková, M. Krone, T. Ritschel, and M. Ropinski, Timo, "Intrinsic-extrinsic convolution and pooling for learning on 3d protein structures," in Proceedings of the International Conference on Learning Representations, 2021.

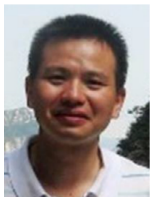
[83] P. Hermosilla and T. Ropinski, "Contrastive representation learning for 3d protein structures," 2022. [Online]. Available: https://openreview.net/forum?id=VINWzIM6_6

[84] L. Wang, H. Liu, Y. Liu, J. Kurtin, and S. Ji, "Learning hierarchical protein representations via complete 3d graph networks," in The Eleventh International Conference on Learning Representations, 2023.

[85] Z. Zhang, M. Xu, A. R. Jamasb, V. Chenthamarakshan, A. Lozano, P. Das, and J. Tang, "Protein representation learning by geometric structure pretraining," in The Eleventh International Conference on Learning Representations, 2023. [Online]. Available: https://openreview.net/forum?id=to3qCB3tOh9
[86] M. Li, Y. Gu, Y. Wang, Y. Fang, L. Bai, X. Zhuang, and P. Lio, "When hypergraph meets heterophily: New benchmark datasets and baseline," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, no. 17, 2025, pp. 18 377–18 384.
[87] T.-H. H. Chan and Z. Liang, "Generalizing the hypergraph laplacian via a diffusion process with mediators," Theoretical Computer Science, vol. 806, pp. 416–428, 2020.
[88] J. Jiang, Y. Wei, Y. Feng, J. Cao, and Y. Gao, "Dynamic hypergraph neural networks." in IJCAI, 2019, pp. 2635–2641.
[89] C. Wendler, M. Püschel, and D. Alistarh, "Powerset convolutional neural networks," Advances in Neural Information Processing Systems 32, vol. 2, pp. 929–940, 2020.
[90] Y. Zhang, N. Wang, Y. Chen, C. Zou, H. Wan, X. Zhao, and Y. Gao, "Hypergraph label propagation network," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, no. 04, 2020, pp. 6885–6892.
[91] N. Yadati, "Neural message passing for multi-relational ordered and recursive hypergraphs," Advances in Neural Information Processing Systems, vol. 33, 2020.
[92] T. Jin, L. Cao, B. Zhang, X. Sun, C. Deng, and R. Ji, "Hypergraph induced convolutional manifold networks." in IJCAI, 2019, pp. 2670–2676.
[93] R. Zhang, Y. Zou, and J. Ma, "Hyper-sagnn: a self-attention based graph neural network for hypergraphs," arXiv preprint arXiv:1911.02613, 2019.
[94] F. Chung, "The laplacian of a hypergraph," Expanding graphs (DIMACS series), pp. 21–36, 1993.
[95] M. Conover, M. Staples, D. Si, M. Sun, and R. Cao, "Angularqa: protein model quality assessment with lstm networks," Computational and Mathematical Biophysics, vol. 7, no. 1, pp. 1–9, 2019.
[96] K. Olechnovivc and v. Venclovas, "Voromqa: Assessment of protein structure quality using interatomic contact areas," Proteins: Structure, Function, and Bioinformatics, vol. 85, no. 6, pp. 1131–1145, 2017.
[97] G. Derevyanko, S. Grudinin, Y. Bengio, and G. Lamoureux, "Deep convolutional networks for quality assessment of protein folds," Bioinformatics, vol. 34, no. 23, pp. 4046–4053, 2018.
[98] J. Zhang, Z. Liu, S. Bai, H. Cao, Y. Li, and L. Zhang, "Efficient antibody structure refinement using energy-guided se (3) flow matching," in 2024 IEEE International Conference on Bioinformatics and Biomedicine (BIBM). IEEE, 2024, pp. 146–153.
[99] M. Li, J. Zhang, B. Feng, W. Zeng, D. Chen, Z. Pan, Y. Li, Z. Liu, and Y. I. Yang, "Enhanced sampling, public dataset and generative model for drug-protein dissociation dynamics," arXiv preprint arXiv:2504.18367, 2025.
[100] D. K. Hammond, P. Vandergheynst, and R. Gribonval, "Wavelets on graphs via spectral graph theory," Applied and Computational Harmonic Analysis, vol. 30, no. 2, pp. 129–150, 2011.
[101] R. A. Horn, "Topics in matrix analysis," 1986.
[102] N. Masuda, M. A. Porter, and R. Lambiotte, "Random walks and diffusion on networks," Physics reports, vol. 716, pp. 1–58, 2017.
[103] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, and T. Eliassi-Rad, "Collective classification in network data," AI magazine, vol. 29, no. 3, pp. 93–93, 2008.
[104] R. Rossi and N. Ahmed, "The network data repository with interactive graph analytics and visualization," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 29, no. 1, 2015.
[105] S. Bai, F. Zhang, and P. H. Torr, "Hypergraph convolution and hypergraph attention," Pattern Recognition, vol. 110, p. 107637, 2021.
[106] F. Tudisco, A. R. Benson, and K. Prokopchik, "Nonlinear higher-order label spreading," in Proceedings of the Web Conference, 2021.
[107] W. Kabsch and C. Sander, "Dictionary of protein secondary structure: pattern recognition of hydrogen-bonded and geometrical features," Biopolymers: Original Research on Biomolecules, vol. 22, no. 12, pp. 2577–2637, 1983.
[108] A. G. Murzin, S. E. Brenner, T. Hubbard, and C. Chothia, "Scop: a structural classification of proteins database for the investigation of sequences and structures," Journal of molecular biology, vol. 247, no. 4, pp. 536–540, 1995.
[109] A. Zemla, "Lga: a method for finding 3d similarities in protein structures," Nucleic Acids Research, vol. 31, no. 13, p. 3370, 2003.
[110] V. Mariani, M. Biasini, A. Barbato, and T. Schwede, "lddt: a local superposition-free score for comparing protein structures and models using distance difference tests," Bioinformatics, vol. 29, no. 21, pp. 2722–2728, 2013.
[111] J. Zhang and Y. Zhang, "A novel side-chain orientation dependent potential derived from random-walk reference state for protein fold selection and structure prediction," PloS one, vol. 5, no. 10, p. e15386, 2010.

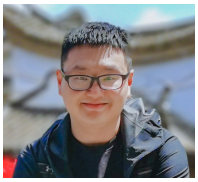
Jiying Zhang is a doctoral assistant at EPFL, Switzerland and an assistant researcher at International Digital Economy Academy (IDEA), China. He received a Master's degree in computer technology at Tsinghua University in 2023. His research interests include hypergraph learning, graph learning, diffusion models, protein design, and molecular dynamics. He has served as a reviewer for conferences such as ICML, NeurIPS, ICLR, CVPR, AAAI, AISTATS, and Journals like TPAMI.

Fuyang Li is a technical manager at China Huaneng Group.,Ltd., engaged in AI technical management related to power trading. He received a Master's degree in computer technology at Tsinghua University in 2023 and has been an intern at Tencent AI-Lab for one year. His research interests focus on time series forecasting, offline reinforcement learning and hypergraph learning, AI4Electricity.



Xi Xiao is an associate professor in Graduate School at Shenzhen, Tsinghua University. He got his Ph.D. degree in 2011 in State Key Laboratory of Information Security, Graduate University of Chinese Academy of Sciences. His research interests focus on machine learning, formation security, and computer network.

Guanzi Chen received a Master's degree in Data Science and Information Technology at Tsinghua University in 2023. Her research interests focus on graph learning and transfer learning.



D r. Tingyang Xu is a Senior Researcher in AI for Science Group at DAMO Academy, Alibaba since 2024 where he focuses on research in deep graph learning, AI for drug discovery, and AI for Science. He earned his master and Ph.D. from The University of Connecticut and his Bachelor's degree from Shanghai Jiaotong University. He has significantly contributed to the development of advanced graph neural networks for tasks like subgraph recognition and node classification. His work has been published in top-tier data mining and machine learning conferences, including NeurIPS, ICML, SIGKDD, AAAI, VLDB, Nature Communication (NC), Internet of Things (IoT), and Annuals of Surgery. Additionally, Dr. Xu has served as a reviewer for prestigious conferences and journals, including ICML, NeurIPS, KDD, WWW, AAAI, PR, and TKDE, and as the Industrial Track Chair for BIBM 2019.

Yu Rong , IEEE Senior Member, is a Senior Staff Algorithm Engineer at DAMO Academy, Alibaba Group, leading the Language And Science AI (LASA) Lab. He received his Ph.D. in System Engineering and Engineering Management from the Chinese University of Hong Kong in 2016, where he also conducted postdoctoral research. In June 2017, he joined Tencent AI Lab as a principal researcher. He joined Alibaba DAMO Academy in June 2024, focusing on large language models and AI for Science research. His primary research interests are in graph deep learning and large language models, with specific applications in the AI for Science domain. His papers have been cited over 10,200 times on Google Scholar, with a single paper receiving over 1,800 citations. Two of his papers (ICLR 2020, WWW 2020) were selected as the most influential papers by PaperDigest. He has also been listed among Stanford/Elsevier's Top 2% Scientists and ranked second in the mainland China machine learning subfield of the AMiner Rising Star list (2020-2023).

Junzhou Huang is the Jenkins Garrett Endowed Professor in the Computer Science and Engineering department at the University of Texas at Arlington. He has been the director of the machine learning center at Tencent AI Lab. He received the B.E. degree from Huazhong University of Science and Technology, Wuhan, China, the M.S. degree from the Institute of Automation, Chinese Academy of Sciences, Beijing, China, and the Ph.D. degree in Computer Science at Rutgers, The State University of New Jersey. His major research interests include machine learning, computer vision, medical image analysis and bioinformatics. His research has been recognized by several awards including the NSF CAREER Award from the National Science Foundation, TensorFlow Model Garden Award from Google, Emerging Leaders from IBM Watson, four Best Paper Awards (MICCAI'10, FIMH'11, STMI'12 and MICCAI'15) as well as two Best Paper Nominations (MICCAI'11 and MICCAI'14). His research projects are supported by both federal/state agencies (NSF, NIH, CPRIT) and industry (Google, Amazon, IBM, Samsung, XtaiPi and Nokia).

Yatao Bian is a tenure-track assistant professor at the National University of Singapore computer science. He received the Ph.D. degree from the Institute for Machine Learning at ETH Zurich. He has been an associated Fellow of the Max Planck ETH Center for Learning Systems from Jun. 2015 to Jan. 2020. He is dedicated to expanding the frontiers of AI capabilities in scientific intelligence, driving transformative technological advancements to address the most critical challenges in scientific research. Some of his research topics include: reasoning-empowered foundation models, LLM/Agent for science, interpretable substructure based robust graph learning, and energy based learning. He has won the National Championship in AMD China Accelerated Computing Contest. He has published several papers on machine learning top conferences/journals such as NeurIPS, ICML, ICLR, T-PAMI, AISTATS etc. He is an area chair for NeurIPS, ICLR and served as a reviewer/PC for conferences like ICML, NeurIPS, ICLR, CVPR, AAAI, STOC and journals such as JMLR, T-PAMI and Nature Machine Intelligence. For more information, please check his homepage at https://yataobian.com/.

Appendix

CONTENTS

9 RELATED WORK

9.1 Detailed Related Work

Deep learning for hypergraphs [11] introduces the first hypergraph architecture *hypergraph neural network*(HGNN), based on the hypergraph Laplacian proposed by [12], however, a flaw in the theory makes the learning of unconnected networks very inefficient (details can be seen in Appendix 12.1). [8] uses the non-linear Laplacian operators [87] to convert hypergraphs to simple graphs by reducing a hyperedge to a subgraph with edge weights related only to its degree, which causes the information loss of hypergraphs and limited the generalization. [88] propose a *dynamic hypergraph neural networks* to update hypergraph structure during training. [89] utilizes signal processing theory to define the convolutional neural network on set functions. [90] combine hypergraph label propagation with deep learning and introduced a *Hypergraph Label Propagation Network* to optimize the hypergraph learning. [91] propose a *generalised-MPNN* on hypergraph which unifies the existing MPNNs [27] on simple graph and also raises a *MPNN-Recursive* framework for recursively-structured data processing, but it performs poorly for other high-order relationships. [24] generalizes the current graph message passing methods to hypergraphs and obtain a UniGCN with self-loop and a deepened hypergraph network UniGCNII. However, it does not give reasons why a self-loop can assist UniGCN in gaining the improvement of performance in the citation network benchmark. Moreover, its experimental results with various depths may show that the HGCNNs have an over-smoothing issue [72], but a theoretical analysis to explain this phenomenon is lacking. In addition, [26] proposed a

general spatial convolution on hypergraphs and compared HGNN and GNN from the spectral perspective. Recently, [86] proposes a significant heterophilic benchmark, including heterophilic hypergraph datasets, and also raises a hypergraph neural network to handle the heterophilic data. However, they only give an explanation of EIVW-hypergraph processing, without analyzing how to process EDVW-hypergraphs. Hypergraph deep learning has also been applied to many areas, such as NLP [10], computer vision [92], recommendation system [93], etc

Hypergraph random walk and spectral theory. In machine learning applications, Laplacian is an important tool to represent graphs or hypergraphs. While the random walk-based graph Laplacian has a well-studied spectral theory, the spectral methods of hypergraphs are surprisingly lagged. The research of hypergraph Laplacian can probably be traced back to [94], which defines the Laplacian of k-uniform hypergraph (each hyperedge contains the same number of vertices). Then [12] defined a two-step random walk-based Laplacian for general hypergraphs. Based on it, many works try to design a more comprehensive random walk on hypergraphs. [13] designed a random walk that considered edge-dependent vertex weights of hypergraphs in the *second step* to replace the edge-independent weights in [12]. Actually, the edge-dependent vertex weights could model the contribution of vertex v to hyperedge e and were widely used in machine learning such as 3D object classification [11], [45] and image segmentation [46] etc., but without spectral guarantees. On the other hand, [14], [15] takes another perspective to gain more fine-grained information from a hypergraph by taking into account the degree of hyperedges to measure the importance between vertices in the *first step*. However, a comprehensive random walk should consider the above two aspects at the same time. Furthermore, different from the two-step random walk Laplacian of [12], [38] and [39] define a s-th Laplacian through random s-walk on hypergraphs. More recently, non-linear Laplacian, as an important tool to represent hypergraphs, has been extended to several settings, including directed hypergraphs [40], [41], submodular hypergraph [42] etc. Meanwhile, liner Laplacian has also been developed. For example, [13] uses a two-step random walk to develop a spectral theory for hypergraphs with edge-dependent vertex weights, and a two-step random walk on hypergraph proposed by [14], [15] also designed a two-step random walk by involving the degree of hyperedges in the first step and leading a linear Laplacian. In this work, we design a unified two-step random walk framework to study the linear Laplacians.

Equivalency between hypergraphs and undigraphs. A core problem in the spectral theoretical study of hypergraphs is the equivalency between hypergraphs and graphs. Once equipped with equivalency, it is possible to represent hypergraphs with lower-order graphs, thus reducing the difficulty of hypergraph learning. So far, equivalence can be established mainly from two perspectives: the spectrum of Laplacian or random walks. For the aspect of the Laplacian spectrum, [22] firstly showed the Laplacian of [12] is equivalent to the Laplacian of the corresponding star expansion graph (having the same eigenvalue problem). Next, [43] claims that the Laplacian defined by [13] is equal to an undigraph Laplacian, but the undigraph Laplacian needs to be constructed by invoking the stationary distribution, which restricts its application. Meanwhile, from the random walk perspective, [13] claims that the hypergraph is equivalent to its clique graph (undirected) when the vertex weight of the hypergraph is edge-independent. However, despite the edge-dependent weights being widely used [45], [46], [51], the equivalency problem remains open when the vertex weight is edge-dependent.

Protein learning. Proteins are biological macromolecules with spatial structures formed by folding chains of amino acids, which have an important role in biology. A protein has a three-level structure: primary, secondary, and tertiary, where primary (sequence of amino acids) and tertiary (3D spatial) structures are often used to model proteins for representation learning. Based on amino acids sequence, many related works of protein learning are proposed, such as [19], [78], [79], [95]. Meanwhile, many 3D structure-based models are also raised for protein learning, such as [20], [80], [81], [81], [82], [96], [97], [98], [99], etc. However, to the best of our knowledge, although the hypergraphs have been developed to model proteins [7], there is still no related work to design the EDVW-hypergraph-based protein learning algorithm.

10 Relations with Previous Random Walks on Hypergraph

10.1 A Classical Random Walk on Hypergraph

It can be traced back to [12] in which they have given a view of random-walk to analyze the hypergraph normalized cut. The transition probability of [12] from current vertex u to next vertex v is denoted as:

$$P(u,v)=\sum_{e\in\mathcal{E}} w(e)\frac{H(u,e)}{d(u)}\frac{H(v,e)}{\delta(e)}. \tag{11}$$

It is easy to find $P(u,v)$ means: (i) choose an arbitrary hyperedge e incident with u with probability $w(e)/d(u)$ where $d(u)=\sum_{e\in\mathcal{E}} w(e)H(u,e)$; (ii) Then choose an arbitrary vertex $v\in e$ with probability $1/\delta(e)$ where $\delta(e)=\sum_{v\in\mathcal{V}} H(v,e)$, which means to select vertex randomly in the hyperedge.

10.2 Intuition and Analysis of The Unified Random Walk on Hypergraph

In this subsection, we would like to explain our intuition of developing the unified random walk framework in Definition 1 from a view of a two-step random walk on a hypergraph. To generalize the notation of hypergraph random walk in the

seminal paper [12], in this work, we tend to explain the process from another perspective. The fact in [12] that

$$d(u) = \sum_{e\in\mathcal{E}} w(e)H(u,e) = \sum_{e\in\mathcal{E}}\sum_{v\in\mathcal{V}} w(e)\frac{H(u,e)H(v,e)}{\delta(e)} = \sum_{v\in\mathcal{V}}\sum_{e\in E(u,v)}\frac{w(e)}{\delta(e)} \tag{12}$$

and

$$P(u,v) = \sum_{e\in\mathcal{E}}\frac{w(e)H(u,e)}{d(u)}\cdot\frac{H(v,e)}{\delta(e)} = \frac{1}{d(u)}\sum_{e\in E(u,v)}\frac{w(e)}{\delta(e)} = \frac{\sum\limits_{e\in E(u,v)}\frac{w(e)}{\delta(e)}}{\sum\limits_{b\in\mathcal{V}}\sum\limits_{e\in E(u,b)}\frac{w(e)}{\delta(e)}} \tag{13}$$

show that the probability $P(u,v)$ also means a normalized weighted adjacency relationship between u and v (Here, $E(u,v)$ denotes the hyperedges contained vertices u and v.). Furthermore, the relationship is actually measured by the sum of $\frac{w(e)}{\delta(e)}$ for all $e\in\mathcal{E}$ concluding pair $\{u,v\}$, which demonstrates that a hyperedge e with larger degree ($\delta(e)$) linked to $\{u,v\}$ contribute less to the transition probability between u and v.

From [15], which proposed a random walk on hypergraphs (a special case of our unified random walk with $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$ and $\rho(\cdot) = (\cdot)^\sigma$), we further explain the intuition of our purpose to design the $P(u,v)$ of our unified random walk. When $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$, $P(u,v)$ in Eq. (3) can be expressed as:

$$P(u,v) = \frac{\sum\limits_{e\in E(u,v)} w(e)\rho(\delta(e))}{\sum\limits_{b\in\mathcal{V}}\sum\limits_{e\in E(u,b)} w(e)\rho(\delta(e))}$$

That is to say, the influence of the hyperedge degree should not be limited to an inverse relationship by introducing the function $\rho(\cdot)$. Thus, by evolving more fine-grained vertexes weights $Q_1(u,e)$ and $Q_2(v,e)$ to replace $H(v,e)$ and $H(u,e)$, we obtain the final transition probability of our unified hypergraph random walk in (3).

10.3 Special Cases of The Unified Random Walk Framework and Laplacian

We show the existing random walks and Laplacians are the special cases of our unified random walks and our unified Laplacians, respectively. Under our framework, the existing random walk models can be seen as a special case with specific p_1, p_2. It is easy to construct the relations of our framework to previous works as follows:

Remark 1 ([12]). When we choose $\rho(\cdot) = (\cdot)^{-1}$ and $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$ in our framework, the probabilities of unified random walk on hypergraph are reformulated as $p_1 = \frac{w(e)H(u,e)}{\sum_{e\in\mathcal{E}} w(e)H(u,e)}$ and $p_2 = \frac{H(v,e)}{\sum_{v\in\mathcal{V}} H(v,e)}$. As the special case of ours satisfying $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$ and $\rho(\cdot) = (\cdot)^{-1}$, [12]'s random walk satisfies both condition (1) and condition (2) in Thm. 2. So we obtain from Corollary 4.1 that $\pi_{zhou}(v) = \frac{d(v)}{\sum_{u\in\mathcal{V}} d(u)}$ and $\mathbf{L}_{zhou} = \mathbf{I} - \mathbf{D}_v^{-1/2}\mathbf{H}\mathbf{W}\mathbf{D}_e^{-1}\mathbf{H}^\top\mathbf{D}_v^{-1/2}$ where $D_v(u,u) = d(u) = \sum_{e\in\mathcal{E}} w(e)H(u,e)$ and $D_e(e,e) = \delta(e) = \sum_{v\in e} H(v,e)$. The forms of π_{zhou} and $\mathbf{L}_{zhou}$ are exactly the same in [12].

Remark 2 ([14], [15]). When we select $\rho(\cdot) = (\cdot)^\sigma$ and $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$, the probabilities of unified random walk on hypergraph are reduced to $p_1 = \frac{w(e)H(u,e)(\sum_{v\in\mathcal{V}} H(v,e))^{\sigma+1}}{\sum_{e\in\mathcal{E}} w(e)H(u,e)(\sum_{v\in\mathcal{V}} H(v,e))^{\sigma+1}}$ and $p_2 = \frac{H(v,e)}{\sum_{v\in\mathcal{V}} H(v,e)}$. As the special case of ours satisfying $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$ and $\rho(\cdot) = (\cdot)^\sigma$, [15]'s random walk satisfies both condition (1) and condition (2) in Thm. 2. So we obtain from Corollary 4.1 that $\pi_{car}(v) = \frac{d(v)}{\sum_{u\in\mathcal{V}} d(u)}$ and $\mathbf{L}_{car} = \mathbf{I} - \mathbf{D}_v^{-1/2}\mathbf{H}\mathbf{W}\mathbf{D}_e^{\sigma}\mathbf{H}^\top\mathbf{D}_v^{-1/2}$ where $D_v(u,u) = d(u) = \sum_{e\in\mathcal{E}} w(e)\delta(e)^{\sigma+1}H(u,e)$ and $\mathbf{D}_e(e,e) = \delta(e) = \sum_{v\in e} H(v,e)$. The forms of π_{car} and $\mathbf{L}_{car}$ are exactly the same in [15]. Note that the random walk proposed by [14] is a special case of [15] with $\sigma = 1$, the π and $\mathbf{L}$ are easy to obtain from our conclusions.

Remark 3 ([13]). When we set $\rho(\cdot) = (\cdot)^{-1}$ and $\mathbf{Q}_1 = \mathbf{H}$, the probabilities of unified random walk on hypergraph become $p_1 = \frac{w(e)H(u,e)}{\sum_{e\in\mathcal{E}} w(e)H(u,e)}$ and $p_2 = \frac{Q_2(v,e)}{\sum_{v\in\mathcal{V}} Q_2(v,e)}$ with an edge-dependent vertex weights $\mathbf{Q}_2$ in the second steps. Actually, as the special case of ours satisfying $\mathbf{Q}_1 = \mathbf{H}$ and $\rho(\cdot) = (\cdot)^{-1}$, [13]'s random walk on hypergraph generates the Laplacian $\mathbf{L}_{chi}$ which could build up a simple relation with our $\mathbf{L}_{rw}$ in Eq. (6) as

$$\mathbf{L}_{chi} = \Phi^{1/2}\mathbf{L}_{rw}\Phi^{1/2} = \Phi - \frac{\Phi\mathbf{P} + \mathbf{P}^\top\Phi}{2}$$

which means that $\mathbf{L}_{rw}$ denotes a form of symmetrization on $\mathbf{L}_{chi}$. Further, as the vertex weights is edge-independent in [13] which means they satisfies condition(1) in Thm. 2, the same π as [13] can be obtained from Corollary 4.1, i.e. $\pi(v) = \frac{\hat{d}(v)}{\sum_v \hat{d}(v)}$.

11 UNIFIED NON-LAZY RANDOM WALKS ON HYPERGRAPH

The non-lazy version is nontrivial to factor and analyze. To ease the studies, we first give the following definition and lemma.

Definition 6 (Reversible Markov chain). Let M be a Markov chain with state space $\mathcal{X}$ and transition probabilities $P(u, v)$, for $u, v \in \mathcal{X}$. We say M is reversible if there exists a probability distribution π over $\mathcal{X}$ such that

$$\pi(u)P(u, v) = \pi(v)P(v, u). \tag{14}$$

Lemma 2 ([13]). Let M be an irreducible Markov chain with finite state space $\mathcal{S}$ and transition probabilities $P(u, v)$ for $u, v \in \mathcal{S}$. M is reversible if and only if there exists a weighted, undirected graph $\mathcal{G}$ with vertex set $\mathcal{S}$ such that a random walk on $\mathcal{G}$ and M are equivalent.

Proof. Let π be the stationary distribution of M (suppose that is irreducible). Note that $\pi(u) \neq 0$ holds due to the irreducibility of M. Let $\mathcal{G}$ be a graph with vertices $\mathcal{S}$. Different from [13], we set the edge weights of $\mathcal{G}$ to be

$$\omega(u, v) = c\pi(u)P(u, v), \quad \forall\, u, v \in \mathcal{S} \tag{15}$$

where $c > 0$ is a constant for normalizing π. With reversibility, $\mathcal{G}$ is well-defined (i.e. $\omega(u, v) = \omega(v, u)$). In a random walk on $\mathcal{G}$, the transition probability from u to v in one time-step is: $\frac{\omega(u,v)}{\sum_{v\in\mathcal{V}}\omega(u,v)} = \frac{c\pi(u)P(u,v)}{\sum_{v\in\mathcal{V}}c\pi(u)P(u,v)} = P(u, v)$, since $\sum_{w\in\mathcal{S}} P(u, w) = 1$. Thus, if M is reversible, recall Definition 4, the stated claim holds. The other direction follows from the fact that a random walk on an undirected graph is always reversible (Aldous & Fill,[70]).

Next, we generalize the non-lazy random walk of [15] to our unified non-lazy random walk with vertex weights $\mathbf{Q}_1, \mathbf{Q}_2$. Thus, the transition matrix of a unified non-lazy random walk can be expressed as:

$$\mathbf{P}_{nl} = \mathbf{D}_{nl}^{-1}(\mathbf{Q}_1 \odot \rho(\mathbf{1}\mathbf{D}_e - \mathbf{Q}_2))\mathbf{W}\mathbf{Q}_2^{\top}, \tag{16}$$

where $\mathbf{D}_{nl}$, $\mathbf{D}_e$ are diagonal matrices with entries $D_{nl}(v, v) = d_{nl}(v)$ and $D_e(e, e) = \delta(e)$, respectively. $\odot$ denotes the Hadamard product and $\mathbf{1}$ denotes a $|\mathcal{V}| \times |\mathcal{E}|$ ones matrix(i.e. all elements equal to 1). $\rho(\mathbf{1}\mathbf{D}_e - \mathbf{Q}_2))$ represents the function ρ acting on each element of $(\mathbf{1}\mathbf{D}_e - \mathbf{Q}_2)$.

Following [13], the modified version of the clique graph without self-loops is defined below:

Definition 7 ([13]). Let $\mathcal{H}(\mathcal{V}, \mathcal{E}, \mathbf{Q})$ be a hypergraph associated with the unified non-lazy random walk in Definition 3. The clique graph of $\mathcal{H}$ without self-loops, $\mathcal{G}_{nl}^{C}$, is a weighted, undirected graph with vertex set $\mathcal{V}$, and edges $\mathcal{E}'$ defined by

$$\mathcal{E}' = \{(v, w) \in \mathcal{V} \times \mathcal{V} : v, w \in e \text{ for some } e \in \mathcal{E},\ \text{and } v \neq w\} \tag{17}$$

Different from the lazy random walk, a unified non-lazy random walk on a hypergraph with condition (1) or condition (2) in Thm. 2 is not guaranteed to satisfy reversibility (see Definition 6). However, if $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$, then reversibility holds, and we obtain the result below.

Theorem 4. Let $\mathcal{H}(\mathcal{V}, \mathcal{E}, \mathbf{W}, \mathbf{Q}_1, \mathbf{Q}_2)$ be a hypergraph associated with the unified non-lazy random walk in Definition 3. Let $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$, i.e. $Q_i(v, e) = 1, i \in \{1, 2\}$ for all vertices v incident hyperedges e. There exist weights $\omega(u, v)$ on the clique graph without self-loops $\mathcal{G}_{nl}^{C}$ such that a non-lazy random walk on $\mathcal{H}$ is equivalent to a random walk on $\mathcal{G}_{nl}^{C}$.

Proof. We first prove that the unified non-lazy random walk on $\mathcal{H}$ is reversible. It is easy to verify the stationary distribution of the unified non-lazy random walk on $\mathcal{H}$ with $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$ is $\pi(v) = \frac{d_{nl}(v)}{\sum_{v\in\mathcal{V}} d_{nl}(v)}$. Let $P(u, v)$ be the probability of going from u to v in a unified non-lazy random walk on $\mathcal{H}$, where $u \neq v$. Then,

$$\begin{aligned}
\pi(u)P(u, v) &= \frac{d_{nl}(u)}{\sum_{u\in\mathcal{V}} d_{nl}(u)} \sum_{e\in\mathcal{E}} \frac{w(e)\rho(\delta(e) - Q_2(u, e))Q_1(u, e)Q_2(v, e)}{d_{nl}(u)} \\
&= \frac{1}{\sum_{u\in\mathcal{V}} d_{nl}(u)} \sum_{e\in\mathcal{E}} (w(e)\rho(\delta(e) - Q_2(u, e))Q_1(u, e)Q_2(v, e)) \\
&= \frac{1}{\sum_{u\in\mathcal{V}} d_{nl}(u)} \sum_{e\in\mathcal{E}} (w(e)\rho(\delta(e) - H(u, e))H(u, e)H(v, e)) \\
&= \frac{1}{\sum_{u\in\mathcal{V}} d_{nl}(u)} \sum_{e\in\mathcal{E}} (w(e)\rho(\delta(e) - H(v, e))H(v, e)H(u, e)) \\
&= \pi(v)P(v, u)
\end{aligned}$$

So the unified non-lazy random walk is reversible. Thus, by Lemma 2, there exists a graph $\mathcal{G}$ with vertex set $\mathcal{V}$ and edge weights

$$\omega(u, v) = d_{nl}(u)P(u, v)$$

such that a random walk on $\mathcal{G}$ and the unified non-lazy random walk on $\mathcal{H}$ is equivalent. The equivalence of the random walks implies that $P(u, v) > 0$ if and only if $\omega(u, v) > 0$, so it follows that $\mathcal{G}$ is the clique graph of $\mathcal{H}$ without self-loops.

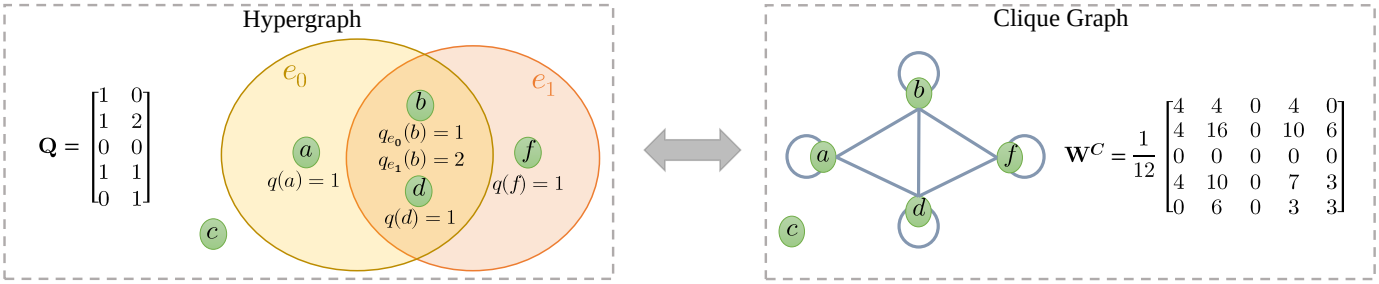


Fig. 6: An example of the disconnected hypergpraph and its equivalent weighted undirected graph. c is an isolated vertex. The characteristics of hypergraph are encoded in the weight incidence matrix $\mathbf{Q}$. The elements in $\mathbf{W}^C := \mathbf{Q}\mathbf{D}_e^{-1}\mathbf{Q}^\top$ denotes the edge-weight matrix of the clique graph.

12 Typical Hypergraph Convolution (HGNN) and Its Spectral Analysis

Recall Remark 1, the random walk and Laplaican proposed by [12] are special cases ($\rho(\cdot) = (\cdot)^{-1}, \mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$) of our unified random walk and unified Laplacian, respectively. Thus, now the degree of a vertex $v \in \mathcal{V}$ is $d(v) = \sum_{e\in\mathcal{E}} w(e)H(v,e)$ and for a hyperedge $e \in \mathcal{E}$, its degree is $\delta(e) = \sum_{v\in\mathcal{V}} H(v,e)$. The edge-degree matrix and vertex-degree matrix can be denoted as diagonal matrices $\mathbf{D}_e \in \mathbb{R}^{|\mathcal{E}|\times|\mathcal{E}|}$ and $\mathbf{D}_v \in \mathbb{R}^{|\mathcal{V}|\times|\mathcal{V}|}$ respectively.

12.1 Typical Spectral Hypergraph Convolution

From the seminal paper, [12], a classical symmetric normalized hypergraph Laplacian is defined from the perspective of spectral hypergraph partition: $\mathbf{L} = \mathbf{I} - \mathbf{D}_v^{-1/2}\mathbf{H}\mathbf{W}\mathbf{D}_e^{-1}\mathbf{H}^\top\mathbf{D}_v^{-1/2}$. Based on it, [11] follow [17] to deduce the *Hypergraph Neural Network* HGNN. Specifically, they first perform an eigenvalue decomposition of $\mathbf{L}$: $\mathbf{L} = \mathbf{U}\Lambda\mathbf{U}^\top$, where Λ is a diagonal matrix of eigenvalues and $\mathbf{U}$ consists of the corresponding regularized eigenvectors. Combining the hypergraph Laplacian with the Fourier Transform of graph signal processing, they derive the primary form of hypergraph spectral convolution: $g * x = \mathbf{U}((\mathbf{U}^\top g) \odot (\mathbf{U}^\top x)) = \mathbf{U}g(\Lambda)\mathbf{U}^\top x$ where $g(\Lambda) = diag(g(\lambda_1), g(\lambda_2), \cdots, g(\lambda_N))$ is a function of the eigenvalues of $\mathbf{L}$. To avoid expensive computation of eigen decomposition [34], they use the truncated Chebyshev polynomials $T_k(x)$ to approximated $g(\Lambda)$, which is recursively deduced by $T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x)$ with $T_0(x) = 1$ and $T_1(x) = x$. $g * x \approx \sum_{k=0}^{K-1} \theta_k T_k(\hat{\Lambda})x$, with scaled Laplacian $\hat{\Lambda} = \frac{2}{\lambda_{max}}\Lambda - \mathbf{I}$ limiting the eigenvalues in $[-1, 1]$ to guarantee requirement of the Chebyshev polynomial [100]. Since the parameters in the neural network could adapt to the scaled constant ($\lambda_{max} \approx 2$), the convolution operation is further simplified to be$g * x \approx \theta_0 x - \theta_1 \mathbf{D}_v^{-\frac{1}{2}}\mathbf{H}\mathbf{W}\mathbf{D}_e^{-1}\mathbf{H}^\top\mathbf{D}_v^{-\frac{1}{2}}x$.

However, in order to obtain the final convolutional layer similar to [17] proposed, **[11] used a specific matrix to fit the scalar parameter θ_0, causing a flaw in the derivation.** In the end, they have managed to obtain a seemingly correct hypergraph spectral convolution (HGNN):

$$\mathbf{Y} = \mathbf{D}^{-1/2}\mathbf{H}\mathbf{W}\mathbf{D}_e^{-1}\mathbf{H}^\top\mathbf{D}_v^{-1/2}\mathbf{X}\mathbf{\Theta}, \tag{18}$$

where $\mathbf{\Theta}$ is the parameter to be learned during the training process. There is no doubt that HGNN can work properly, but it could have low efficiency when the hypergraph of the datasets is disconnected. Our experiments in subsection 7.1 and Figure 6 verify our conclusion.

12.2 Spectral Analysis of HGNN

We give the definition of Rayleigh quotient to study the spectrum of Laplacian.

Lemma 3 (Rayleigh Quotient [101]). Assume that $\mathbf{M} \in \mathbb{R}^{n\times n}$ is a real symmetric matrix and $\mathbf{x} \in \mathbb{R}^n$ is an arbitrary nonzero vector. Let $\lambda_1 \le \lambda_2 \le \cdots \le \lambda_n$ denote the eigenvalues of $\mathbf{M}$ in order. Then the Rayleigh quotient satisfies the property:

$$\lambda_1 \le R(\mathbf{M}, \mathbf{x}) := \frac{\mathbf{x}^\top\mathbf{M}\mathbf{x}}{\mathbf{x}^\top\mathbf{x}} \le \lambda_n.$$

The equality holds that $R(\mathbf{M}, \mathbf{u}_1) = \lambda_1$ and $R(\mathbf{M}, \mathbf{u}_n) = \lambda_n$ where $\mathbf{u}_i$ is the eigenvector related to $\lambda_i, i \in \{1, n\}$.

The Lemma 3 is mainly used to get all exact or approximated eigenvalues and eigenvectors of $\mathbf{M}$.

In this part, we utilize it to prove that the smallest eigenvalue λ_{min} of $\mathbf{L} = \mathbf{I} - \mathbf{D}_v^{-\frac{1}{2}}\mathbf{H}\mathbf{W}\mathbf{D}_e^{-1}\mathbf{H}^\top\mathbf{D}_v^{-\frac{1}{2}}$ is 0 and figure out the corresponding eigenvector $\mathbf{u}_{min}$ is $\mathbf{D}_v^{\frac{1}{2}}\mathbf{1}$.

Specifically, let g denote an arbitrary column vector which assigns to each vertex $v \in \mathcal{V}$ a real value $g(v)$. We demonstrate $R(\mathbf{L}, g)$ as below in order to describe our conclusion distinctly:

$$\begin{aligned}\lambda_{min} \leq \frac{g^\top \mathbf{L} g}{g^\top g} &= \frac{g^\top (\mathbf{I} - \mathbf{D}_v^{-\frac{1}{2}} \mathbf{H} \mathbf{W} \mathbf{D}_e^{-1} \mathbf{H}^\top \mathbf{D}_v^{-\frac{1}{2}}) g}{g^\top g} \\ &= \frac{(\mathbf{D}_v^{\frac{1}{2}} f)^\top (\mathbf{I} - \mathbf{D}_v^{-\frac{1}{2}} \mathbf{H} \mathbf{W} \mathbf{D}_e^{-1} \mathbf{H}^\top \mathbf{D}_v^{-\frac{1}{2}}) (\mathbf{D}_v^{\frac{1}{2}} f)}{(\mathbf{D}_v^{\frac{1}{2}} f)^\top (\mathbf{D}_v^{\frac{1}{2}} f)} \\ &= \frac{f^\top (\mathbf{D}_v - \mathbf{H} \mathbf{W} \mathbf{D}_e^{-1} \mathbf{H}^\top) f}{f^\top \mathbf{D}_v f} \\ &= \frac{\sum_e \sum_v \sum_u \frac{w(e) h(v,e) h(u,e)}{\delta(e)} (f(u) - f(v))^2}{2 \sum_v f(v)^2 d(v)}\end{aligned}$$

where $g = \mathbf{D}_v^{1/2} f$ and $f \in \mathbb{R}^{|\mathcal{V}| \times 1}$. It is easy to see the fact: $R(\mathbf{L}, g) \geq 0$, and $R(\mathbf{L}, g) = 0$ if and only if $f = \mathbf{1}$ (i.e. $g = \mathbf{D}_v^{1/2} \mathbf{1}$). Then with lemma 3 we get $\lambda_{min} = \min_g R(\mathbf{L}, g) = R(\mathbf{L}, \mathbf{D}_v^{1/2} \mathbf{1}) = 0$ and $\mathbf{u}_{min} = \mathbf{D}_v^{\frac{1}{2}} \mathbf{1}$.

Actually, since the Laplacian of [12] is our unified Laplacian in Corollary 4.1, the property above can be directly led by Thm. 5.

13 Derivation and Analysis of H-GNNs

13.1 GNN induced Hypergraph NNs

We define $\mathbf{K}$ as $\mathbf{Q}_2 \mathbf{W} \rho(\mathbf{D}_e) \mathbf{Q}_2^\top$. Then the unified hypergraph Laplacian matrix $\mathbf{L}$ in Thm. 4.1 can be denoted as

$$\mathbf{L} = \mathbf{I} - \hat{\mathbf{D}}_v^{-1/2} \mathbf{K} \hat{\mathbf{D}}_v^{-1/2}, \tag{19}$$

H-GCN. Following [34], the convolutional kernel is approximated with Chebyshev polynomial to avoid eigenvalue decomposition of $\mathbf{L}$.

$$\mathbf{g} \star \mathbf{x} = \theta_0 \mathbf{x} - \theta_1 \hat{\mathbf{D}}_v^{-1/2} \mathbf{K} \hat{\mathbf{D}}_v^{-1/2} \mathbf{x}. \tag{20}$$

Then we adopt the same method proposed by [17] to set the parameter $\theta = \theta_0 = -\theta_1$ and obtain the following expression:

$$\mathbf{g} \star \mathbf{x} = \theta \left(\mathbf{I} + \hat{\mathbf{D}}_v^{-1/2} \mathbf{K} \hat{\mathbf{D}}_v^{-1/2} \right) \mathbf{x}. \tag{21}$$

According to Theorem 5, it is easy to know the eigenvalues of $\mathbf{I} + \hat{\mathbf{D}}_v^{-1/2} \mathbf{K} \hat{\mathbf{D}}_v^{-1/2}$ range in $[0, 2]$, which may cause the unstable numerical instabilities and gradient explosion problem. So we use the renormalization trick [17]:

$$\mathbf{I} + \hat{\mathbf{D}}_v^{-1/2} \mathbf{K} \hat{\mathbf{D}}_v^{-1/2} \rightarrow \tilde{\mathbf{D}}_v^{-1/2} \tilde{\mathbf{K}} \tilde{\mathbf{D}}_v^{-1/2} \triangleq \tilde{\mathbf{T}}, \tag{22}$$

where $\tilde{\mathbf{K}} = \mathbf{K} + \mathbf{I}$ can be regarded as the weighted adjacency matrix of the equivalent weighted graph with self-loops, and $\tilde{\mathbf{D}}(v, v) = \sum_u \tilde{\mathbf{K}}(v, u)$ is a $|\mathcal{V}| \times |\mathcal{V}|$ diagonal matrix. This self-loop makes the methods more robust to process disconnected hypergraph datasets. Finally, we drive the generalized hypergraph convolutional network(**H-GCN**):

$$\mathbf{X}^{(l+1)} = \psi(\tilde{\mathbf{T}} \mathbf{X}^{(l)} \mathbf{\Theta}) \tag{23}$$

$\mathbf{\Theta}$ is a learnable parameter and $\psi(\cdot)$ denotes an activation function.

There are several leading message-passing models of undirected graph which are easy to convert to the corresponding Hypergraph version under our GHSC framework.

H-APPNP.

$$\begin{aligned}&\mathbf{Z}^{(0)} = \mathbf{H} = f_\theta(\mathbf{X}); \\ &\mathbf{Z}^{(k+1)} = (1 - \alpha) \tilde{\mathbf{T}} \mathbf{Z}^{(k)} + \alpha \mathbf{H}; \\ &\mathbf{Z}^{(k)} = softmax((1 - \alpha) \tilde{\mathbf{T}}) \mathbf{Z}^{(k-1)} + \alpha \mathbf{H}),\end{aligned} \tag{24}$$

where $\mathbf{X}$ is the original node feature matrix and $\mathbf{H}$ is the feature embedding generating by a prediction neural network f_θ. Similar with H-GCN, $\tilde{\mathbf{T}}$ is the symmetrical normalized weighted adjacnecy matrix of the equivalent weighted graph with self-loops.

H-SSGC.

$$\mathbf{Z} = softmax(\frac{1}{K} \sum_{k=1}^{K} ((1 - \alpha) \tilde{\mathbf{T}}^k \mathbf{X} + \alpha \mathbf{X}) \mathbf{\Theta}), \tag{25}$$

where K is a hyperparameter for the diffusion steps and $\mathbf{\Theta}$ is the learnable parameters matrix.

H-ChebNet.

$$\mathbf{Z} = Relu(\sum_{k=0}^{K} \tilde{\mathbf{T}}^k \mathbf{X}\mathbf{\Theta}^{(k)}), \tag{26}$$

where $\mathbf{\Theta}^{(k)}(k = 0, \cdots, K)$ are the learnable paremeter matrix for K-th chebyshev Polynomial of $\tilde{\mathbf{T}}$.
H-GCNII.

$$\mathbf{Z}^{(l+1)} = \sigma(((1-\alpha_l)\tilde{\mathbf{T}}\mathbf{Z}^{(l)} + \alpha_l \mathbf{Z}^{(0)})((1-\beta_l)\mathbf{I} + \beta_l \mathbf{\Theta}^{(l)})), \tag{27}$$

where $\mathbf{Z}^{(l)}$ is the output of l-th layer of GCNII and $\mathbf{Z}^{(0)} = f_\theta(\mathbf{X})$.

Remark 4. Most of the existing hypergraph Laplacians [12], [14], [15] can be viewed as special forms of the Laplacian matrix $\mathbf{L}$ and can be derived from special cases of GHSC Framework.

13.2 Over-smoothing Analysis of GHSC

Informal Definition of over-smoothing on Hypergraphs. In our paper, the over-smoothing issue for hypergraphs means the node embeddings of hypergraphs are indistinguishable. The equivalent clique graphs have the same vertices as hypergraphs, so the over-smoothing issue for hypergraphs is defined upon the corresponding undirected graphs.

In order to formalize the analysis of over-smoothing on Hypergraphs, we further define a quantity to measure this phenomenon as follows. For real-world datasets, we always use non-Euclidean graphs with finite-dimensional features of vertices. It is difficult to use the absolute *diffusion distance* [102] which measures the distance walked from the starting vertex in the diffusion process. Therefore, we use the reciprocal of the mean l_1-norm error between t-step transition probability $\mathbf{P}^t$ and stationary distribution π as the measure of over-smoothing in arbitrary starting vertex. Formally, we name the measure by **over-smoothing energy** as follows:

$$e(i,t) = \frac{1}{\frac{1}{N}\|\mathbf{f}(i)\mathbf{P}^t - \pi\|_1}$$

where i denotes the sign of starting vertex i and t denotes the number of diffusion step. As mentioned in the Corollary. 4.2, $\mathbf{f}(i)$ is an initial distribution which means the walker diffuse starting from vertex i(i.e. $\mathbf{f}(i)_i = 1$ and $\mathbf{f}(i)_j = 0, \forall j \neq i$). It is easy to observe that if the limit of $e(i,t)$ with respect to t converges to infinity, it indicates the model based on $\mathbf{P}$ undergoes a severe over-smoothing problem. Recall Corollary 4.2, we get a low-bounded quantity of $e(i,t)$ as $e_{low}(i,t)$:

$$e(i,t) \geq \frac{N}{\sum_{j=1}^{N}(1-\lambda_H)^t \frac{\sqrt{\hat{d}(j)}}{\sqrt{\hat{d}(i)}}} = \frac{N\sqrt{\hat{d}(i)}}{(1-\lambda_H)^t \varphi(H)} = e_{low}(i,t)$$

where $\varphi(H) = \sum_{j=1}^{N}\sqrt{\hat{d}(j)}$ can be seen as a constant associated with $\mathcal{H}$ which is independent with i or t. Recall Corollary 4.2 that λ_H is smallest nonzero eigenvalue of $\mathbf{L}$.

Over-smoothing analysis of H-GCN. Here, H-GCN is used as an example of a GHSC framework-derived spectral convolutional network for the analysis of over-smoothing problems. As shown in Table 16, we can see our proposed H-GCN model also face with the over-smoothing phenomenon. Assume that we get a K-layers H-GCN model with the form: $\mathbf{X}^{(l+1)} = \mathrm{ReLU}(\tilde{\mathbf{T}}\mathbf{X}^{(l)}\mathbf{\Theta})$. To get a simple mathematical form from the above incremental convolution formula (remove the activation layer), we could get approximated K-layers H-GCN as:

$$\widetilde{\mathbf{X}}^{(K)} = \tilde{\mathbf{T}}^K \mathbf{X}^{(0)} \mathbf{\Theta}$$

Intuitively, from the simple form we find the *over-smoothing energy* of H-GCN in vertex i can be represented by $e_{low}(i,K)$. It is easy to analyze that as $K \to \infty$, $e_{low}(i,K)$ converge to infinity which implies that the *over-smoothing energy* $e(i,t)$ converge to infinity. Furthermore, the convergence of *over-smoothing energy* to infinity means the error between the initial signal and π converges to 0 as the number of layers increases, which describes the reason for over-smoothing undergone by H-GCN from a quantitative perspective. HGNN [11] also suffers from over-smoothing as shown in Table 16. Meanwhile, the theoretical analysis of over-smoothing on HGNN can be covered by the analysis of H-GCN because HGNN is a special case of H-GCN (w/o and w re-normalization).

14 Details of Theories and Proofs

14.1 Proof of Theorem 1

Theorem 1 (Equivalency between generalized hypergraph and weighted digraph). Let $\mathcal{H}(\mathcal{V}, \mathcal{E}, \mathbf{W}, \mathbf{Q}_1, \mathbf{Q}_2)$ denote the generalized hypergraph in Definition 2. There exists a weighted directed clique graph $\mathcal{G}^C$ such that the random walk on $\mathcal{H}$ is equivalent to a random walk on $\mathcal{G}^C$. Moreover, the weighted matrix of $\mathcal{G}^C$ is $\mathbf{Q}_1\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}_2^\top$.

Proof.

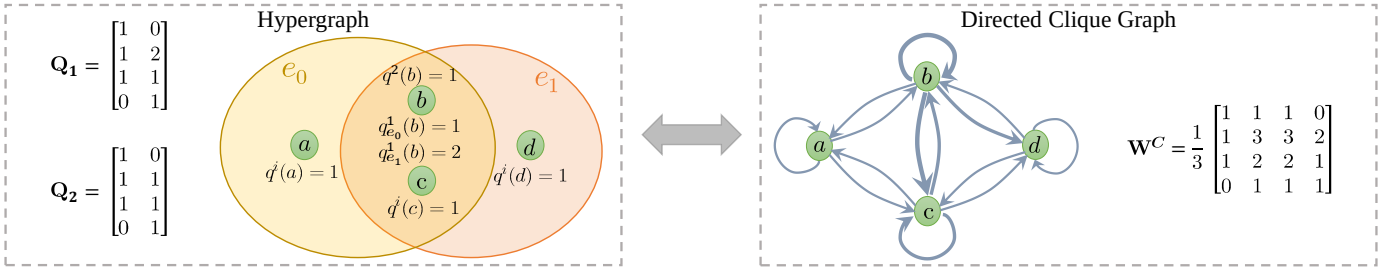


Fig. 7: An example of hypergraph and its equivalent weighted directed graph, where $q^i(\cdot) = Q_i(\cdot, e), i \in \{1, 2\}$. Here, $\mathbf{Q}_1$ is edge-dependent and $\mathbf{Q}_2$ is edge-independent. $\mathbf{W}^C := \mathbf{Q}_1\mathbf{D}_e^{-1}\mathbf{Q}_2^\top$ denotes the edge-weight matrix of the clique graph.

Let $\omega(u, v)$ be the weight of edge (u, v) in $\mathcal{G}^C$. We set $\omega(u, v) = d(u)P(u, v)$ in Definition 1, then the weighted clique graph is a directed graph, where $P(u, v)$ is the transition probabilities in Eq Eq. (3). The random walk on the directed weighted $\mathcal{G}^C$ with transition probabilities

$$P^C(u, v) = \frac{\omega(u, v)}{\sum_{v\in\mathcal{V}} \omega(u, v)} = \frac{d(u)P(u, v)}{\sum_{v\in\mathcal{V}} d(u)P(u, v)} = P(u, v)$$

is equivalent to $\mathcal{H}$ with transition probabilities $\mathbf{P}(u, v)$. The matrix form of edge weight of $\mathcal{G}^C$ is :

$$\mathbf{W}^C = \mathbf{Q}_1\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}_2^\top$$

Suppose $\mathbf{W} = \mathbf{I}$ and let $\rho(\cdot) = (\cdot)^{-1}$, we can obtain the equivalent weighted clique graph shown in Figure 7.

14.2 Proof of Lemma 1

Lemma 1. Let $\mathcal{H}(\mathcal{V}, \mathcal{E}, \mathbf{W}, \mathbf{Q}_1, \mathbf{Q}_2)$ denote the generalized hypergraph in Definition 2. Let $\mathcal{F}_{(Q_1,Q_2)}(u, v) := \sum_{e\in\mathcal{E}} w(e)\rho(\delta(e))Q_1(u, e)Q_2(v, e)$ and $\mathcal{T}_{(Q)}(u) := \sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(\delta(e))Q(u, e)$. When $\mathbf{Q}_1, \mathbf{Q}_2$ satisfies the following equation

$$\begin{aligned}&\mathcal{T}_{(Q_2)}(u)\mathcal{T}_{(Q_1)}(v)\mathcal{F}_{(Q_1,Q_2)}(u, v)\\ &= \mathcal{T}_{(Q_2)}(v)\mathcal{T}_{(Q_1)}(u)\mathcal{F}_{(Q_1,Q_2)}(v, u), \forall u, v \in \mathcal{V}\end{aligned} \tag{5}$$

there exists a weighted undirected clique graph $\mathcal{G}^C$ such that a random walk on $\mathcal{H}$ is equivalent to a random walk on $\mathcal{G}^C$ with edge weights $\omega(u, v) = \mathcal{T}_{(Q_2)}(u)\mathcal{F}_{(Q_1,Q_2)}(u, v)/\mathcal{T}_{(Q_1)}(u)$ if $\mathcal{T}_{(Q_1)}(u) \neq 0$ and 0 otherwise.

And the symmetrical Laplacian of $\mathcal{G}^C$ is

$$L(u, v) = \mathcal{I}_{\{u=v\}} - \mathcal{T}_{(Q_2)}(u)^{-1/2}\omega(u, v)\mathcal{T}_{(Q_2)}(v)^{-1/2} \tag{28}$$

where $\mathcal{I}_{\{\cdot\}}$ denotes the indicator function.

Proof.

Form Lemma 2, we just need to prove Markov chain with state space $\mathcal{V}$ is reversible. The transition probabilities of the unified random walk on $\mathcal{H}$ are:

$$P(u, v) = \sum_{e\in\mathcal{E}} \frac{w(e)\rho(\delta(e))Q_1(u, e)Q_2(v, e)}{d(u)} = \frac{\mathcal{F}_{(Q_1,Q_2)}(u, v)}{\mathcal{T}_{(Q_1)}(u)}$$

By Eq. (5), we have:

$$\mathcal{T}_{(Q_2)}(u)\frac{\mathcal{F}_{(Q_1,Q_2)}(u, v)}{\mathcal{T}_{(Q_1)}(u)} = \mathcal{T}_{(Q_2)}(v)\frac{\mathcal{F}_{(Q_1,Q_2)}(v, u)}{\mathcal{T}_{(Q_1)}(v)},$$

i.e.

$$\mathcal{T}_{(Q_2)}(u)P(u, v) = \mathcal{T}_{(Q_2)}(v)P(v, u),$$

Comparing with Eq. (14), we normalize $\mathcal{T}_{(Q_2)}(u)$ to define the stationary distribution of M:

$$\pi(u) := \frac{\mathcal{T}_{(Q_2)}(u)}{\sum_{u\in\mathcal{V}} \mathcal{T}_{(Q_2)}(u)}, \forall u \in \mathcal{V}, \tag{29}$$

which can be easy to verify by $\sum_u \pi(u)P(u, v) = \pi(v)$. Thus, $\pi(u)P(u, v) = \pi(v)P(v, u)$, which suggests that M is reversible. By Lemma 2, the edge weight $\omega(u, v)$ of $\mathcal{G}^C$ is

$$\omega(u, v) = c\pi(u)P(u, v) = \mathcal{T}_{(Q_2)}(u)P(u, v) = \frac{\mathcal{T}_{(Q_2)}(u)}{\mathcal{T}_{(Q_1)}(u)}\mathcal{F}_{(Q_1,Q_2)}(u, v)$$

To gain the Laplacian of $\mathcal{G}^C$, we calculate the sum of row of edge weights matrix,

$$\sum_{v\in\mathcal{V}}\omega(u,v)=\frac{\mathcal{T}_{(Q_2)}(u)}{\mathcal{T}_{(Q_1)}(u)}\sum_{v\in\mathcal{V}}\mathcal{F}_{(Q_1,Q_2)}(u,v)=\mathcal{T}_{(Q_2)}(u)$$

Then we get the symmetrical Laplacian of $\mathcal{G}^C$:

$$L(u,v)=\mathcal{I}_{\{u=v\}}-\mathcal{T}_{(Q_2)}(u)^{-\frac{1}{2}}\omega(u,v)\mathcal{T}_{(Q_2)}(v)^{-\frac{1}{2}} \tag{30}$$

14.3 Proof of Theorem 2

Theorem 2 (Equivalency between generalized hypergraph and weighted undigraph). Let $\mathcal{H}(\mathcal{V},\mathcal{E},\mathbf{W},\mathbf{Q}_1,\mathbf{Q}_2)$ denote the generalized hypergraph in Definition 2. When $\mathcal{H}$ satisfies any of the conditions below:
Condition (1): $\mathbf{Q}_1$ and $\mathbf{Q}_2$ are both edge-independent;
Condition (2): $\mathbf{Q}_1=k\mathbf{Q}_2$ $(k\in\mathbb{R})$,
there exists a weighted undirected clique graph $\mathcal{G}^C$ such that a random walk on $\mathcal{H}$ is equivalent to a random walk on $\mathcal{G}^C$.

Proof. 1) Proof based on Lemma 1 For condition (1),

$$\begin{aligned}\mathcal{F}_{(Q_1,Q_2)}(u,v)&=\sum_{e\in\mathcal{E}}w(e)\rho(\delta(e))Q_1(u,e)Q_2(v,e)=q_1(u)q_2(v)\sum_{e\in\mathcal{E}}w(e)\rho(\delta(e))H(u,e)H(v,e)\\ \mathcal{T}_{(Q_i)}(u)&=\sum_{e\in\mathcal{E}}w(e)\delta(e)\rho(\delta(e))Q_i(u,e)=q_i(u)\sum_{e\in\mathcal{E}}w(e)\delta(e)\rho(\delta(e))H(u,e),\ i=\{1,2\}\end{aligned} \tag{31}$$

Substituting the above equations into Eq. (5), then Eq. (5) holds.

For condition (2),

$$\begin{aligned}\mathcal{F}_{(Q_1,Q_2)}(u,v)&=\sum_{e\in\mathcal{E}}w(e)\rho(\delta(e))Q_1(u,e)Q_2(v,e)=k\sum_{e\in\mathcal{E}}w(e)\rho(\delta(e))Q_2(u,e)Q_2(v,e)\\ \mathcal{T}_{(Q_1)}(u)&=\sum_{e\in\mathcal{E}}w(e)\delta(e)\rho(\delta(e))Q_1(u,e)=k\sum_{e\in\mathcal{E}}w(e)\delta(e)\rho(\delta(e))Q_2(u,e)=k\mathcal{T}_{(Q_2)}(u)\end{aligned} \tag{32}$$

Similarly, substituting the above equations into Eq. (5), then Eq. (5) holds.

2) Proof based on Lemma 2

I) For Condition (1):

Because $\mathbf{Q}_1$ and $\mathbf{Q}_2$ are both edge-independent, we set $Q_1(u,e)=q_1(u)$ and $Q_2(v,e)=q_2(v)$ for all hyperedge e. From Lemma 2, the key is to prove the unified hypergraph random walk on $\mathcal{H}$ under condition (1) is reversible (see Definition 6). It is hard to find the explicit form of π before we get Corollary 2.1. Fortunately, by Kolmogorov's criterion, that is equal to prove:

$$p_{v_1,v_2}p_{v_2,v_3}\cdots p_{v_n,v_1}=p_{v_1,v_n}p_{v_n,v_{n-1}}\cdots p_{v_2,v_1}$$

for arbitrary subset $\{v_1,\cdots,v_n\}\subseteq\mathcal{V}$. The transition probabilities of the generalized random walk on $\mathbf{H}$ are:

$$\begin{aligned}p(u,v)&=\sum_{e\in\mathcal{E}}\frac{w(e)\rho(\delta(e))Q_1(u,e)Q_2(v,e)}{d(u)}=\frac{q_1(u)q_2(v)}{d(u)}\sum_{e\in\mathcal{E}}w(e)\rho(\delta(e))H(u,e)H(v,e)\\ &=\frac{q_1(u)q_2(v)}{d(u)}\sum_{e\in E(u,v)}w(e)\rho(\delta(e))\end{aligned}$$

where $E(u,v)=\{e\in\mathcal{E}:u\in e,\ v\in e\}$ to be the set of hyperedges incident to both v and u. Then we have:

$$\begin{aligned}&p_{v_1,v_2}\cdots p_{v_n,v_1}\\ &=\left(\frac{q_1(v_1)q_2(v_2)}{d(v_1)}\sum_{e\in E(v_1,v_2)}w(e)\rho(\delta(e))\right)\cdots\left(\frac{q_1(v_n)q_2(v_1)}{d(v_n)}\sum_{e\in E(v_n,v_1)}w(e)\rho(\delta(e))\right)\\ &=\left(\prod_{i=1}^{n}\frac{q_1(v_i)q_2(v_i)}{d(v_i)}\right)\cdot\left(\prod_{i=1}^{n}\sum_{e\in E(v_i,v_{i+1})}w(e)\rho(\delta(e))\right),\ \text{where set } v_{n+1}=v_1\\ &=\left(\frac{q_1(v_1)q_2(v_n)}{d(v_1)}\sum_{e\in E(v_n,v_1)}w(e)\rho(\delta(e))\right)\cdots\left(\frac{q_1(v_2)q_2(v_1)}{d(v_2)}\sum_{e\in E(v_1,v_2)}w(e)\rho(\delta(e))\right)\\ &=p_{v_1,v_n}\cdots p_{v_2,v_1}\end{aligned}$$

So the unified random walk on $\mathcal{H}$ under condition (1) is reversible.

As a matter of fact, we can succinctly obtain the reversibility via the stationary distribution. Specifically, we can verify the stationary distribution is

$$\pi(u) = \frac{\sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(\delta(e))Q_2(u,e)}{\sum_{v\in\mathcal{V}}\sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(\delta(e))Q_2(v,e)} \tag{33}$$

by $\sum_u \pi(u)P(u,v) = \pi(v)$. Then with $\pi(u)P(u,v) = \pi(v)P(v,u)$ holding, we know our unified random walk on $\mathcal{H}$ under condition (1) is reversible. Since $\mathcal{H}$ is connected, the unified random on $\mathcal{H}$ is irreducible. From Lemma.2, We know that the current unified random walk on $\mathcal{H}$ is equivalent to a random walk on an undirected graph $\mathcal{G}$ with vertex set $\mathcal{V}$. By Eq. (15), the edge weights of $\mathcal{G}$ are:

$$\omega(u,v) = c\pi(u)P(u,v) = \sum_{e\in\mathcal{E}} w(e)\rho(\delta(e))Q_2(u,e)Q_2(v,e) \tag{34}$$

where $c = \sum_{v\in\mathcal{V}}\sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(\delta(e))Q_2(v,e)$. Note that $P(u,v) > 0$ if and only if $\omega(u,v) > 0$, so $\mathcal{G}$ is the clique graph of $\mathcal{H}$.

II) For condition (2) that $\mathbf{Q}_1 = k\mathbf{Q}_2$,
we have $\mathbf{P} = k\mathbf{D}_v^{-1}\mathbf{Q}_2\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}_2^\top$. Thanks to the underlying symmetry adjacency matrix of $\mathbf{P}$ (i.e. $\mathbf{Q}_2\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}_2$ is symmetrical), we have: $\mathbf{1}^\top\hat{D}_v\mathbf{P} = \mathbf{1}^\top\hat{D}_v^\top$. Thus, the stationary distribution is

$$\pi(u) = \frac{\hat{d}(u)}{\sum_v \hat{d}(v)} = \frac{\sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(\delta(e))Q(u,e)}{\sum_{v\in\mathcal{V}}\sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(\delta(e))Q(v,e)} \tag{35}$$

Further, we would prove that the unified random walk on $\mathcal{H}$ under current condition (2) is reversible.

$$\begin{aligned}\pi(u)p(u,v) &= \frac{\hat{d}(u)}{\sum_{v'}\hat{d}(v')}\sum_{e\in\mathcal{E}}\frac{w(e)\rho(\delta(e))Q(u,e)Q(v,e)}{d(u)} \\ &= \frac{d(v)}{\sum_{v'}d(v')}\sum_{e\in\mathcal{E}}\frac{w(e)\rho(\delta(e))Q(v,e)Q(u,e)}{d(v)} = \pi(v)p(v,u)\end{aligned}$$

So the unified hypergraph random walk on $\mathcal{H}$ under condition (2) is reversible. Since $\mathcal{H}$ is connected, the unified random on $\mathcal{H}$ is irreducible. From Lemma.2, We know that the current unified random walk is equivalent to a random walk on a weighted, undirected graph $\mathcal{G}$ with vertex set $\mathcal{V}$. By Eq. (15), the edge weights of $\mathcal{G}$ are:

$$\omega(u,v) = c\pi(u)P(u,v) = \sum_{e\in\mathcal{E}} w(e)\rho(\delta(e))Q(u,e)Q(v,e) \tag{36}$$

where $c = \sum_{v'} d(v') = \sum_{v\in\mathcal{V}}\sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(\delta(e))Q(v,e)$. Note that $P(u,v) > 0$ if and only if $\omega(u,v) > 0$, so $\mathcal{G}$ is the clique graph of $\mathcal{H}$.

On the whole, we get the specific equivalent weighted undigraphs under our condition (1) and condition (2). Note that they have the same edge weights expression (Eq. (34),Eq. (36)):

$$\mathbf{W}^C = \mathbf{Q}_2\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}_2^\top$$

which further suggests the connection between condition (1) and condition (2) stated in Corollary 2.1 and Thm. 4.1.
Remark: Let $\rho(\cdot) = (\cdot)^{-1}$ and $\mathbf{W} = \mathbf{I}$, we can get the undigraph shown in Figure 3.

14.4 Nonequivalent Condition

Theorem 3. There at least exists one generalized hypergraph $\mathcal{H}(\mathcal{V},\mathcal{E},\mathbf{W},\mathbf{Q}_1,\mathbf{Q}_2)$ (Definition 2) with edge-dependent weights and $\mathbf{Q}_1 \neq k\mathbf{Q}_2$, such that a random walk on $\mathcal{H}$ is not equivalent to a random walk on its undirected clique graph $\mathcal{G}^C$ for any choice of edge weights on $\mathcal{G}^C$.

Proof. When the vertex weights are edge-dependent, the reversibility is not always satisfied in our unified random walk on a hypergraph. By Lemma 2, if the unified random walk is not time-reversible, it could not be equivalent to random walks on digraphs for any choice of edge weights.

An irreversible example can be seen in Figure 1. The probability transition matrix of the unified random walk on the left hypergraph (with $\mathbf{W} = \mathbf{I}, \rho(\cdot) = (\cdot)^{-1}$) is given below:

$$\mathbf{P} = \mathbf{D}_v^{-1}\mathbf{Q}_1\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}_2^\top = \mathbf{D}_v^{-1}\mathbf{Q}_1\mathbf{D}_e^{-1}\mathbf{Q}_2^\top = \begin{bmatrix}\frac{1}{3} & \frac{1}{3} & \frac{1}{3} & 0\\ \frac{1}{6} & \frac{5}{12} & \frac{7}{24} & \frac{1}{8}\\ \frac{1}{6} & \frac{5}{12} & \frac{7}{24} & \frac{1}{8}\\ 0 & \frac{1}{2} & \frac{1}{3} & \frac{1}{3}\end{bmatrix}, \tag{37}$$

and the stationary distribution of the unified random walk on hypergraph with $\mathbf{P}$ is:

$$\pi = [\frac{3}{17}, \frac{7}{17}, \frac{5}{17}, \frac{2}{17}] \tag{38}$$

We verify $\pi(1)P(1,2) = \frac{3}{17} \times \frac{1}{3} \neq \frac{7}{17} \times \frac{1}{6} = \pi(2)P(2,1)$, which indicates that the Markov chain represented by the unified random walk in Figure 1 with transition probabilities $\mathbf{P}$ is irreversible.

14.5 Failure Condition of Function ρ.

As a matter of fact, the ρ in our random walk framework can not always work and we give its failure condition as follows.

Proposition 1. $\rho(\cdot)$ fails to work in the unified random walk (Definition 1) if the hyperedge degrees are edge-independent (i.e. $\delta(e) = \delta(e'), \forall e' \in \mathcal{E}$).

Proof. Given $\delta(e)$ are edge-independent, let us suppose $\delta(e) = k$, then

$$\begin{aligned} P(u,v) &= \sum_{e\in\mathcal{E}} \frac{w(e)Q_1(u,e)Q_2(v,e)\rho(\delta(e))}{d(u)} = \sum_{e\in\mathcal{E}} \frac{w(e)Q_1(u,e)Q_2(v,e)\rho(\delta(e))}{\sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(\delta(e))Q_1(v,e)} \\ &= \sum_{e\in\mathcal{E}} \frac{w(e)Q_1(u,e)Q_2(v,e)\rho(k)}{\sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(k)Q_1(v,e)} = \sum_{e\in\mathcal{E}} \frac{\rho(k)w(e)Q_1(u,e)Q_2(v,e)}{\rho(k)\sum_{e\in\mathcal{E}} w(e)\delta(e)Q_1(v,e)} \\ &= \sum_{e\in\mathcal{E}} \frac{w(e)Q_1(u,e)Q_2(v,e)}{\sum_{e\in\mathcal{E}} w(e)\delta(e)Q_1(v,e)} \end{aligned}$$

which indicates $P(u,v)$ is independent to $\rho(\cdot)$, i,e. $\rho(\cdot)$ fails to work.

It is obvious that the k-uniform hypergraph has edge-independent hyperedge degrees. Formally, we have the following proposition.

Proposition 2. Let $\mathcal{H}(\mathcal{V}, \mathcal{E}, \mathbf{W}, \mathbf{Q}_1, \mathbf{Q}_2)$ denote the generalized hypergraph in Definition 2 with $\rho(\cdot) = (\cdot)^\sigma$ and $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$, then $\mathcal{H}$ is a k-uniform hypergraph(i.e each hyperedge contains the same number of vertices) if and only if $P(u,v)$ in Eq. (3) is independent with σ.

Proof.

For $\mathbf{Q} = \mathbf{H}$, then $\delta(e) = \sum_{v\in e} h(v,e) = k$.
So, for all $u, v \in \mathcal{V}$, we have:

$$K^{(\sigma)}(u,v) := \sum_{e\in\mathcal{E}} w(e)(\delta(e))^\sigma q_e(u)q_e(v) = k^\sigma \sum_{e\in\mathcal{E}} w(e)H(u,e)H(v,e) \tag{39}$$

Then we get the entries of the transition matrix $\mathbf{P}^{(\sigma)}$:

$$P^{(\sigma)}(u,v) := \frac{K^{(\sigma)}(u,v)}{\sum_b K^{(\sigma)}(u,b)} = \frac{k^\sigma \sum_{e\in\mathcal{E}} w(e)H(u,e)H(v,e)}{\sum_{b\in\mathcal{V}} k^\sigma \sum_{e\in\mathcal{E}} w(e)H(u,e)H(b,e)} \tag{40}$$

$$= \frac{\sum_{e\in\mathcal{E}} w(e)H(u,e)H(v,e)}{\sum_{b\in\mathcal{V}} \sum_{e\in\mathcal{E}} w(e)H(u,e)H(b,e)} \tag{41}$$

It is easy to see that $P^{(\sigma)}(u,v)$ is independent with σ.
The other direction, we find when $P^{(\sigma)}(u,v)$ in is independent with σ,then:

$$\frac{\partial P^{(\sigma)}(u,v)}{\partial\sigma} = \frac{\frac{\partial d(u)}{\partial\sigma}K^{(\sigma)}(u,v) - \frac{\partial K^{(\sigma)}(u,v)}{\partial\sigma}d(u)}{d^2(u)} \tag{42}$$

$$= \frac{\sum_b \sum_e \sum_{e'} w(e)w(e')h(u,e)h(u,e')h(v,e')h(b,e)\delta(e)^\sigma\delta(e')^\sigma(\ln(\delta(e)) - \ln(\delta(e')))}{d^2(u)} \tag{43}$$

$$= 0, \forall\ u, v \in \mathcal{V} \tag{44}$$

It is equal to $\exists k \in \mathbf{Z}^{++}$ s.t.:

$$\delta(e) = \delta(e') = k\ for\ all\ e \in \mathcal{E}$$

.i.e. the hypergraph is k-uniform. With the condition that $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{H}$, our unified random walk on hypergraph reduces to a random walk on hypergraph that leaving only incident relationships between hyperedges and vertices. Actually, the reduced random walk mentioned is a common structure in real-world applications. The proposition is to say when the hypergraph based on the reduced random walk is uniform, the function $\rho(\cdot)$ does not impact the importance between vertices anymore.

14.6 Proof of Corollary 4.1.

Before proving the Corollary 4.1, we further illustrate the fact that Lemma 2.1 states.

Corollary 2.1. Let $\mathcal{H}(\mathcal{V}, \mathcal{E}, \mathbf{W}, \mathbf{Q}_1, \mathbf{Q}_2)$ denote the generalized hypergraph in Definition 2. Given a $\mathcal{H}$ satisfying condition (1) in Thm. 2, there exists a $\mathcal{H}'$ satisfying the condition (2) such that the random walk on $\mathcal{H}$ is equivalent to that on $\mathcal{H}'$.

Proof of Lemma 2.1. When the condition (2) in Thm. 2 holds (i.e. $\mathbf{Q}_1 = \mathbf{Q}_2$), the transition probabilities of the unified random walk on $\mathcal{H}$ are

$$P^{c2}(u, v) = \frac{\sum_{e\in\mathcal{E}} w(e)\rho(\delta(e))Q_2(u, e)Q_2(v, e)}{\sum_{e\in\mathcal{E}} w(e)\rho(\delta(e)\delta(e)Q_2(u, e)} \tag{45}$$

Meanwhile, if the condition (1) in Thm. 2 holds (i.e. $\mathbf{Q}_1, \mathbf{Q}_2$ are both edge-independent), for all hyperedge e, we represent $Q_1(u, e)$, $Q_2(v, e)$ as $q_1(u)$, $q_2(v)$ respectively, and the transition probabilities are

$$\begin{aligned} P^{c1}(u, v) &= \frac{\sum_{e\in\mathcal{E}} w(e)\rho(\delta(e))Q_1(u, e)Q_2(v, e)}{\sum_{e\in\mathcal{E}} w(e)\rho(\delta(e)\delta(e)Q_1(u, e)} \\ &= \frac{q_1(u)q_2(v)\sum_{e\in\mathcal{E}} w(e)\rho(\delta(e))H(u, e)H(v, e)}{q_1(u)\sum_{e\in\mathcal{E}} w(e)\rho(\delta(e)\delta(e)H(u, e)} \qquad (46) \\ &= \frac{q_2(u)q_2(v)\sum_{e\in\mathcal{E}} w(e)\rho(\delta(e))H(u, e)H(v, e)}{q_2(u)\sum_{e\in\mathcal{E}} w(e)\rho(\delta(e)\delta(e)H(u, e)} \qquad (47) \\ &= \frac{\sum_{e\in\mathcal{E}} w(e)\rho(\delta(e))Q_2(u, e)Q_2(v, e)}{\sum_{e\in\mathcal{E}} w(e)\rho(\delta(e)\delta(e)Q_2(u, e)} \qquad (48) \end{aligned}$$

It can be observed that $P^{c1}(u, v) = P^{c2}(u, v), \forall u, v \in \mathcal{V}$. By definition 4, the Lemma 2.1 holds.

Next, based on Lemma 2.1, we prove the following main conclusion.

Corollary 4.1. Let $\mathcal{H}(\mathcal{V}, \mathcal{E}, \mathbf{W}, \mathbf{Q}_1, \mathbf{Q}_2)$ be the generalized hypergraph in Definition 2. Let $\hat{\mathbf{D}}_v$ be a $|\mathcal{V}| \times |\mathcal{V}|$ diagonal matrix with entries $\hat{D}_v(v, v) := \hat{d}(v) := \sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(\delta(e))Q_2(v, e)$. No matter $\mathcal{H}$ satisfies condition (1) or condition (2) in Thm. 2, it obtains the unified explicit form of stationary distribution π and Laplacian matrix $\mathbf{L}$ as:

$$\pi = \frac{\mathbf{1}^\top \hat{\mathbf{D}}_v}{\mathbf{1}^\top \hat{\mathbf{D}}_v \mathbf{1}} \text{ and } \mathbf{L} = \mathbf{I} - \hat{\mathbf{D}}_v^{-1/2}\mathbf{Q}_2\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}_2^\top \hat{\mathbf{D}}_v^{-1/2}. \tag{7}$$

Proof of Corollary 4.1.

1) Proof based on Lemma 1 When the generalized hypergraph satisfies Theorem 2, we obtain the Eq. (31) and Eq. (32) in Appendix 14.2. substituting the equations into Eq. (30) and Eq. (29), we can obtain the Corollary 4.1.

2) Proof based on Lemma 2.1

We know whether $\mathcal{H}$ satisfying condition (1) or condition (2) in Theorem 2, there exists a weighted undirected clique graph $\mathcal{G}^C$ equivalent to $\mathcal{H}$. Recall Lemma 2.1, when $\mathcal{H}$ satisfies any of the two conditions, the probability transition matrix $\mathbf{P}$ of $\mathcal{H}$ can be denoted as:

$$P(u, v) = \frac{\sum_{e\in\mathcal{E}} w(e)\rho(\delta(e))Q_2(u, e)Q_2(v, e)}{\sum_{e\in\mathcal{E}} w(e)\rho(\delta(e)\delta(e)Q_2(u, e)}$$

Then, we obtain the unified form of a probability transition matrix $\mathbf{P}$ under any of the two conditions:

$$\mathbf{P} = \hat{\mathbf{D}}_v^{-1}\mathbf{Q}_2\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}_2^\top \tag{49}$$

where $\hat{\mathbf{D}}_v$ is a $|\mathcal{V}| \times |\mathcal{V}|$ diagonal matrix with entries $\hat{D}_v(v, v) = \sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(\delta(e))Q_2(v, e)$ Remark the $\mathbf{P}$ is also the probability transition matrix of the equivalent undigraph in Theorem 2. Due to the symmetric adjacency matrix underlying $\mathbf{P}$ (i.e. $\mathbf{Q}_2\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}_2^\top$ is symmetrical), it is easy to derive the unified stationary distribution π by $\mathbf{1}^\top\hat{\mathbf{D}}_v\mathbf{P} = \mathbf{1}^\top\hat{\mathbf{D}}_v$. Thus, the stationary distribution is:

$$\pi = \frac{\mathbf{1}^\top \hat{\mathbf{D}}_v}{\mathbf{1}^\top \hat{\mathbf{D}}_v \mathbf{1}} \quad \left(\pi(u) = \frac{\hat{d}(u)}{\sum_v \hat{d}(v)} = \frac{\sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(\delta(e))Q_2(u, e)}{\sum_v \sum_{e\in\mathcal{E}} w(e)\delta(e)\rho(\delta(e))Q_2(v, e)}\right) \tag{50}$$

Substitute $\mathbf{P}$ and π into Eq. (6), we finally get the unified hypergraph Laplacian $\mathbf{L}$ as:

$$\mathbf{L} = \mathbf{I} - \hat{\mathbf{D}}_v^{-1/2}\mathbf{Q}_2\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}_2^\top \hat{\mathbf{D}}_v^{-1/2}$$

Remark. The stationary distribution with edge-independent vertex weights in [13] can be seen as a special case of π in Eq. (50) by setting $\mathbf{Q}_1 = \mathbf{H}, Q_2(u, e) = \gamma(u)$ and $\rho(\cdot) = (\cdot)^{-1}$:

$$\pi^{Ch}(u) = \frac{\gamma(u)\sum_{e\in\mathcal{E}} w(e)H(u, e)}{\sum_v \gamma(v)\sum_{e\in\mathcal{E}} w(e)H(v, e)} = \frac{\gamma(u)d_{Ch}(u)}{\sum_v \gamma(v)d_{Ch}(v)}$$

where $d_{Ch}(u) = \sum_{e\in\mathcal{E}} w(e)h(u, e)$ is the definition of vertex degree in [13].

14.7 Spectral Range of Laplacian Matrix

The following Theorem proves the upper bound for eigenvalues of $\mathbf{L}$ which meets the requirement of Chebyshev polynomials to enable the derivation for our proposed H-GCN(a simple case of GHSC) in Appendix 13.1 and leads to Corollary . 4.2 in the paper.

Theorem 5. Let $\mathbf{L} = \mathbf{I} - \hat{\mathbf{D}}_v^{-1/2}\mathbf{Q}_2\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}_2^\top\hat{\mathbf{D}}_v^{-1/2}$ denote the unified hypergraph Laplacian matrix in Corollary 4.1 and $\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$ denote the eigenvalues of $\mathbf{L}$ in order.

1) $\lambda_{min} = \lambda_1 = 0$ and $\mathbf{u}_1 = \hat{\mathbf{D}}_v^{\frac{1}{2}}\mathbf{1}$(the eigenvector associated with λ_1)
2) For $k = 2, 3, \cdots, n$, we have

$$\lambda_k = \inf_{f \perp \hat{\mathbf{D}}_v^{\frac{1}{2}} S_{k-1}} \frac{\sum_{e,u,v}\beta(e,u,v)(f(u)-f(v))^2}{\sum_v f(v)^2 d(v)},$$

here, S_{k-1} is the subspace spanned by eigenvectors $\{\mathbf{u}_1, \cdots, \mathbf{u}_{k-1}\}$ where $\mathbf{u}_i$ related to λ_i and $\beta(e,u,v) = w(e)\rho(\delta(e))Q_2(v,e)Q_2(u,e)$.
3) $\lambda_{max} = \lambda_n \leq 2$

Proof: To analyze the generalized Laplacian proposed in our work, we use the Rayleigh quotient mentioned in the Lemma 3 to give out some conclusions about the eigenvalues and eigenvectors of the Laplacian.
1) Deduce the Rayleigh quotient of $\mathbf{L}_2$:

$$\begin{aligned}
\frac{g^\top\mathbf{L}g}{g^\top g} &= \frac{g^\top(\mathbf{I} - \hat{\mathbf{D}}_v^{-\frac{1}{2}}\mathbf{Q}_2\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}_2^\top\hat{\mathbf{D}}_v^{-\frac{1}{2}})g}{g^\top g} \\
&= \frac{(\hat{\mathbf{D}}_v^{\frac{1}{2}}f)^\top(\mathbf{I} - \hat{\mathbf{D}}_v^{-\frac{1}{2}}\mathbf{Q}_2\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}_2^\top\hat{\mathbf{D}}_v^{-\frac{1}{2}})(\hat{\mathbf{D}}_v^{\frac{1}{2}}f)}{(\hat{\mathbf{D}}_v^{\frac{1}{2}}f)^\top(\hat{\mathbf{D}}_v^{\frac{1}{2}}f)} \\
&= \frac{f^\top(\hat{\mathbf{D}}_v - \mathbf{Q}\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}^\top)f}{f^\top\hat{\mathbf{D}}_v f} \\
&= \frac{\sum_u\sum_v\sum_e w(e)\rho(\delta(e))Q_2(v,e)Q_2(u,e)(f(u)-f(v))^2}{2\sum_v f(v)^2\hat{d}(v)}
\end{aligned}$$

where $g = \hat{\mathbf{D}}_v^{1/2}f$ and $f \in \mathbb{R}^{|\mathcal{V}|\times 1}$. It is easy to see the fact: $R(\mathbf{L}, g) \geq 0$ and $R(\mathbf{L}, g) = 0$ if and only if $f = \mathbf{1}$ (i.e. $g = \hat{\mathbf{D}}_v^{1/2}\mathbf{1}$), Then with Lemma 3 we get $\lambda_1 = \lambda_{min} = \min_g R(\mathbf{L}, g) = R(\mathbf{L}, \hat{\mathbf{D}}_v^{1/2}\mathbf{1}) = 0$ and $\mathbf{u}_1 = \hat{\mathbf{D}}_v^{\frac{1}{2}}\mathbf{1}$.
2) From Courant–Fischer theorem [101], we have:

$$\begin{aligned}
\lambda_k &= \inf_{g \perp S_{k-1}} R(\mathbf{L}, g) = \inf_{g \perp S_{k-1}} \frac{g^\top\mathbf{L}g}{g^\top g} \\
&= \inf_{f \perp \hat{\mathbf{D}}_v^{\frac{1}{2}} S_{k-1}} \frac{\sum_u\sum_v\sum_e w(e)\rho(\delta(e))Q_2(v,e)Q_2(u,e)(f(u)-f(v))^2}{2\sum_v f(v)^2\hat{d}(v)}
\end{aligned}$$

3) We have the facts:

$$(f(u) - f(v))^2 \leq 2(f^2(u) + f^2(v))$$

And from Lemma 3, we get:

$$\begin{aligned}
\lambda_n &= \lambda_{max} = R(\mathbf{L}, \mathbf{u}_{max}) \\
&= \inf_{f \perp \hat{\mathbf{D}}_v^{\frac{1}{2}} S_{n-1}} \frac{\sum_u\sum_v\sum_e w(e)\rho(\delta(e))Q_2(u,e)Q_2(v,e)(f(u)-f(v))^2}{2\sum_v f(v)^2\hat{d}(v)} \\
&\leq \inf_{f \perp \hat{\mathbf{D}}_v^{\frac{1}{2}} S_{n-1}} \frac{\sum_u\sum_e w(e)\rho(\delta(e))\delta(e)Q_2(u,e)f(u)^2 + \sum_v\sum_e w(e)\rho(\delta(e))\delta(e)Q_2(v,e)f(v)^2}{2\sum_v f(v)^2\hat{d}(v)} \\
&\leq \inf_{f \perp \hat{\mathbf{D}}_v^{\frac{1}{2}} S_{n-1}} \frac{2\sum_u \hat{d}(u)f^2(u)}{\sum_v \hat{d}(v)f^2(v)} = 2
\end{aligned}$$

14.8 Proof of Corollary 4.2

Corollary 4.2. Let $\mathcal{H}(\mathcal{V}, \mathcal{E}, \mathbf{W}, \mathbf{Q}_1, \mathbf{Q}_2)$ denote the generalized hypergraph in Definition 2. When $\mathcal{H}$ satisfies any of two conditions in Thm. 2, let $\mathbf{L}$ and π be the hypergraph Laplacian matrix and stationary distribution from Corollary 4.1. Let λ_H denote the smallest nonzero eigenvalue of $\mathbf{L}$. Assume an initial distribution $\mathbf{f}$ with $f(i) = 1$ ($f(j) = 0, \forall\, j \neq i$) which means the corresponding walk starts from vertex v_i. Let $\mathbf{p}^{(k)} = \mathbf{f}\mathbf{P}^k$ be the probability distribution after k steps unified random walk where $\mathbf{P}$ denotes the transition matrix, then $p^{(k)}(j)$ denotes the probability of finding the walker in vertex v_j after k steps. We have:

$$\left|p^{(k)}(j) - \pi(j)\right| \leq \sqrt{\frac{\hat{d}(j)}{\hat{d}(i)}}(1 - \lambda_H)^k. \tag{8}$$

Proof: From Eq. (49):

$$\mathbf{P} = \hat{\mathbf{D}}_v^{-1}\mathbf{Q}\mathbf{W}\rho(\mathbf{D}_e)\mathbf{Q}^\top = \hat{\mathbf{D}}_v^{-\frac{1}{2}}(\mathbf{I} - \mathbf{L})\hat{\mathbf{D}}_v^{\frac{1}{2}}$$

Then we have:

$$\mathbf{1}^\top\hat{\mathbf{D}}_v\mathbf{P} = \mathbf{1}^\top\hat{\mathbf{D}}_v$$

Then the stationary distribution $\pi \in \mathbb{R}^{1\times|\mathcal{V}|}$ of $\mathbf{P}$ can be denoted as:

$$\pi = \frac{\mathbf{1}^\top\hat{\mathbf{D}}_v}{c} \tag{51}$$

, where $c = \mathbf{1}^\top\hat{\mathbf{D}}_v\mathbf{1} = \sum_{v\in\mathcal{V}}\hat{d}(v) > 0$ is the sum of $\mathbf{1}^\top\hat{\mathbf{D}}_v$. Let $\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$ denote the eigenvalues of $\mathbf{L}$ in order. Then we assume that $\mathbf{f}\hat{\mathbf{D}}_v^{-\frac{1}{2}} = \sum_{i=1}^{n} a_i\tilde{\mathbf{u}}_i^\top \in \mathbb{R}^{1\times|\mathcal{V}|}$, where $\tilde{\mathbf{u}}_i \in \mathbb{R}^{|\mathcal{V}|\times 1}$ denotes the l_2-norm orthonormal eigenvector related with λ_i. From Thm .5, we know $\tilde{\mathbf{u}}_1 = \frac{\mathbf{u}_1}{\|\mathbf{u}_1\|_2} = \frac{\hat{\mathbf{D}}_v^{\frac{1}{2}}\mathbf{1}}{\sqrt{c}}$. Then a_1 can be computed as:

$$a_1 = \frac{\tilde{\mathbf{u}}_1^\top(\mathbf{f}\hat{\mathbf{D}}_v^{-\frac{1}{2}})^\top}{\tilde{\mathbf{u}}_1^\top\cdot\tilde{\mathbf{u}}_1} = \tilde{\mathbf{u}}_1^\top(\mathbf{f}\hat{\mathbf{D}}_v^{-\frac{1}{2}})^\top = \frac{(\hat{\mathbf{D}}_v^{\frac{1}{2}}\mathbf{1})^\top(\mathbf{f}\hat{\mathbf{D}}_v^{-\frac{1}{2}})^\top}{\sqrt{c}} = \frac{1}{\sqrt{c}}$$

with the fact that $\mathbf{f} \in \mathbb{R}^{1\times|\mathcal{V}|}$ is a probability distribution(i.e. $\mathbf{f}\mathbf{1} = 1$).
Then,

$$\begin{aligned}
|p_j(k) - \pi_j| &= \left|(\mathbf{f}\mathbf{P}^k)_j - \pi_j\right| \\
&= \left|(\mathbf{f}\mathbf{P}^k - \frac{\mathbf{1}^\top\hat{\mathbf{D}}_v}{c})_j\right| = \left|(\mathbf{f}\mathbf{P}^k - a_1\tilde{\mathbf{u}}_1^\top\hat{\mathbf{D}}_v^{\frac{1}{2}})_j\right| \\
&= \left|(\mathbf{f}\hat{\mathbf{D}}_v^{-\frac{1}{2}}(\mathbf{I} - \mathbf{L})^k\hat{\mathbf{D}}_v^{\frac{1}{2}} - a_1\tilde{\mathbf{u}}_1^\top\hat{\mathbf{D}}_v^{\frac{1}{2}})_j\right| \\
&= \left|(\sum_{i=1}^{n} a_i\tilde{\mathbf{u}}_i^\top(\mathbf{I} - \mathbf{L})^k\hat{\mathbf{D}}_v^{\frac{1}{2}} - a_1\tilde{\mathbf{u}}_1^\top\hat{\mathbf{D}}_v^{\frac{1}{2}})_j\right| \\
&= \left|(\sum_{i=1}^{n} a_i\tilde{\mathbf{u}}_i^\top(1 - \lambda_i)^k\hat{\mathbf{D}}_v^{\frac{1}{2}} - a_1\tilde{\mathbf{u}}_1^\top\hat{\mathbf{D}}_v^{\frac{1}{2}})_j\right| \\
&= \left|(\sum_{i=2}^{n} a_i\tilde{\mathbf{u}}_i^\top(1 - \lambda_i)^k\hat{\mathbf{D}}_v^{\frac{1}{2}})_j\right| \\
&\leq (1 - \lambda_H)^k\sqrt{\hat{d}(j)}\left|(\sum_{i=1}^{n} a_i\tilde{\mathbf{u}}_i^\top)_j\right| \\
&= (1 - \lambda_H)^k\sqrt{\hat{d}(j)}\left|(\mathbf{f}\hat{\mathbf{D}}_v^{-1/2})_j\right| \\
&\leq (1 - \lambda_H)^k\frac{\sqrt{\hat{d}(j)}}{\sqrt{\hat{d}(i)}}
\end{aligned}$$

15 GENERALIZED HYPERGRAPH PARTITION

For arbitrary vertex subset $S \subset \mathcal{V}$, S^c denotes the compliment of S. We define the hyperedge boundary ∂S as $\partial S := \{e \in E \mid e \cap S \neq \emptyset, e \cap S^c \neq \emptyset\}$. We will generalize the hypergraph partition problem in [12] to our unified hypergraph framework. Hypergraph partition is to cut vertex set $\mathcal{V}$ into two parts S and S^c. As shown in Theorem 1, we are inspired to adopt the custom of directed graph normalized cut to formulate the hypergraph partition in a mathematical expression. We need to reuse the definition of $\mathrm{vol}(S)$ and $\mathrm{vol}(\partial S)$ for directed graphs as:

$$\mathrm{vol}(S) = \sum_{u \in S} \pi(u), \quad \mathrm{vol}(\partial S) = \sum_{u \in S} \sum_{v \in S^c} \pi(u) P(u, v)$$

The objective function of the hypergraph partition can be expressed as:

$$\underset{\emptyset \neq S \subset \mathcal{V}}{\operatorname{argmin}} \, c(S) := \mathrm{vol}\, \partial S \left(\frac{1}{\mathrm{vol}(S)} + \frac{1}{\mathrm{vol}(S^c)} \right). \tag{52}$$

As we need the explicit form of stationary distribution to formulate this problem, we set the unified random walk on a hypergraph that satisfies with the conditions in Thm. 2. Then we have:

$$\begin{aligned} \mathrm{vol}(S) &= \frac{\sum_{u \in S} \hat{d}(u)}{\mathrm{vol}(\mathcal{V})}, \\ \mathrm{vol}(\partial S) &= \frac{1}{\mathrm{vol}(\mathcal{V})} \sum_{e \in \partial S} \sum_{u \in e \cap S} \sum_{v \in e \cap S^c} w(e) \rho(\delta(e)) Q_2(u, e) Q_2(v, e) \\ &= \frac{1}{\mathrm{vol}(\mathcal{V})} \sum_{e \in \partial S} w(e) \rho(\delta(e)) m(e \cap S) m(e \cap S^c) \end{aligned}$$

where $m(e \cap S) = \sum_{u \in e \cap S} Q_2(u, e)$ is a measure on the subset of V which explicitly demonstrates the difference between [12] (where $m(e \cap S) = \sum_{u \in e \cap S} H(u, e)$) and our work. We consider more fine-grained information on vertices when we establish a measure on an arbitrary subset of $\mathcal{V}$ and involve the degree of hyperedges to impact the corresponding hyperedge weights rather than use the a priori hyperedge weights.

16 DETAILS OF EXPERIMENTS

Note that our H-GNNs are based on the Laplacian led by the equivalent condition in Thm .2, which means the vertex weights we use in our experimental datasets should satisfy the condition (1) or condition (2) in Thm. 2.

16.1 Instructions for Constructing EDVW and EIVW Hypergraphs

The determination of whether a dataset should adopt EIVW (Edge-Independent Vertex Weights) or EDVW (Edge-Dependent Vertex Weights) depends on the nature of the relationships modeled within the data: a) **EIVW-Hypergraphs** are suited for scenarios where all vertices in a hyperedge contribute equally to the edge's characteristics. For example, in citation networks, each paper (vertex) in a hyperedge (e.g., a set of co-cited papers) typically has an equal role without needing a fine-grained weight assignment. b) **EDVW-Hypergraphs**, on the other hand, are appropriate for data where vertices within the same hyperedge contribute differently. For example, in visual object classification, features from different views (e.g., MVCNN, GVCNN) have varying importance, requiring edge-dependent vertex weights to model these differences. In protein analysis, spatial and sequential relationships between amino acids demand varying contributions based on 3D structure proximity or sequence distance.

16.2 Hyper-parameter Strategy

We use grid search strategies to adjust the hyper-parameters of our H-GCN and H-SSGC. The range of hyper-parameters listed in Table 7, Table 8, Table 9, and Table 10.

Extra parameter settings for Visual Object Classification include $k = 6$.

16.3 Baselines of Spectral Convolutions

For HGNN [11], in addition to the flaw during the derivation (i.e. use a specific matrix to fit the scalar parameter θ_0), when doing the Visual Object Classification experiment in their public code, it also takes Gaussian kernel as the incidence matrix for hypergraph learning, which lacks theoretical guarantee. So in our experiment, we just use the incidence matrix $\mathbf{H}$, which only consists of elements $\{0, 1\}$ as the incidence matrix used in HGNN.

HGAT and HNHN we use the same grid search strategy as our methods. For PhenomNN, we reuse the results reported by the paper. For ED-HNN, we reproduce the results based on its public code.

TABLE 7: Hyper-parameter search range for citation network classification and visual object classification.

Methods	Hyper-parameter	Range
H-GNNs	σ	{-2,-1,-0.5,0,0.5,1,2}
	γ(visual object classification)	{0.1, 0.2, 0.4, 0.5, 0.8,1.0}
	Learning rate	{0.001, 0.005, 0.01}
	Hidden dimension	{64,128}
	Layers	{2,4}
	Weight decay	{1e-3,1e-4,5e-4, 1e-5}
	Dropout rate	{0.1,0.2,0.3, 0.4, 0.5}
	Optimizer	Adam
	Epoch	1000
	Early stopping patience	100
	GPU	GTX 2080Ti
H-SSGC	α	{1,0.98,0.95,0.93,0.92,0.9,0.85,0.8}
	β	{1,0.95,0.9,0.85,0.80, 0.75}
	γ(visual object classification)	{0.1, 0.2, 0.4, 0.5, 0.8,1.0}
	Learning rate	{0.001, 0.005, 0.01}
	Hidden dimension	{64,128}
	Layers	{2,4,6,8,16,32,64 }
	Weight decay	{1e-3,1e-4, 1e-5}
	Dropout rate	{0.1,0.2,0.3, 0.4, 0.5}
	Optimizer	Adam
	Epochs	1000
	Early stopping patience	100
	GPU	GTX 2080Ti

16.4 Citation Network Classification

Datasets. The datasets we use for citation network classification include co-authorship and co-citation datasets: PubMed, Citeseer, Cora [103] and DBLP [104]. We adopt the hypergraph version of those datasets directly from [8], where hypergraphs are created on these datasets by assigning each document as a node and each hyperedge represents (a) all documents co-authored by an author in the co-authorship dataset and (b) all documents cited together by a document in co-citation dataset. The initial features of each document (vertex) are represented by bag-of-words features. The details about vertices, hyperedges, and features are shown in Table 11.

Settings and baselines. We adopt the same dataset and train-test splits (10 splits) as provided by in their publically available implementation[1]. Note that this dataset just has the edge-independent vertex weights $\mathbf{H}$, which is also called the incidence matrix. So this experiment can be regarded as a special case of the specific application of our model (i.e. $\mathbf{Q} = \mathbf{H}$).

For baselines MLP+HLR, HNHN [23] , HyperSAGE [73], UniGNN [24], HGNN [11], FastHyperGCN [8], HyperGCN [8], and UniGNN [24] we reuse the results reported by [24]. For PhenomNN, we reuse the results reported by the paper. For HNHN [23], HGAT [10], and ED-HNN, we implement them according to their public code.

We use cross-entropy loss and Adam SGD optimizer with early stopping with a patience of 100 epochs to train GHSC. For other hyperparameters, we use the grid search strategy. More details of hyper-parameters can be found in Table 7.

Results on ED-GNN split. The experimental results on Citation network classification with ED-GNN split can be found in Table 12. The results suggest that our method achieves the best performance among the 7 datasets, revealing that our framework can efficiently utilize the powerful GNNs to address hypergraph-relevant tasks.

1. https://github.com/malllabiisc/HyperGCN, Apache License

TABLE 8: Hyper-parameter search range for H-GNNs and H-SSGC model in fold classification.

Methods	Hyper-parameter	Range	best
H-GNNs	Amino-acids type input size	{32,64,128}	64
	Secondary-structure input size	{32,64,128}	64
	Readout layer input size	{256,512,1024}	1024
	Layers L	{2,3,4,5,6}	4
	Batch size	{64,128,256}	128
	Weight decay	{1e-3,1e-4, 1e-5}	1e-3
	Learning rate	{0.0001,0.001, 0.005, 0.01}	0.0001
	Dropout rate	{0.1,0.2,0.3, 0.4, 0.5}	0.2
	σ	{-1}	-1
	γ	{0.1,0.2,,0.4,0.5,0.8,1.0}	0.1
	Optimizer	Adam	-
	Epochs	300	-
	Early stopping patience	30	-
	GPU	GTX 2080Ti	-
H-SSGC	Amino-acids type input size	{32,64,128}	64
	Secondary-structure input size	{32,64,128}	64
	Readout layer input size	{256,512,1024}	512
	Layers L	{2,4,6,8,12,16,32}	12
	Batch size	{64,128,256}	64
	Weight decay	{1e-3,1e-4, 1e-5}	1e-5
	Learning rate	{0.0001,0.001, 0.005, 0.01}	0.0001
	Dropout rate	{0.1,0.2,0.3, 0.4, 0.5}	0.3
	σ	{-1}	-1
	γ	{0.1,0.2,,0.4,0.5,0.8,1.0}	0.4
	α	{1,0.97,0.95,0.9,0.85,0.8,0.75,0.7,0.65}	0.97
	β	{ 1,0.95,0.90,0.85,0.8 }	0.85
	Optimizer	Adam	-
	Epochs	300	-
	Early stopping patience	30	-
	GPU	GTX 2080Ti	-

TABLE 9: Hyper-parameter search range for H-GNNs and H-SSGC model in protein Quality Assessment (CASP10,CASP11,CASP13 for training, CASP12 for testing).

Methods	Hyper-parameter	Range	best
H-GNNs	Amino-acids type input size	{128,256,512}	512
	Secondary-structure input size	{32,64,128}	64
	Readout layer input size	{256,512,1024}	1024
	Layers L	{2,3,4,5,6}	4
	Batch size	{64,128,256}	64
	Weight decay	{1e-3,1e-4, 1e-5}	1e-5
	Learning rate	{0.0001,0.001, 0.005, 0.01}	0.0001
	Dropout rate	{0.1,0.2,0.3, 0.4, 0.5}	0.5
	σ	{-1}	-1
	γ	{0.1,0.2,,0.4,0.5,0.8,1.0}	0.1
	Optimizer	Adam	-
	Epochs	200	-
	Early stopping patience	20	-
	GPU	Tesla P40	-
H-SSGC	Amino-acids type input size	{128,256,512}	512
	Secondary-structure input size	{32,64,128}	128
	Readout layer input size	{256,512,1024}	256
	Layers L	{2,4,6,8,12,16,18,32}	18
	Batch size	{64,128,256}	128
	Weight decay	{1e-3,1e-4, 1e-5}	1e-5
	Learning rate	{0.0001,0.001, 0.005, 0.01}	0.0005
	Dropout rate	{0.1,0.2,0.3, 0.4, 0.5}	0.3
	σ	{-1}	-1
	γ	{0.1,0.2,,0.4,0.5,0.8,1.0}	0.2
	α	{1,0.97,0.95,0.9,0.85,0.8,0.75,0.7,0.65,0.6}	0.65
	β	{ 1,0.95,0.90,0.85,0.8 }	0.95
	Optimizer	Adam	-
	Epochs	200	-
	Early stopping patience	20	-
	GPU	Tesla P40	-

TABLE 10: Hyper-parameter search range for H-GNNs model in protein Quality Assessment (CASP10-12 for training, and CASP13 for testing).

Methods	Hyper-parameter	Range	best
H-GNNs	Amino-acids type input size	{128,256,512}	128
	Secondary-structure input size	{32,64,128}	64
	Readout layer input size	{256,512,1024}	1024
	Layers L	{2,3,4,5,6}	4
	Batch size	{64,128,256}	1024
	Weight decay	{1e-3,1e-4, 1e-5}	1e-5
	Learning rate	{0.0001,0.001, 0.005, 0.01}	0.001
	Dropout rate	{0.1,0.2,0.3, 0.4, 0.5,0.6,0.7}	0.7
	σ	{-1}	-1
	γ	{0.1,0.2,,0.4,0.5,0.8,1.0}	0.1
	Optimizer	Adam	-
	Epochs	200	-
	Early stopping patience	20	-
	GPU	Tesla P40	-
H-SSGC	Amino-acids type input size	{128,256,512}	128
	Secondary-structure input size	{32,64,128}	64
	Readout layer input size	{256,512,1024}	1024
	Layers L	{2,4,6,8,12,16,18,32}	8
	Batch size	{64,128,256}	1024
	Weight decay	{1e-3,1e-4, 1e-5}	1e-5
	Learning rate	{0.0001,0.001, 0.005, 0.01}	0.001
	Dropout rate	{0.1,0.2,0.3, 0.4, 0.5}	0.2
	σ	{-1}	-1
	γ	{0.1,0.2,,0.4,0.5,0.8,1.0}	0.2
	α	{1,0.97,0.95,0.9,0.85,0.8,0.75,0.7,0.65,0.6}	0.65
	β	{ 1,0.95,0.90,0.85,0.8 }	0.95
	Optimizer	Adam	-
	Epochs	200	-
	Early stopping patience	20	-
	GPU	Tesla P40	-

TABLE 11: Real-world hypergraph datasets used in our citation network classification task.

Dataset	# vertices	# Hyperedges	# Features	# Classes	# isolated vertices
Cora (co-authorship)	2708	1072	1433	7	320(11.8%)
DBLP (co-authorship)	43413	22535	1425	6	0 (0.0%)
Pubmed (co-citation)	19717	7963	500	3	15877 (80.5%)
Cora (co-citation)	2708	1579	1433	7	1274 (47.0%)
Citeseer (co-citation)	3312	1079	3703	6	1854 (55.9%)

TABLE 12: Prediction Accuracy (%). **Bold font** highlights when H-GCNII significantly (difference in means $> 0.5 \times$ std) outperforms all baselines. The best baselines are underlined.

Homophilic Hypergraphs	Cora(Homo.)	Citeseer(Homo.)	Pubmed(Homo.)	Cora-CA(Homo.)	DBLP-CA(Homo.)	Senate(Hete.)	House(Hete.)
HGNN [24]	79.39 ± 1.36	72.45 ± 1.16	86.44 ± 0.44	82.64 ± 1.65	91.03 ± 0.20	48.59 ± 4.52	61.39 ± 2.96
HCHA [105]	79.14 ± 1.02	72.42 ± 1.42	86.41 ± 0.36	82.55 ± 0.97	90.92 ± 0.22	48.62 ± 4.41	61.36 ± 2.53
HNHN [23]	76.36 ± 1.92	72.64 ± 1.57	86.90 ± 0.30	77.19 ± 1.49	86.78 ± 0.29	50.93 ± 6.33	67.80 ± 2.59
HyperGCN [8]	78.45 ± 1.26	71.28 ± 0.82	82.84 ± 8.67	79.48 ± 2.08	89.38 ± 0.25	42.45 ± 3.67	48.32 ± 2.93
UniGCNII [24]	78.81 ± 1.05	73.05 ± 2.21	88.25 ± 0.40	83.60 ± 1.14	91.69 ± 0.19	49.30 ± 4.25	67.25 ± 2.57
HyperND [106]	79.20 ± 1.14	72.62 ± 1.49	86.68 ± 0.43	80.62 ± 1.32	90.35 ± 0.26	52.82 ± 3.20	51.70 ± 3.37
AllDeepSets [25]	76.88 ± 1.80	70.83 ± 1.63	88.75 ± 0.33	81.97 ± 1.50	91.27 ± 0.27	48.17 ± 5.67	67.82 ± 2.40
AllSetTransformer [25]	78.58 ± 1.47	73.08 ± 1.20	88.72 ± 0.37	83.63 ± 1.47	91.53 ± 0.23	51.83 ± 5.22	69.33 ± 2.20
ED-GNN [29]	80.31 ± 1.35	73.70 ± 1.38	89.03 ± 0.53	83.97 ± 1.55	91.90 ± 0.19	64.79 ± 5.14	72.45 ± 2.28
PhenomNN [28]	82.29 ± 1.42	75.10 ± 1.59	88.07 ± 0.48	85.81 ± 0.90	91.91 ± 0.21	-	71.77±1.68
H-GCNII (ours)	**87.96 ± 2.66**	**85.56 ± 3.82**	**92.49 ± 1.71**	**86.62 ± 0.65**	**92.30 ± 0.44**	**83.93± 1.18**	**82.82** ± 9.50

16.5 Visual Object Classification

TABLE 13: summary of the ModelNet40 and NTU datasets

Dataset	ModelNet40	NTU
Objects	12311	2012
MVCNN Feature	4096	4096
GVCNN Feature	2048	2048
Training node	9843	1639
Testing node	2468	373
Classes	40	67

Datasets and Settings. We employ two public benchmarks: Princeton ModelNet40 dataset [74] and the National Taiwan University (NTU) 3D model dataset [75], as shown in Table 13.

In this experiment, each 3D object is represented by the feature vectors, which are extracted by Multi-view Convolutional Neural Network (MVCNN) [76] or Group-View Convolutional Neural Network (GVCNN) [56]. The features generated by different methods can be considered as multimodal features. The hypergraph structure we designed is similar to [45](but they did not give spectral guarantees for supporting the rationality of their practices). We represent the hypergraph structure as an edge-dependent vertex weight $\mathbf{Q}$ to satisfy the condition (2) in Thm. 2 (i.e. $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{Q}$). Specifically, we first generate hyperedges by *k-NN* approach, i.e., each time one object can be selected as a centroid and its k nearest neighbors are used to generate one hyperedge including the centroid itself (in our experiment, we set $k = 10$). Then, given the features of the data, the vertex-weight matrix $\mathbf{Q}$ is defined as

$$\mathbf{Q}(v,e) = \begin{cases} \exp(\frac{-d(v,v_c)}{\gamma \hat{d}^2}), & \text{if } v \in e \\ 0, & \text{otherwise,} \end{cases} \tag{53}$$

where $d(v, v_c)$ is the euclidean distance of features between an object v and the centroid object v_c in the hyperedge and $\hat{d}$ is the average distance between objects. γ is a hyper-parameter to control the flatness. Because we have two-modality features generated by MVCNN and GVCNN, we can obtain the matrix $\mathbf{Q}_{\{i\}}$ which corresponds to the data of the i-th modality ($i \in \{1, 2\}$). After all the hypergraphs from different features have been generated, these matrices $\mathbf{Q}_{\{i\}}$ can be concatenated to build the multi-modality hypergraph matrix $\mathbf{Q} = [\mathbf{Q}_{\{1\}}, \mathbf{Q}_{\{2\}}]$. The features generated by GVCNN or MVCNN can be used singly, or concatenated to a multi-modal feature for constructing the hypergraphs.

For baselines, we compare with the HGNN method for multi-modality learning, following the settings of Appendix 16.3. We also compare our methods using two-modality features to recent SOTA methods on the ModelNet40 dataset. And we use the datasets provided by its public Code [2]. We use cross-entropy loss and Adam SGD optimizer with early stopping with a patience of 100 epochs to train H-GNNs. More details of hyper-parameters can be found in Table 7.

16.6 Protein Quality Assessment and Fold Classification

Protein hypergraph modeling.

At a high level, a protein is a chain of amino acids (residues) that will form a 3D structure by spatial folding. In order to simultaneously consider protein sequence and spatial structure information, we build sequence hyperedges and distance hyperedges. Specifically, given a protein with $|S|$ amino acids, we choose τ consecutive amino acids $(v_i, v_{i+1}, \cdots, v_{i+\tau})$ to connect to form a sequence hyperedge and choose amino acids whose spatial Euclidean distance is less than a threshold $\epsilon > 0$ to connect to form a spatial hyperedge, where $v_i (i = 1, \cdots, |S|)$ represent the i-th amino acid in the sequence. Let $\mathcal{E}_s$ and $\mathcal{E}_e$ denote sequence hyperedge and spatial hyperedge, respectively. Then, we design an edge-dependent vertex-weight matrix $\mathbf{Q}$ for capturing the more granular high-order relationships of proteins below (actually, our models, H-GNNs, allow one to design a more comprehensive $\mathbf{Q}$ for learning proteins better):

$$Q(v,e) = \begin{cases} 1, & \text{if } v \in e \,\text{and}\, e \in \mathcal{E}_s \\ \exp(\frac{-d(v,v_c)}{\gamma \hat{d}^2_{v_c}}), & \text{if } v \in e \,\text{and}\, e \in \mathcal{E}_e \\ 0, & \text{otherwise} \end{cases} \tag{54}$$

where $d(v, v_c) < \epsilon$ is the euclidean distance between an amino acid v and the centroid amino acid v_c in the hyperedge and $\hat{d}_{v_c}$ is the average distance between v_c and the other amino acids $\{v_i\}_{i \neq c}$. γ is a hyperparameter to control the flatness. Here we just design an edge-dependent vertex-weights matrix $\mathbf{Q}$ for satisfying the condition (2) in Thm. 2 (i.e. $\mathbf{Q}_1 = \mathbf{Q}_2 = \mathbf{Q}$). Note that this $\mathbf{Q}$ matrix pays more attention to the information of the sequence (Figure 5).

Experimental settings In our experiment, we set $\tau = 6$ and $\epsilon = 8\mathring{A}$. The initial node features, following [81], are composed of amino acid types and 3D spatial features including dihedral angles, surface accessibility, and secondary structure type generated by DSSP [107]. Note that both Quality Assessment and Fold Classification include graph-level tasks, which means we should add global pooling layers to readout the node representations from H-GNNs.

Here, for the sake of simplicity, we adopt the permutation-invariant operator *mean pooling* and a *single layer MLP* as **Readout** layers to obtain a global hypergraph embedding and add the *softmax* (classification) or *sigmoid* (regression) activation function before output.

2. https://github.com/iMoonLab/HGNN, MIT License

16.6.1 Protein Fold Classification

Datasets and settings. The dataset that we used for training, validation, and testing is the SCOPe 1.75 dataset of [77]. This dataset includes 16,712 proteins covering 7 structural classes with a total of 1195 folds. The 3D structures of proteins are obtained from the SCOpe 1.75 database [108], in which each protein is saved in a PDB file. The datasets have three test sets: 1) Fold, where proteins from the same superfamily do not appear in the training set; 2) Family, in which proteins from the same family are not present in the training set; 3) Family, where proteins from the same family that are present in the training set.

For baselines, we include sequence-based methods pre-trained unsupervised on millions of protein sequences: [19], [78], [79], 3D structure-based model: [17], [20], [81], and [80], who process the sequence with LSTM first and then apply GCNN. The accuracy of the above baselines is reused from [82] reported. We adopt the Adam optimizer to minimize the cross-entropy loss of our methods. The hyperparameter search range can be seen in Table 8.

The experimental results are shown in Table 5.

16.6.2 Protein Quality Assessment (QA).

We add these additional experiments to provide a new task to evaluate the hypergraph algorithms and verify our framework can be used for Protein QA tasks.

Protein QA is used to estimate the quality of computational protein models in terms of divergence from their native structure. It is a regression task aiming to predict how close the decoys are to the unknown, native structure.

Inspired by [81], we train our models on Global Distance Test Score [109], which is the global-level score, and the Local Distance Difference Test [110], an amino-acids-level score. The loss function of QA is defined as the Mean Squared Error (MSE) losses:

$$\mathcal{L}_g = MSE(\mathcal{P}^g_{pred} - \text{GDT\_TS}) \qquad \mathcal{L}_l = \sum_{i=1}^{|S|} MSE(\mathcal{P}^l_{pred_i} - \text{LDDT}_i) \tag{55}$$

where $\mathcal{P}^g_{pred}$ and $\mathcal{P}^l_{pred}$ denote predicted score of global and local respectively.

TABLE 14: summary of CASP datasets

Dataset	Targets	Decoys	Usage
CASP 10	103	26254	Train
CASP 11	85	12563	Train
CASP 12	40	6924	Test
CASP 13	82	12336	Train

TABLE 15: Comparison of our method to others on protein Quality Assessment tasks. At the residue level, We report *Pearson correlation* across all residues of all decoys of all targets (R) and *Pearson correlation* all residues of per decoys and then average all decoys (R_{decoy}) with LDDT scores. At the global level, we report *Pearson correlation* across all decoys of all targets (R) and *Pearson correlation* per target and then average over all targets (R_{target}) with GDT_TS scores.

Test set	Methods	GDT_TS		LDDT	
		R	R_{target}	R	R_{model}
CASP12	VoroMQA	-	0.557	-	-
	RWplus	-	0.313	-	-
	3D CNN	-	0.607	-	-
	AngularQA	0.651	0.439	-	-
	HGNN	0.667	**0.582**	0.632	0.319
	H-GCN (ours)	0.737	**0.609**	0.656	0.340
	H-SSGC (ours)	**0.760**	0.554	**0.678**	**0.449**

Dataset and settings. We use the data from past years' editions of CASP, including CASP10-13. We randomly split the CASP10, CASP11, and CASP13 for training and validation, with the ratio of training: validation = 9:1. CASP 12 is set aside for testing against other methods. More details about the datasets can be found in Table 14. For the baseline, we compare our methods with other start-of-the-art methods, including random walk-based methods : RWplus [111], sequence-based methods: AngularQA [95], and 3D structrue-based methods: VOroMQA [96], 3DCNN [97]. The results of these baselines we reused from [81] reports. Another baseline HGNN [11] is reproduced by us with the same training strategy as our methods.

Because our hypergraph-based methods can jointly learn node and graph embeddings, the losses in Eq. (55) can be weighted as $\mathcal{L}_{total} = \mu\mathcal{L}_l + (1-\mu)\mathcal{L}_g$ and co-optimized by Adam Optimizer with L_2 regularization, where $\mu = 0.5$ in our experiment. We use grid search to select hyper-parameters and more details can be found in Table 9.

Results. Table 15 shows that our methods outperform most of SOTAs, which demonstrates our methods can learn protein more efficiently. The outstanding performance of H-SSGC indicates that it is able to jointly learn better the node and graph level representation. But H-SSGC trained to predict only global scores obtains $R = 0.712$, which suggests that the local information can help the assessment of the global quality.

16.7 Over-Smoothing Analysis

Table 16 reveals that HyperGCN, HGNN, and our H-GCN all suffer from severe over-smoothing issue, limiting the power of neural network to capture high-order relationships.

The results show that H-SSGC and H-GCN inherit the properties of SSGC [35] and GCN [17] when the model layers are deep. H-GCN suffers from the over-smoothing issue, while H-SSGC significantly alleviates the performance descending with the increase of layers. Given the same layers, it can be observed that our H-SSGC almost outperforms the other methods for all cases, especially at a deep layer, which demonstrates the benefits of the deep model and the long-range information around the hypergraph. Furthermore, it should be noted that the optimal layer numbers K of our H-SSGC are generally larger than other models, due to only one linear layer avoiding the over-fitting problem.

We also compare the performance of H-GCNII and ED-GNN in Figure 8 (d). The curves suggest our model, H-GCNII, maintains high accuracy and benefits from deeper architectures. In contrast, ED-GNN's performance declines sharply with depth, a sign of severe oversmoothing.

TABLE 16: Summary of classification accuracy (%) results with various depths. In our H-SSGC, the number of layers is equivalent to K in Eq. (25). We report mean test accuracy over 10 train-test splits.

Dataset	Method	Layers					
		2	4	8	16	32	64
Cora (co-authorship)	HyperGCN	60.66	57.50	31.09	31.10	30.09	31.09
	HGNN	69.23	67.23	60.17	29.28	27.15	26.62
	H-GCN (ours)	**74.79**	72.86	63.99	31.03	30.46	31.09
	H-SSGC (ours)	74.60	**75.78**	**75.70**	**75.04**	**75.26**	**74.79**
DBLP (co-authorship)	HyperGCN	84.82	54.65	22.37	23.96	23.04	24.13
	HGNN	88.55	88.28	85.38	27.64	27.62	27.56
	H-GCN (ours)	**89.04**	**88.90**	85.15	27.61	27.61	27.62
	H-SSGC (ours)	86.63	88.26	**89.00**	**89.17**	**89.05**	**88.60**
Cora (co-citation)	HyperGCN	62.35	58.29	31.09	31.17	31.09	29.68
	HGNN	55.60	55.72	42.10	26.16	24.40	24.43
	H-GCN (ours)	**69.03**	**69.45**	57.37	28.21	26.27	26.95
	H-SSGC (ours)	62.21	64.57	**67.59**	**68.96**	**69.37**	**68.15**
Pubmed (co-citation)	HyperGCN	68.12	63.59	39.99	39.97	40.01	40.02
	HGNN	46.41	47.16	40.93	40.24	40.30	40.29
	H-GCN (ours)	**75.37**	74.76	60.65	40.38	40.31	40.42
	H-SSGC (ours)	74.39	**74.91**	**74.41**	**73.90**	**72.79**	**71.49**
Citeseer (co-citation)	HyperGCN	56.94	36.75	20.72	20.41	20.16	18.95
	HGNN	39.93	38.98	36.67	19.91	19.86	19.79
	H-GCN (ours)	**62.67**	61.50	49.94	21.95	21.84	21.93
	H-SSGC (ours)	61.63	**62.75**	**63.86**	**64.62**	**65.14**	**65.10**

16.8 Robustness Analysis

We evaluate the robustness of our method by simulating three types of perturbations: (a) randomly dropping existing hyperedges, (b) adding heterophilic hyperedges (i.e., connecting nodes with different labels), and (c) masking vertex weights in the edge-dependent incidence matrix. The results are highlighted as follows, **1)** In Figure. 8(a), we observe that performance degrades only slightly as the edge drop ratio increases, indicating that our method remains robust under edge removal. **2)** In Figure. 8(b), while our model is somewhat sensitive when the added heterophilic edge ratio is below 0.3, it becomes more resilient as the ratio increases. In contrast, ED-GNN suffers a steady decline in performance across all addition ratios. **3)** In Figure. 8(c), masking vertex weights leads to performance degradation; however, our method shows less than a 2% accuracy drop, demonstrating stronger robustness than ED-GNN. Interestingly, performance dips the most around a missing ratio of 0.7, possibly due to excessive weight loss causing strong heterophilic interactions that confuse label prediction. Overall, while perturbations do affect model performance, our method demonstrates greater resilience and robustness than ED-GNN.

16.9 Ablation Analysis

In order to verify the effectiveness of our proposed edge-dependent vertex weight $\mathbf{Q}$ and the necessity of the re-normalization trick, we conduct an ablation analysis and report the results in Table 17. Specifically, *w/o* $\mathbf{Q}$ is a variant of our methods that replaces the edge-dependent vertex weight $\mathbf{Q}$ with the edge-independent vertex weight matrix $\mathbf{H}$. *w/o renormalization* represents the variants without the re-normalization trick. Form the reported results we can learn that both $\mathbf{Q}$ and renormalization are efficient for hypergraph learning, respectively. Moreover, on Cora and Pubmed, the performance gap between *w/o renormalization* and *with both* H-GCN reveals the advantage of renormalization on disconnected hypergraph datasets. This phenomenon is mainly caused by the row in the adjacent matrix corresponding to an isolated point is 0 (Figure 6), resulting in a direct loss of its vertex information (Figure 6). The renormalization trick

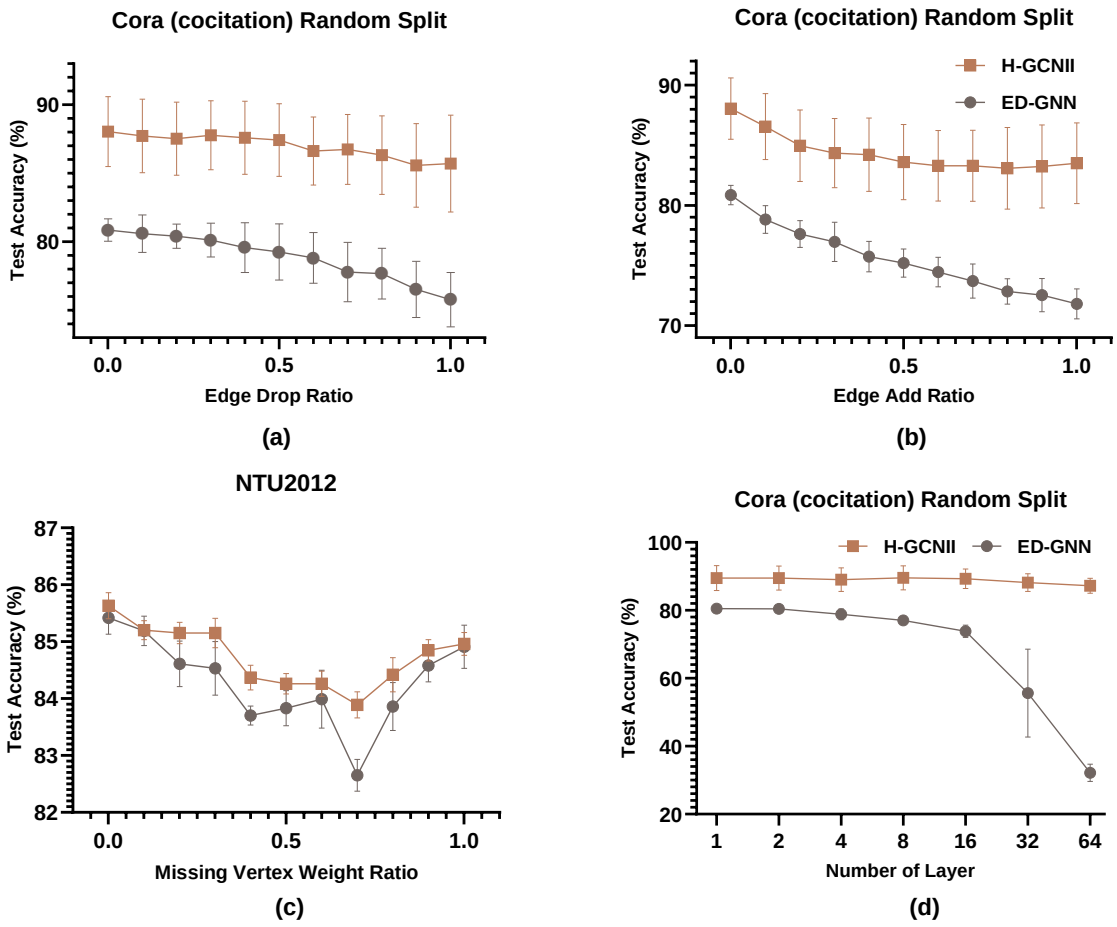


Fig. 8: Robustness and oversmoothing analysis of H-GCNII (ours). We evaluate robustness by varying the ratio of dropped, added, or missing components relative to the original hypergraph and comparing our method with the strong baseline ED-GNN [29]. (a) Randomly dropping original hyperedges at varying ratios. (b) Adding different ratios of heterophilic hyperedges (degree 4). (c) Setting various proportions of vertex weights to zero in EDVW-hypergraphs. (d) Performance of models with increasing numbers of layers—commonly used to analyze oversmoothing in hypergraph neural networks [72].

of adding the self-loop to the matrix $\mathbf{K}$ can maintain the features of isolated vertices during aggregation. Compared with H-GCN, the performance impact of renormalization on H-SSGC seems to be smaller. This is because we add initial features to the information gathered by the neighbors, which reduces the information loss of isolated vertices.

TABLE 17: Test accuracy (%) of our methods for ablation analysis. We report mean ±standard deviation. Cora and Pubmed are the datasets that do not contain $\mathbf{Q}$, so we just report the w/o renormalization results. NTU and ModelNet40 constructed by [11] are both connected hypergraph networks in this work.

methods	-	Cora (co-authorship)	Pubmed (co-citation)	NTU	ModelNet40
GHCN	with both	**74.79±0.91**	**75.37±1.2**	**85.15±0.34**	**97.28±0.15**
	w/o $\mathbf{Q}$	-	-	84.93±0.31	97.18±0.20
	w/o renormalization	69.23±1.6	46.41±0.70	84.85±0.30	97.20±0.15
	w/o both	-	-	84.21±0.25	97.15±0.14
SHSC	with both	**75.91±0.75**	**74.60±1.4**	**83.35±0.30**	**97.74±0.05**
	w/o $\mathbf{Q}$	-	-	82.84±0.40	97.65±0.08
	w/o renormalization	75.76±0.69	73.83±2.1	82.92±0.47	**97.74±0.05**
	w/o both	-	-	82.55±0.46	97.69±0.07

16.10 Sensitivity Analysis of $\rho = (\cdot)^{\sigma}$

We conduct a sensitivity analysis of σ to highlight the importance of the function $\rho(\cdot)$ and offer practical recommendations for hyperparameter selection, as illustrated in Figure. 9. From the figure, we observe that: **1)** Model performance generally peaks when $\sigma \in [-3, -1]$, indicating that the function is most effective when it downweights high-degree hyperedges.

The trend is consistent across diverse datasets, including co-citation, co-authorship, heterophilic hypergraphs (e.g., Senate), and visual object classification tasks. **2)** The superior performance of negative σ values can be explained by the nature of the data. For example, in co-authorship graphs, papers with a large number of authors often involve interdisciplinary collaborations, which may introduce noise rather than strengthen predictive power. A similar pattern holds for co-citation networks and Senate committee data, where high-degree hyperedges tend to represent more generic or less informative groupings. Thus, assigning a lower weight to such hyperedges via a negative σ effectively reduces noise and improves model generalization. **3)** Extremely negative values (e.g. $\sigma = -4$ or $\sigma = -5$) lead to a decline in performance. This is likely because overly penalizing high-degree hyperedges suppresses even informative connections, resulting in underutilization of relevant structural information and reducing the effectiveness of representation learning.

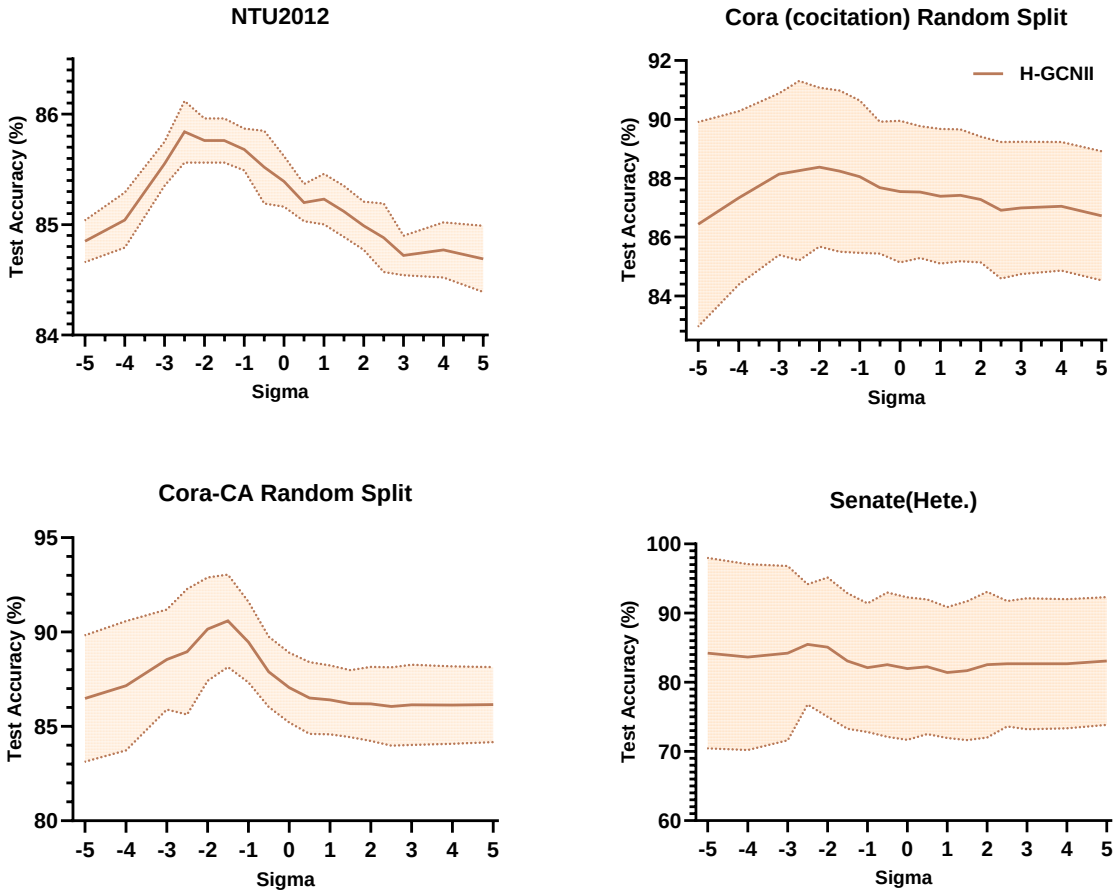


Fig. 9: Sensitivity analysis of $\rho(\cdot) = (\cdot)^\sigma$ in H-GCNII. Test accuracy across datasets with varying σ values, illustrating its impact on model performance.

16.11 Running Time and Computational Complexity

The H-GNNs have the same theoretical computational complexity as the undigraph NNs. For example, the computational cost of H-GCN is $\mathcal{O}(|E|d)$, where $|E|$ is the total edge count in equivalent undigraph. Each sparse matrix multiplication $\tilde{\mathbf{T}}\mathbf{X}$ costs $|E|d$. And the computational of HGNN is the same as H-GCN.

Then, we compare the running time of our H-GNNs with existing models in Table 18, and the results illustrate that our methods are of the same order of magnitude as SOTA's approach UniGNN and outperform the HyperGCN and HGAT.

16.12 Comparison between edge-dependent vertex weights and hypergraph attention

The node-level and edge-level attention in HGAT [10] can be considered as the learnable EDVWs. We visualize EDVWs in Figure 10.

TABLE 18: The average training time per epoch with different methods on citation network classification task is shown below and timings are measured in seconds. The float in parentheses is the standard deviation.

Methods	cora coauthorship	dblp coauthorship	cora cocitation	pubmed cocitation	citeseer cocitation
HyperGCN	0.150±0.058	1.181±0.071	0.151±0.029	1.203±0.104	0.130±0.029
HGNN	0.005±0.002	0.081±0.006	0.005±0.040	0.008±0.002	0.005±0.002
UniGNN	0.014±0.044	0.042±0.040	0.014±0.042	0.023±0.043	0.0168±0.043
HNHN	0.001±0.0026	0.007±0.014	0.0010±0.004	0.009±0.006	0.001±0.003
HGAT	0.381±0.080	OOM	0.279±0.083	1.329±0.016	0.286±0.087
H-ChebNet	0.027±0.005	0.063±0.001	0.073±0.008	0.050±0.019	0.067±0.0198
H-SSGC	0.055±0.001	0.291±0.001	0.205±0.003	0.135±0.056	0.193±0.057
H-GCN	0.005±0.036	0.020±0.079	0.005±0.039	0.011±0.075	0.081±0.091
H-APPNP	0.147±0.013	1.017±0.080	0.033±0.006	0.319±0.037	0.123±0.011
H-GCNII	0.141±0.01	0.252±0.017	0.135±0.013	0.121±0.014	0.079±0.004

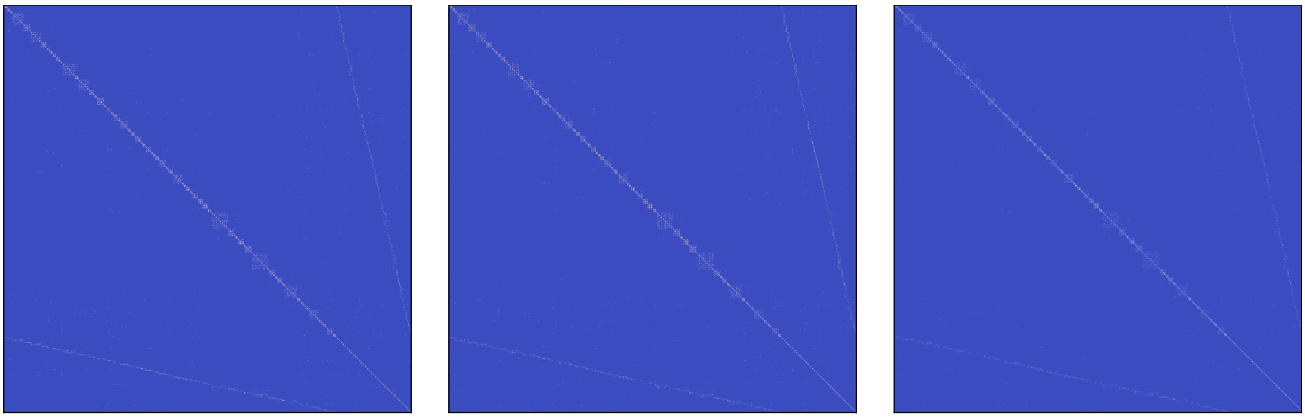


Fig. 10: The edge-dependent vertex weights of NTU2012 dataset (MVCNN feature + MVCNN structure). The figure on the left represents the distance-based vertex weights matrix $\mathbf{Q}$ used in H-GCN. The middle ($\mathbf{Q}_1$) and right ($\mathbf{Q}_2$) denote the node-level and edge-level attention coefficient matrix of HGAT [10], respectively.